



THM:Security101

Section 01 : Starting your CyberSecurity journey

Room 01, 02, 03 - Part of Pre-Security.

Room 04 : Search Skills

This room is designed to help us to improve our information-searching skills in the context of cybersecurity and hacking. It aims to teach you how to :

- Evaluate the reliability of information sources
- Use search engines more effectively
- Explore specialized search engines
- Read and understand technical documentation
- Leverage social media for information
- Verify information from news outlets

SEARCH OPERATORS

- Exact phrase : **Double quotes** indicate that you are looking for pages with the exact word or phrase. **For example : “passive reconnaissance”** .
- Site specific : **“site:” operator** lets you specify the domain name to which you want to limit your search. For example – **site:tryhackme.com success stories**

- The minus : The **minus sign** allows you to **omit search** results that contain a particular word or phrase. For example, we might be interested in learning about the pyramids, but don't want to view tourism websites; one approach is to search for "**pyramids - tourism**".
- File type : This search operator is indispensable for **finding files instead of web pages**. Some of the file types we can search for are PDF, DOC, XLS, PPT etc. Example - **filetype: ppt cyber security**.
- Link to advance search operators for d/f search engines, social media platforms, Operating systems, Databases etc.
<https://github.com/cipher387/Advanced-search-operators-list>

SPECIALIZED SEARCH ENGINES

1. [Shodan](#) – A search engine for devices connected to the internet. It allows you to **search for specific types and versions of servers, networking equipment, industrial control systems, and IoT devices**. You may want to see how many servers are still running Apache 2.4.1 and the distribution across countries. To find the answer, we can search for apache 2.4.1, which will return the list of servers with the string "apache 2.4.1" in their headers.
Shodan Search Query example -
<https://www.shodan.io/search/examples>
2. At first glance, [Censys](#) appears similar to Shodan. However, Shodan focuses on Internet-connected devices and systems, such as servers, routers, webcams, and IoT devices. Censys, on the other hand, focuses on **Internet-connected hosts, websites, certificates, and other Internet assets**. Some of its use cases include **enumerating domains in use, auditing open ports and services, and discovering rogue assets within a network**. Must check - [Censys Search Use Cases](#).

3. [VirusTotal](#) is an online website that provides a **virus-scanning service for files using multiple antivirus engines**. It allows users to upload files or provide URLs to scan them against numerous antivirus engines and website scanners in a single operation. They can even input file hashes to check the results of previously uploaded files.
4. [Have I Been Pwned](#) (HIBP) does one thing; it tells you if an email address has appeared in a leaked data breach.

VULNERABILITIES AND EXPLOITS

1. CVE : We can think of the Common Vulnerabilities and Exposures (CVE) program as a **dictionary of vulnerabilities**. It provides a standardized identifier for vulnerabilities and security issues in software and hardware products. Each vulnerability is assigned a CVE ID with a standardized format like CVE-2024-29988. This unique identifier (CVE ID) ensures that everyone from security researchers to vendors and IT professionals is referring to the same vulnerability, [CVE-2024-29988](#) in this case.

The **MITRE Corporation maintains the CVE system**. For more information and to search for existing CVEs, visit the [CVE Program](#) website. Alternatively, visit the [National Vulnerability Database](#) (NVD) website. The screenshot below shows CVE-2014-0160, also known as Heartbleed.

2. [Exploit Database](#) : The Exploit Database **lists exploit codes** from various authors ; some of these exploit codes are tested and marked as verified.
3. [GitHub](#), a web-based platform for software development, can **contain many tools related to CVEs, along with proof-of-concept (PoC) and exploit codes**.

Product Documentation : Documentation provides an official and reliable source of information about the product features and functions.

Examples include [Snort Official Documentation](#), [Apache HTTP Server Documentation](#), [PHP Documentation](#), and [Node.js Documentation](#).

< Secion 2 is about Linux fundamental rooms ; which we have already seen in Pre-Security >

Section 03 : Windows and AD Fundamentals

Room 1 , 2 and 3 are windows fundamentals ; which is covered in Pre-Security.

Room 04 : Active Directory Basics

In a small business with only a few computers and employees, you could manually manage each computer and account. However, as the business grows and the network becomes more complex, managing individual computers and users manually becomes unfeasible. To handle this efficiently, a **Windows Domain** is used.

Windows Domain:

- **Centralized Management:** Instead of configuring each computer separately, you can use a Windows domain to centralize the management of users, computers, and security policies.
- **Active Directory (AD):** The **centralized repository** that holds all user and computer data, as well as configurations, is **Active Directory (AD)**.
- **Domain Controller (DC):** The server that manages Active Directory and provides authentication and security policies is called a **Domain Controller (DC)**.

Advantages of a Windows Domain:

1. **Centralized Identity Management:** User accounts are managed from a single place, making it easier to apply changes or updates across the entire network.

2. **Managing Security Policies:** You can configure security policies (like password requirements, access restrictions, etc.) and apply them to users and computers across the domain, ensuring consistent security measures.

Real-World Example:

- In schools or universities, a **Windows Domain** allows students or staff to use the same login credentials across multiple machines (Lab PC). When you log in, the machine checks the credentials in **Active Directory** (AD) rather than having the credentials stored on each machine individually.
- **Active Directory** also allows administrators to enforce policies, such as restricting access to system settings (e.g., control panel).

Core of a Windows Domain: Active Directory Domain Service (AD DS)

- **Active Directory (AD)** is the core service of a Windows Domain. It acts as a **catalogue** containing information about the objects in the network (such as users, groups, machines, printers, etc.).

Objects in Active Directory:

1. **Users:**

- Users are considered **security principals**, meaning they can be authenticated and assigned privileges over network resources.
- Users can represent:
 - **People:** Employees who need access to the network.
 - **Services:** Accounts used by services like IIS or MSSQL, with restricted privileges specific to the service they run.

2. **Machines:**

- Each computer that joins the domain has a **machine object**.
- Machine accounts are also security principals with limited rights in the domain.

- Machine accounts are named with the computer's name followed by a dollar sign (e.g., **DC01\$**).

3. **Security Groups:**

- Security groups are used to assign permissions over resources (e.g., files, printers).
- **Security principals** themselves, groups can have privileges over resources.
- Can contain **users**, **machines**, and other **groups**.
- **Important Default Groups:**
 - **Domain Admins:** Full administrative rights across the domain.
 - **Server Operators:** Can administer Domain Controllers but can't modify admin groups.
 - **Backup Operators:** Can access any file, ignoring permissions.
 - **Account Operators:** Can create/modify user accounts.
 - **Domain Users:** Contains all user accounts in the domain.
 - **Domain Computers:** Contains all machine accounts in the domain.
 - **Domain Controllers:** Contains all Domain Controllers in the domain.

4. **Organizational Units (OUs):**

- OUs are containers within Active Directory used to group users, machines, and groups.
- OUs reflect the structure of an organization, such as departments like **Sales** or **IT**.
- Policies can be applied to OUs to manage configurations specific to each group.
- **A user can only be part of one OU at a time.**
- Example: A department such as **Sales** or **IT** can have its own OU to apply tailored policies.

5. AD Users and Computers:

- **Active Directory Users and Computers** is the tool used to configure users, groups, and machines in Active Directory.
- Within the **AD Users and Computers** console, you can see the hierarchy of OUs, groups, and objects.
- By default, Windows creates several important containers like:
 - **Builtin**: Default groups available to any Windows host.
 - **Computers**: Machines joined to the domain.
 - **Domain Controllers**: Contains Domain Controllers.
 - **Users**: Default users and groups.
 - **Managed Service Accounts**: Accounts used by services.

6. Security Groups vs OUs:

- **OUs**: Used for **applying policies** to sets of users or computers (e.g., security policies, configurations).
 - A user can only be in one OU at a time.
- **Security Groups**: Used to **grant permissions** to network resources (e.g., shared folders, printers).
 - A user can be part of **multiple security groups**.

One of the nice things you can do in AD is to give specific users some control over some OUs. This process is known as **delegation** and allows you to grant users specific privileges to perform advanced tasks on OUs without needing a Domain Administrator to step in.

1. OU Deletion Protection:

- **Default setting**: OUs are **protected against accidental deletion**.
- **Action required to delete an OU**:
 - Enable **Advanced Features** in the **View** menu.

- Uncheck the "**Protect object from accidental deletion**" box in the **Properties** of the OU.

2. **Organizational Units (OUs):**

- OUs represent **departments or groups** in Active Directory.
- They are used to **organize users, groups, and other AD objects**.
- When a department is closed, its **OU and all objects within it** must be **deleted**.

3. **Delegating Control:**

- **Delegation:** Granting specific permissions (like resetting passwords) to certain users for particular OUs.
- **Example: Phillip** (IT support) can be delegated the task of resetting passwords for users in specific OUs like **Sales**.
- **Steps for Delegation:**
 1. Right-click on the OU (e.g., **Sales OU**).
 2. Select **Delegate Control**.
 3. Choose the user (e.g., **Phillip**) and the permissions (e.g., **reset passwords**).

4. **PowerShell for AD Management:**

- Use **PowerShell** commands to perform tasks like **resetting user passwords**:

- Example Command to reset password:

```
Set-ADAccountPassword sophie -Reset -NewPassword (Read-Host  
-AsSecureString -Prompt 'New Password') -Verbose
```

- To force a user to change their password at the next login :

```
Set-ADUser -ChangePasswordAtLogon $true -Identity sophie -Verbose
```

5. Testing Delegated Permissions:

- After delegating control (e.g., password reset), **test the permissions**:
 - Log in as the delegated user (e.g., **Phillip**) and verify that the task (e.g., resetting a user's password) can be performed.

6. User/Group Management:

- **Users** in AD are typically organized within OUs.
- When the organizational chart changes, it may be necessary to **add, modify, or delete users** to reflect the new structure.

1. Default OU for Computers:

- By default, when machines join the domain, they are placed in the "**Computers**" container in AD.
- This includes all machines (servers, laptops, PCs) unless specifically moved to another OU.

2. Organizing Devices into OUs:

- **Why organize devices?** To apply different **Group Policy Objects (GPOs)** and settings for different types of devices (e.g., workstations vs. servers).
- **Common Device Categories in AD:**
 - **Workstations:** Devices used by regular users for daily activities (browsing, email, etc.).
 - **Servers:** Devices providing services to users or other servers.
 - **Domain Controllers (DCs):** Devices managing the Active Directory Domain; very sensitive because they store hashed passwords of all domain users.

1. Group Policy Objects (GPOs):

- **GPOs** are collections of **settings** that can be applied to **users** or **computers** in Active Directory.
- **GPOs** allow administrators to **enforce policies** like security baselines or configurations across different **OUs**.

2. Group Policy Management:

- Use the **Group Policy Management tool** to manage GPOs.
- To apply a policy, you **create** a GPO and then **link it to the relevant OU**.
- A GPO can apply to an **OU** and all **sub-OUs** under it.

3. GPO Settings:

- **User Configuration:** Policies that affect users (e.g., access control to specific system settings).
- **Computer Configuration:** Policies that affect computers (e.g., security settings, screen locking).

4. GPO Scope and Security Filtering:

- **Scope:** Defines where a GPO is applied (e.g., a specific OU or domain).
- **Security Filtering:** Restricts the GPO to specific **users** or **computers** under an OU. By default, GPOs apply to the **Authenticated Users** group.

5. GPO Distribution:

- GPOs are distributed over a **network share called SYSVOL**.
Location: C:\Windows\SYSVOL\sysvol\ on Domain Controllers.
- Changes to GPOs may take up to **2 hours** to propagate across the domain.
- To force immediate GPO updates, use the command : `gpupdate /force`

6. Creating and Linking GPOs:

- **Example 1: Restrict Control Panel Access GPO:**

- Created to restrict non-IT users from accessing the Control Panel.
- Linked to Sales, Marketing, and Management OUs.
- Policy: Prohibit Access to Control Panel under User Configuration.

- **Example 2: Auto Lock Screen GPO:**

- Created to lock workstations and servers after 5 minutes of inactivity.
- Linked to the root domain so it affects all computers.
- Policy: Interactive Logon: Machine inactivity limit under Computer Configuration.

Authentication Protocols in Windows Domains:

1. Kerberos Authentication:

- Default protocol for recent versions of Windows.
- Uses tickets for authentication:
 - Ticket Granting Ticket (TGT) for general authentication.
 - Ticket Granting Service (TGS) for specific service access.
- Steps for Kerberos Authentication:
 - User sends encrypted timestamp and username to KDC.
 - KDC returns a TGT and Session Key.
 - User sends TGT and Session Key to request a TGS for a service.
 - KDC sends the TGS and Service Session Key.
 - User sends TGS to the service for authentication and connection.

- *Security: The TGT is encrypted using the KDC's password hash, and the Session Key is part of the TGT content.*

2. NetNTLM Authentication (Legacy protocol for compatibility):

- *Uses a challenge-response mechanism:*
 - *Client sends authentication request.*
 - *Server generates and sends a challenge.*
 - *Client generates a response using its NTLM password hash and sends it back.*
 - *Server forwards the response to the Domain Controller.*
 - *Domain Controller verifies the response by recalculating it using the challenge.*
 - *Authentication result is returned to the client via the server.*
- *Security: The user's password (or hash) is never transmitted over the network.*

Single Domain vs. Multiple Domains :

- *Single Domain: Suitable for smaller companies or early stages, but may not scale well as the company grows and faces additional requirements.*
- *Multiple Domains: Needed when there's a need for better management, legal/regulatory differences, or organizational changes, such as expanding to a new country.*

Trees :

- *A Tree is a group of Windows domains that share the same namespace.*

- *Example: If the company expands, you might split thm.local into uk.thm.local and us.thm.local, each with its own domain controller (DC).*
- *This provides better management as each IT team manages their respective domain, but the domains still share a common namespace.*
- *Each domain in the tree is independent but can have shared resources through trust relationships.*

Enterprise Admins :

- *A special security group with administrative privileges over all domains in the enterprise, unlike Domain Admins who only control a single domain.*

Forests :

- *A Forest is a collection of multiple domain trees.*
 - *Example: When two companies merge, they may have different domain namespaces. The union of these different trees creates a forest.*
 - *A forest allows better compartmentalization of networks and resources for independent management by different IT teams.*

Trust Relationships :

- *Trusts allow users in one domain to access resources in another domain.*
 - *One-Way Trust: Domain A trusts Domain B, so users in Domain B can access resources in Domain A, but not vice versa.*
 - *Two-Way Trust: Both domains trust each other, allowing mutual access to resources.*
 - *Trust relationships enable resource sharing, but authorization must still be explicitly configured.*

Section 04 : Command Line

Room 01 : Windows command line

- Difference between a GUI and a CLI interface

Feature	GUI (Graphical User Interface)	CLI (Command Line Interface)
Interface Type	Visual interface using windows, icons, menus	Text-based interface using commands
Ease of Use	Pro: Easy for beginners Con: Limited advanced control	Pro: Powerful for experts Con: Difficult to learn
Speed	Pro: Good for simple tasks Con: Slower for repetitive/complex tasks	Pro: Very fast once commands are known Con: Slow for new users
Resource Usage	Pro: Rich visual experience Con: High memory and CPU usage	Pro: Low system resource usage Con: No visual feedback
User Interaction	Pro: Mouse + keyboard interaction Con: Mouse dependency can reduce efficiency	Pro: Keyboard-only, efficient Con: No mouse support
Automation	Pro: Simple basic operations Con: Hard to automate tasks	Pro: Easy scripting and automation Con: Requires scripting knowledge
Error Handling	Pro: Visual cues reduce errors Con: Less detailed error info	Pro: Detailed error messages Con: Small mistakes cause errors
Examples	Windows, macOS, Android UI	Linux Terminal, Command Prompt

- Windows executes commands only from locations listed in the Path environment variable.
Use command “set” to view all environment variables and check the Path entry.
- The “ver” command is used to identify the Windows operating system version.
- The “systeminfo” command provides detailed system information, including OS details, host name, system type, processor, and memory.
- For commands that produce long output, use piping with more (e.g., driverquery | more) to display results one page at a time.
Press Space to continue and Ctrl + C to exit.
- help – displays help for commands
- cls – clears the command prompt screen

Windows provides several *command-line networking tools* to view network configuration, monitor connections, and troubleshoot issues without using the GUI.

Network Configuration

- `ipconfig` displays basic network information such as:
 - IPv4 address
 - Subnet mask
 - Default gateway
- `ipconfig /all` shows detailed network configuration, including:
 - MAC (physical) address
 - DHCP status and server
 - DNS servers
 - Lease information
 - Adapter details

Network Troubleshooting

- `ping target_name` checks connectivity to another host by sending ICMP packets.
 - Successful replies confirm the target is reachable.
 - Output includes packet loss and round-trip time.
- `tracert target_name` traces the route packets take to reach a destination.
 - Displays each router (hop) along the path.
 - Helps identify where network delays or failures occur.

Name Resolution

- `nslookup domain_name` resolves a domain name to its IP address.
 - Uses the default DNS server by default.
 - You can specify a DNS server (e.g., `nslookup example.com 1.1.1.1`) to compare results.

Network Connections

- `netstat` displays active network connections and listening ports.
 - Helps identify services and open connections (e.g., SSH on port 22).
- Common `netstat` options:
 - `-a` → shows all connections and listening ports
 - `-b` → shows the program using each connection
 - `-o` → displays the process ID (PID)
 - `-n` → shows numerical addresses and ports
- `netstat -abon` combines these options to show:
 - Listening services, Associated executables & Process IDs for each connection.

Working with Directories

- `cd` (without parameters) displays the current drive and directory, similar to asking “where am I?”
- `dir` lists files and subdirectories in the current directory.
 - `dir /a` → shows hidden and system files
 - `dir /s` → shows files in the current directory and all subdirectories
- `tree` displays a visual directory structure, showing folders and subfolders.
- `cd directory_name` changes to a specified directory.
- `cd ..` moves one level up in the directory hierarchy.
- `mkdir directory_name` creates a new directory.
- `rmdir directory_name` deletes an empty directory.

Working with Files

- `type filename` displays the contents of a text file in the terminal. For long files, use `more` to view content page by page.
- `copy source destination` copies a file from one location to another.
- `move source destination` moves a file to a new location (also used to rename files).
- `del` or `erase` deletes files.
- The wildcard `*` represents multiple files.
 - Example: `copy *.md C:\Markdown` copies all `.md` files to the specified directory.

Managing Processes in Windows Command Line

Listing Running Processes

- `tasklist` displays all running processes along with their PID (Process ID), memory usage, and session information.
- To filter the process list, use `/FI` followed by the filter condition. For example:
 - `tasklist /FI "imagename eq sshd.exe"` lists only processes related to `sshd.exe`.

Terminating Processes

- To terminate a process, use `taskkill` with the PID of the target process:
 - `taskkill /PID target_pid`
 - Example: `taskkill /PID 4567` will kill the process with PID 4567.

Helpful Tips

- To view more `tasklist` filters, use:
 - `tasklist /?` to get the full help page.

Some more commands :

- `chkdsk` : checks the file system and disk volumes for errors and bad sectors.
- `Driverquery` : displays a list of installed device drivers.
- `sfc /scannow` : scans system files for corruption and repairs them if possible.

It is important to know that `/?` can be used with most commands to display a help page.

In this room, we used the command `more` in two ways:

- Display text files : `more file.txt`
- Pipe long output to view it page by page : `some_command | more`

Equipped with this knowledge, we now know how to display the help page of a new command and how to display long output one page at a time.

Room 02 : Windows Power shell

PowerShell — Overview

- PowerShell is a task automation and configuration management tool developed by Microsoft.
- It is made up of:
 - A command-line shell
 - A scripting language
 - A configuration management framework
- Designed to help IT professionals and system administrators automate repetitive tasks and manage systems efficiently.

Key Characteristics of PowerShell

- Built on .NET
 - Gives PowerShell access to rich system APIs and modern Windows components.
- Object-oriented (not text-based)
 - Commands return objects, not plain text.
- Cross-platform
 - Works on Windows, macOS, and Linux (via PowerShell Core).
- More powerful than traditional shells
 - Can handle complex data types and system interactions easily.

Why PowerShell Was Needed

- Early Windows automation relied on:
 - `cmd.exe`
 - Batch files
- These tools were:
 - Text-based
 - Limited in functionality
 - Poor at handling complex enterprise environments
- As Windows grew in enterprise use, Microsoft needed:
 - Better automation
 - Deeper system integration
 - A modern scripting solution

Origin and History of PowerShell

- Early 2000s
 - Enterprise Windows environments became more complex.
 - Existing tools could not manage modern APIs effectively.
- Jeffrey Snover (Microsoft engineer) identified a core issue:
 - Unix systems treat everything as text.
 - Windows systems rely on structured data and APIs.
- Porting Unix-style tools to Windows didn't work well because of this difference.

Snover's Solution - Man who created power shell

- Introduced an object-oriented command-line approach
- Combined:
 - Simple scripting
 - The power of the .NET framework
- Result:
 - Commands manipulate objects directly
 - No need for complex text parsing

Release Timeline

- 2006 – PowerShell released
 - Initially Windows-only
 - Deep integration with Windows components
- 2016 – PowerShell Core released
 - Open-source
 - Cross-platform (Windows, macOS, Linux)
- This made PowerShell a universal automation tool.

Understanding Objects in PowerShell

What is an Object?

- An object is a combination of:
 - Properties → data / characteristics
 - Methods → actions it can perform

Example (Conceptual):

- Car Object:
 - Properties: Color, Model, FuelLevel
 - Methods: Drive(), HonkHorn(), Refuel()

Objects in PowerShell

- PowerShell objects can represent:
 - Files
 - Processes
 - Users
 - Services
- Objects contain:
 - Properties → file size, name, status
 - Methods → copy, stop, delete, start

Cmdlets in PowerShell

- PowerShell commands are called cmdlets (*pronounced command-lets*)
- Cmdlets:
 - Return objects, not text
 - Follow a Verb-Noun format (e.g., Get-Process)
- This allows:
 - Easy data filtering
 - Sorting
 - Pipelining between commands

PowerShell vs Traditional Command Shell

Feature	Traditional Shell (cmd.exe)	PowerShell
Output	Plain text	Objects
Data handling	Text parsing required	Direct object manipulation
Complexity support	Limited	High
Automation capability	Basic	Advanced
System integration	Minimal	Deep (.NET & APIs)

Why Object-Based Output Matters

- No need to:
 - Parse text manually
 - Use complex string operations
- You can:
 - Access object properties directly
 - Chain commands smoothly
- Makes scripts:
 - Cleaner
 - More readable
 - More reliable

Launching PowerShell

- PowerShell can be launched in various ways:
 1. Start Menu: Type powershell, click on the result.
 2. Run Dialog: Press Win + R, type powershell, press Enter.
 3. File Explorer: Navigate to any folder, type powershell in the address bar, and press Enter.
 4. Task Manager: Open Task Manager, go to *File > Run new task*, type powershell, press Enter.
 5. Command Prompt: Type powershell and press Enter.

PowerShell Syntax: Verb-Noun

- Cmdlets (commands) follow a Verb-Noun pattern:
 - Verb = action (e.g., Get, Set, Add)
 - Noun = target object (e.g., Content, Location, Package)
- Examples:
 - Get-Content: Retrieves content of a file.
 - Set-Location: Changes the current directory.

Basic Cmdlets

- Get-Command: Lists all available cmdlets and functions in the current session.

```
PS C:\Users\captain> Get-Command
```

- Filter by Command Type:
 - To list only functions : *PS C:\Users\captain> Get-Command -CommandType "Function"*
- Get-Help: Shows detailed information about cmdlets.
Example : *PS C:\Users\captain> Get-Help Get-Date*

Cmdlet Aliases

- PowerShell includes aliases (shortcuts) for traditional commands:
 - `dir` → `Get-ChildItem`
 - `cd` → `Set-Location`
- To list all aliases : *PS C:\Users\captain> Get-Alias*

Searching for Cmdlets Online

- Find-Module: Search for modules (collections of cmdlets) in online repositories (e.g., PowerShell Gallery).
 - Example : *PS C:\Users\captain> Find-Module -Name "PowerShell"*
- Install-Module: Download and install a module from an online repository.
 - Example : *PS C:\Users\captain> Install-Module -Name "PowerShellGet"*

PowerShell File System Cmdlets

1. **Get-ChildItem** (Equivalent to *dir* or *ls* in other systems)

- Lists files and directories in a specified location.
- Without specifying -Path, it shows the contents of the current working directory.

Example : `PS C:\Users\captain> Get-ChildItem`

Output example :

Mode	LastWriteTime	Length	Name
----	-----	-----	----
d-r---	5/8/2021 9:15 AM		Desktop
d-r---	9/4/2024 10:58 AM		Documents

2. **Set-Location** (Equivalent to *cd* in Command Prompt)

- Changes the current directory to a specified path.

Example:

```
PS C:\Users\captain> Set-Location -Path ".\Documents"
```

```
PS C:\Users\captain\Documents>
```

3. **New-Item** (Creates files and directories)

- Creates new items (files or directories) by specifying the path and type.

Examples:

- Create a **directory**:

```
PS C:\Users\captain\Documents> New-Item -Path ".\captain-cabin\captain-wardrobe" -ItemType "Directory"
```

- Create a **file**:

```
PS C:\Users\captain\Documents> New-Item -Path ".  
  \captain-cabin\captain-wardrobe\captain-boots.txt"  
  -ItemType "File"
```

4. **Remove-Item** (Equivalent to *rmdir* and *del* in Command Prompt)

- Deletes files or directories.

Examples:

- Remove a **file**:

```
PS C:\Users\captain\Documents> Remove-Item -Path ".  
  \captain-cabin\captain-wardrobe\captain-boots.txt"
```

- Remove a **directory**:

```
PS C:\Users\captain\Documents> Remove-Item -Path ".  
  \captain-cabin\captain-wardrobe"
```

5. **Copy-Item** (Equivalent to *copy* in Command Prompt)

- Copies files or directories to a new location.

Example:

```
PS C:\Users\captain\Documents> Copy-Item -Path .\captain-  
  cabin\captain-hat.txt -Destination .\captain-cabin\captain-  
  hat2.txt
```

6. **Move-Item** (Equivalent to *move* in Command Prompt)

- Moves files or directories to a new location.

7. **Get-Content** (Equivalent to *type* or *cat* in Unix-like systems)

- Reads and displays the contents of a file.

Example : PS C:\Users\captain\Documents\captain-cabin> Get-Content -Path ".\captain-hat.txt"

Summary of Cmdlet Equivalents

PowerShell Cmdlet	Traditional CLI Command
Get-ChildItem	dir, ls
Set-Location	cd
New-Item	mkdir, echo (for files)
Remove-Item	rmdir, del
Copy-Item	copy
Move-Item	move
Get-Content	type, cat

Piping

- Piping allows the output of one command to become the input of another.
- Represented by the pipe symbol (|).

- Common in:
 - Windows Command Prompt
 - Unix/Linux shells
 - PowerShell (more advanced)

Key Difference in PowerShell

Traditional CLI	PowerShell
Passes plain text	Passes objects
Limited manipulation	Rich data with properties & methods

PowerShell Objects

- Commands return objects, not just text.
- Objects contain:
 - Data (e.g., file name, size)
 - Properties (Length, Name, Extension, etc.)
 - Methods (actions that can be performed)

This enables powerful filtering, sorting, and analysis.

Get-ChildItem (Listing Files)

- Retrieves files and directories as objects.

Example : *Get-ChildItem*

Sorting Objects with Sort-Object

- Sorts objects based on a property.

Example: Sort files by size

```
Get-ChildItem | Sort-Object Length
```

Explanation:

- Get-ChildItem → gets file objects
- | → passes objects
- Sort-Object Length → sorts by file size

Filtering Objects with Where-Object

- Filters objects based on conditions.
- Only objects that meet the condition are returned.

Example: Show only .txt files

```
Get-ChildItem | Where-Object -Property Extension -eq ".txt"
```

Common Comparison Operators

Operator	Meaning	Description
-eq	Equal to	Matches exact value
-ne	Not equal	Excludes matching value
-gt	Greater than	Strictly greater
-ge	Greater than or equal	Non-strict
-lt	Less than	Strictly less
-le	Less than or equal	Non-strict
-like	Pattern match	Uses wildcards (*)

Filtering Using Patterns (-like)

Example: Files starting with “ship”

```
Get-ChildItem | Where-Object -Property Name -like "ship*"
```

- * acts as a wildcard
- Matches any text after “ship”

Selecting Specific Properties with Select-Object

- Used to:
 - Display only required properties
 - Limit output details

Example : Show only file name and size

```
Get-ChildItem | Select-Object Name, Length
```

Chaining Multiple Cmdlets

- Pipelines can include multiple cmdlets.
- Example use cases:
 - Sort files
 - Filter by size
 - Display only selected properties

Typical Pipeline Flow

Command → Filter → Sort → Select

Searching File Content with Select-String

- Searches for text inside files.
- Similar to:

- grep (Linux)
- findstr (Windows CMD)

Example: Search for a word in a file

```
Select-String -Path ".\captain-hat.txt" -Pattern "hat"
```

Output shows :

- File name
- Line number
- Matching text

Regular Expression (Regex) Support

- *Select-String* fully supports regex.
- Enables:
 - Advanced pattern matching
 - Complex searches in logs or documents

Summary Table

Cmdlet	Purpose
Get-ChildItem	List files and folders
Sort-Object	Sort objects by property
Where-Object	Filter objects by condition
Select-Object	Choose properties / limit output
Select-String	Search text inside files

PowerShell Cmdlets for System and Network Information

PS C:\Users\captain> *Get-ComputerInfo Cmdlet*

- Purpose: Retrieves detailed system information.
- Includes:
 - Operating system details (e.g., version, build)
 - Hardware specs (e.g., RAM, processor)
 - BIOS information
 - Windows product info
- Comparison to systeminfo: Get-ComputerInfo provides more comprehensive data in a single command.

PS C:\Users\captain> *Get-LocalUser Cmdlet*

- Purpose: Lists all local user accounts on the system.
- Includes:
 - Username
 - Account status (Enabled/Disabled)
 - Account description

PS C:\Users\captain> *Get-NetIPConfiguration Cmdlet*

- Purpose: Retrieves network interface details.
- Includes:
 - Interface alias (e.g., Ethernet)

- IPv4/IPv6 addresses
- DNS servers
- Gateway configuration

PS C:\Users\captain> *Get-NetIPAddress Cmdlet*

- Purpose: Displays all IP addresses assigned to network interfaces, including inactive ones.
- Includes:
 - IP Address (both IPv4 and IPv6)
 - Interface Index and Alias
 - Address family (IPv4/IPv6)
 - Address state (e.g., Preferred)

PowerShell Cmdlets for Advanced System Monitoring and Analysis

PS C:\Users\captain> *Get-Process*

- Purpose: Displays all currently running processes and their resource usage.
- Includes:
 - Handles, NPM (Non-Paged Memory), PM (Paged Memory), WS (Working Set), CPU time, Process ID (PID), and Process Name.
 - Useful for performance monitoring and troubleshooting.

PS C:\Users\captain> *Get-Service*

- Purpose: Retrieves information about the status of services on the machine.
- Includes:

- Service name, display name, and status (Running, Stopped, Paused).
- Helps in managing and troubleshooting services, especially for system admins and forensics analysts.

PS C:\Users\captain> *Get-NetTCPConnection*

- Purpose: Displays active TCP network connections.
- Includes:
 - Local/remote IP addresses, local/remote ports, connection state (e.g., Listening, Established, TimeWait), and owning process ID.
 - Essential for incident response, malware analysis, and detecting suspicious connections.

PS C:\Users\captain> *Get-FileHash*

- Purpose: Generates a hash for a file to verify its integrity.
- Uses:
 - Crucial for incident response and threat hunting to detect file tampering or alterations.

PS C:\Users\captain> *Get-Item -Stream* (Alternate Data Streams - ADS)

- Purpose: Views alternate data streams attached to a file.
- Uses:
 - Helps identify hidden or malicious data streams attached to files in NTFS file systems.

PowerShell Scripting in Cyber Security

What is Scripting?

- Definition: Writing and executing a series of commands in a text file (script) to automate tasks that are manually performed in a shell (e.g., PowerShell).
- Purpose:
 - Saves time.
 - Reduces errors.
 - Automates complex or tedious tasks.
- Analogy: Scripting is like giving a computer a to-do list, where each line represents a task to be performed automatically.

Why is PowerShell Scripting Important in Cyber Security?

- Blue Team (Defenders):
 - Incident Responders, Malware Analysts, Threat Hunters:
 - Automate log analysis, anomaly detection, IOC extraction.
 - Reverse-engineer malicious code (malware).
 - Automate intrusion detection on systems.
- Red Team (Attackers):
 - Penetration Testers, Ethical Hackers:
 - Automate system enumeration and remote command execution.
 - Craft obfuscated scripts to bypass security defenses.
 - Simulate attacks and test system resilience.
- System Administrators:
 - Automate integrity checks and system configuration management.

- Enforce security policies and monitor system health.
- Automate responses to security incidents in large-scale environments.
- Key Insight: PowerShell scripting is a powerful tool for both offensive (red team) and defensive (blue team) cyber security roles, making it an essential skill in the field.

Invoke-Command Cmdlet

- Purpose: Executes commands on local or remote systems, critical for automation and remote management.
- Uses:
 - System Administrators: Remotely manage and automate tasks across multiple machines.
 - Security Engineers: Execute commands on remote systems for troubleshooting or monitoring.
 - Penetration Testers: Run commands or payloads on target systems during an engagement.
- Example Commands:
 - Run a script on a remote server:
`Invoke-Command -FilePath c:\scripts\test.ps1 -ComputerName Server01`
 - Runs a script on Server01 and returns results to the local computer.
 - Run a single command on a remote server:
`Invoke-Command -ComputerName Server01 -Credential Domain01\User01 -ScriptBlock { Get-Culture }`
 - Executes Get-Culture on Server01 using the credentials of Domain01\User01.

Power and Simplicity of Invoke-Command

- Easy Automation: Combine Invoke-Command with custom scripts to automate tasks across multiple remote computers.
- No Scripting Required: Even without complex scripting, you can execute commands remotely by simply using the -ScriptBlock parameter. This allows running commands as if you were on the remote system itself.

Room 03 : Linux Shells

Shell

- *A shell is a command-line interface used to interact with the operating system.*
- Most Linux systems use Bash (Bourne Again Shell) by default.
- The shell lets you run commands to navigate files, view content, and perform tasks.

Current Working Directory

- When you open the shell, you usually start in your home directory.
- To check where you are, use : `pwd`
- Example output : `/home/user`

Changing Directories

- Use the `cd` command to move between directories.
`cd Desktop`
- This changes your current location to the Desktop folder.

Listing Files and Folders

- To see the contents of the current directory, use : `ls`
- Example output :
Desktop Documents Downloads Music Pictures Videos

Viewing File Contents

- To read what's inside a file, use : `cat filename.txt`
- This displays the entire contents of the file in the terminal.

Searching Text with grep

- `grep` is used to search for specific words or patterns inside files.
- Very useful for large files.

Example : `grep THM dictionary.txt`

- This command finds and *displays lines* that contain the word THM.

 Linux provides multiple types of shells, each with different features. Similar to Command Prompt and PowerShell in Windows.

Check Current Shell

To see which shell you are currently using : `echo $SHELL`

Example Output : `/bin/bash`

List Available Shells

- Installed shells are listed in the file `/etc/shells`.

`cat /etc/shells`

Example output includes:

- bash
- sh
- dash
- zsh
- fish (if installed)

Switch Between Shells

- To switch to another shell, simply type its name : `zsh`
- The prompt changes, indicating a new shell is active.

Change Default Shell (Permanent)

To change your default shell : `chsh -s /usr/bin/zsh`

- This makes the selected shell the default for future terminal sessions.

Common Linux Shells & Features

Feature	Bash	Fish	Zsh
Full Name	Bourne Again Shell	Friendly Interactive Shell	Z Shell
Default Shell	Default shell in most Linux distributions	Not default in most distributions	Not installed by default in most distributions
Primary Focus	Stability, compatibility, and scripting	User-friendliness and ease of use	Modern features and high customization
Scripting Support	Widely used for scripting with extensive documentation and community support	Limited scripting features compared to Bash and Zsh	Excellent scripting support, combining Bash features with additional enhancements
Syntax Simplicity	Traditional syntax, may feel complex for beginners	Very simple and beginner-friendly syntax	Similar to Bash but extended with more advanced features
Tab Completion	Basic tab completion for commands and filenames	Advanced tab completion with intelligent suggestions based on previous commands	Advanced tab completion, extendable using plugins
Command History	Maintains a history file; supports arrow keys and history command	Maintains command history with interactive suggestions	Maintains command history with powerful search features
Auto Spell Correction	Not available	Available (auto-corrects commands)	Available

Feature	Bash	Fish	Zsh
Syntax Highlighting	Not available	Built-in syntax highlighting	Available via plugins
Customization	Basic customization options	Customizable prompts and themes using built-in tools	Highly customizable using frameworks like oh-my-zsh
Performance Impact	Lightweight and fast	Lightweight and responsive	Can be slower due to heavy customization
Best Suited For	Servers, scripting, system administration	Beginners and users who prefer interactive shells	Power users and developers who want customization

Choosing the Right Shell

- Bash → Best for compatibility, scripting, and servers
- Fish → Best for beginners and interactive use
- Zsh → Best for power users who want customization

Shell Scripting

- A shell script is a file containing a set of Linux commands.
- Used to automate repetitive tasks instead of typing commands repeatedly.
- All common Linux shells (Bash, Zsh, Fish) support scripting.
- Shell scripting is only one way to script; other languages can also be used (Python, etc.).
- This topic focuses on shell (Bash) scripting.

Creating a Shell Script

Steps to Create a Script

1. Open the terminal.
2. Use a text editor (e.g., nano) to create a file.
3. File must have a .sh extension.

Type : *nano first_script.sh*

Shebang (Interpreter Definition)

- Every script must start with a shebang.
- Shebang tells the OS which interpreter to use.

#!/bin/bash

- *#!* → special characters
- */bin/bash* → path to Bash interpreter

Variables in Shell Scripts

- A variable stores a value (e.g., name, path, URL).
- Makes scripts cleaner, reusable, and easier to maintain.

Example: Using Variables & User Input

#!/bin/bash

echo "Hey, what's your name?"

read name

echo "Welcome, \$name"

Saving & Executing a Script

Save the Script

Press CTRL + X → Y → ENTER

Give Execution Permission

```
chmod +x first_script.sh
```

Run the Script

```
./first_script.sh
```

Why ./ is required:

- ./ tells Linux to execute a file in the current directory
- Without it, the shell searches in PATH and fails to find the script

Loops in Shell Scripts

- A loop repeats a task multiple times.
- Useful when performing the same action on multiple values.

Example : Printing Numbers 1 to 10

```
#!/bin/bash  
for i in {1..10};  
do  
echo $i  
done
```


Conditional Statements

- Used to execute code based on conditions
- Commonly used for authorization, validation, and decision-making

Example : Authorization Check

```
#!/bin/bash
echo "Please enter your name first:"
read name
if [ "$name" = "Stewart" ]; then
    echo "Welcome Stewart! Here is the secret:
    THM_Script"
else
    echo "Sorry! You are not authorized to access
    the secret."
fi
```

Comments in Shell Scripts

- Comments are ignored by the shell
- Used to explain code for better readability and maintenance
- Start with #

Example with Comments

```
#!/bin/bash
# Asking the user to enter a name
echo "Please enter your name first:"

# Storing user input
read name
```

```
# Checking authorization  
if [ "$name" = "Stewart" ]; then  
        echo "Welcome Stewart! Here is the secret:  
THM_Script"  
else  
        echo "Sorry! You are not authorized to access  
the secret."  
fi
```

Section 05 : Networking

Room 01 : Networking Concepts

OSI Model

- OSI (Open Systems Interconnection) model is a theoretical framework created by ISO.
- It explains how data moves from one computer to another over a network.
- It has 7 layers, each with a specific role.
- Learning it helps you understand how networking really works, even if real networks don't follow it perfectly.

Why is the OSI Model Important?

- Helps break complex networking into simple, manageable layers
- Makes it easier to troubleshoot network problems
- Important for understanding terms like Layer 3 switch or Layer 7 firewall

OSI Layers Order (Bottom → Top)

Mnemonic to remember : **P**lease **D**o **N**ot **T**hrow **S**pinach **P**izza **A**way

Layer Number	Layer Name	Main Function	Example Protocols and Standards
Layer 7	Application layer	Providing services and interfaces to applications	HTTP, FTP, DNS, POP3, SMTP, IMAP
Layer 6	Presentation layer	Data encoding, encryption, and compression	Unicode, MIME, JPEG, PNG, MPEG
Layer 5	Session layer	Establishing, maintaining, and synchronising sessions	NFS, RPC
Layer 4	Transport layer	End-to-end communication and data segmentation	UDP, TCP
Layer 3	Network layer	Logical addressing and routing between networks	IP, ICMP, IPSec
Layer 2	Data link layer	Reliable data transfer between adjacent nodes	Ethernet (802.3), WiFi (802.11)
Layer 1	Physical layer	Physical data transmission media	Electrical, optical, and wireless signals

TCP/IP - Transmission Control Protocol/Internet Protocol

- Developed in the 1970s by the U.S. Department of Defense (DOD)
- It is a practical, implemented networking model (unlike the theoretical OSI model)
- Forms the foundation of the Internet

Why was TCP/IP Developed ?

- Designed to keep networks working even if any of parts fail, for instance, due to military attack.
- Important for military and critical communications
- Uses adaptive routing, so data can find new paths if part of the network goes down

In our presentation of the ISO OSI model, we went from bottom to top, from layer 1 to layer 7. In this task, let's look at things from a different perspective, from top to bottom. From top to bottom, we have:

- Application Layer: The OSI model application, presentation and session layers, i.e., layers 5, 6, and 7, are grouped into the application layer in the TCP/IP model.
- Transport Layer: This is layer 4.
- Internet Layer: This is layer 3. The OSI model's network layer is called the Internet layer in the TCP/IP model.
- Link Layer: This is layer 2.

Layer Number	ISO OSI Model	TCP/IP Model (RFC 1122)	Protocols
7	Application Layer	Application Layer	HTTP, HTTPS, FTP, POP3, SMTP, IMAP, Telnet, SSH
6	Presentation Layer		
5	Session Layer		
4	Transport Layer	Transport Layer	TCP, UDP
3	Network Layer	Internet Layer	IP, ICMP, IPSec
2	Data Link Layer	Link Layer	Ethernet 802.3, WiFi 802.11
1	Physical Layer		

OSI vs TCP/IP Model

Basis of Comparison	OSI Model	TCP/IP Model
Nature of the model	Conceptual and theoretical model because it was designed to <i>explain and standardize</i> how communication <i>should</i> work	Practical and implementation-oriented model because it was designed alongside real protocols, reflecting how <i>networking actually works in practice</i>
Developed by	ISO (International Organization for Standardization) for creating a universal reference usable by vendors and educators	DARPA (U.S. Department of Defense) to develop a working communication system for military and research networks
Primary purpose	Standardization and learning, as it helps understand networking concepts clearly and in isolation	Reliable data communication over real networks, focusing on ensuring systems can communicate efficiently and robustly
Adoption in real networks	Rarely implemented directly because OSI does not define mandatory protocols, so vendors rarely implement it as-is	Universally implemented (Internet standard) because TCP/IP includes concrete, proven protocols used in practice
Protocol dependency	Protocol-independent since OSI defines <i>what functions should exist</i> , not which protocols must be used	Protocol-specific because TCP/IP was built around specific protocols that were tested and deployed
Flexibility	More rigid because OSI strictly separates responsibilities, which can complicate real implementations	More flexible because TCP/IP allows overlapping responsibilities to improve efficiency
Approach to reliability and error handling	Designed as a clear, step-by-step logical process to make understanding easier	Designed for robustness and fault tolerance so communication can survive network failures
Ease of learning	Easier to understand conceptually since it breaks communication into well-defined responsibilities	Harder to learn theoretically but easier to use practically because complexity is hidden for efficiency
Design timeline	Model designed first, protocols later, following a top-down academic approach	Protocols designed first, model later, following a bottom-up engineering approach based on working systems

Term	Which model	Meaning	Why that model is called so
Top-Down	OSI Model	Designing from theory to implementation: you first define <i>what each layer should do in principle</i> , then decide how to implement it	OSI was designed academically. ISO thought: “Let’s define a perfect model of communication first, and then protocols can be created to fit that model.” Hence, top-down: theory → implementation
Bottom-Up	TCP/IP Model	Designing from practical protocols to the overall model: you start with working protocols and then generalize a model to describe them	TCP/IP was created to solve real-world networking problems. Engineers said: “Let’s make it work first, then define a model to describe what we already implemented.” Hence, bottom-up: practice → abstraction

In simple words:

- Top-down = start with ideas, then make it real.
- Bottom-up = start with what works, then make a framework.

Term	Which model	Meaning	Why that model is called so
Horizontal approach	OSI Model	Each layer is independent and only communicates with the layer directly above or below	OSI separates responsibilities strictly. For example, the transport layer only talks to the layer above (application) and below (network). So, data flows “horizontally” across the network through these layers.
Vertical approach	TCP/IP Model	Layers can interact more flexibly, and some responsibilities overlap across layers	TCP/IP was built for practical communication, not pure theory. Layers may share tasks, like error handling in multiple layers, so communication is “vertical” in the sense that responsibilities are not strictly isolated—they can cut across layers.

- “Horizontal” = strictly layer-to-layer, like horizontal slices of cake — neat and separated.
- “Vertical” = some responsibilities run across multiple layers, like vertical pillars through the floors — more flexible and integrated.

IP Address

- An IP address is a unique identifier for a device (host) on a network.
- It allows devices to find and communicate with each other without confusion.
- Common examples:
 - 192.168.0.1
 - 172.16.159.243
- These examples are IPv4 addresses (IP version 4).

Why Do We Need IP Addresses?

- Every device on a network must have a unique IP address
- Without a unique address:
 - A device cannot be located
 - Communication becomes ambiguous
- Required for communication when using the TCP/IP protocol suite

IP Address Analogy

- An IP address is like a home postal address
- It:
 - Identifies your location uniquely
 - Allows others to send data (letters/parcels) to you
- Without it, communication (like online shopping) wouldn't work

IPv4 vs IPv6

Aspect	IPv4	IPv6
Full Form	Internet Protocol version 4	Internet Protocol version 6
Address Length	32 bits	128 bits
Address Format	Decimal, dotted quad (e.g., 192.168.1.1)	Hexadecimal, colon-separated (e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334)
Address Space	~4.3 billion addresses (2^{32})	~ 3.4×10^{38} addresses (2^{128}), virtually unlimited
Header Complexity	Simple but limited options	Larger header, more fields, improved functionality (like flow label)
Configuration	Can be manual (static) or via DHCP	Auto-configuration via Stateless Address Autoconfiguration (SLAAC) and DHCPv6
Fragmentation	Performed by routers and hosts	Performed only by hosts, not by routers (improves router efficiency)
Security	Security optional (IPSec optional)	IPSec support mandatory for IPv6
Checksum	Header checksum present	No header checksum (relies on transport layer checks)
Broadcast	Supports broadcast	No broadcast; uses multicast and anycast instead
NAT Requirement	Often required due to address shortage	Not needed due to vast address space
Mobility	Limited support	Better support for mobile devices (Mobile IPv6)
Quality of Service (QoS)	Type of Service (ToS) field exists	Flow Label field supports improved QoS
DNS Support	Supports IPv4 addresses (A records)	Supports IPv6 addresses (AAAA records)
Deployment	Widely deployed	Increasing adoption, coexistence with IPv4 via dual-stack or tunneling
Packet Size	Minimum MTU: 68 bytes, Maximum: 65,535 bytes	Minimum MTU: 1280 bytes, Maximum: 4,294,967,295 bytes
Routing Efficiency	Routing tables larger due to limited aggregation	Simplified and hierarchical addressing improves routing efficiency

- If a text says “**IP address**” without specifying, it usually means **IPv4**

Structure of an IPv4 Address

- An IPv4 address:
 - Has 32 bits
 - Is divided into 4 octets (bytes)
- Each octet:
 - Is 8 bits
 - Can represent numbers from 0 to 255
- Example format : 192.168.66.89

Network and Broadcast Addresses

- In a subnet :
 - Network address → identifies the network
 - Broadcast address → sends data to *all* hosts in that network
- Example:
 - 192.168.1.0 → Network address
 - 192.168.1.255 → Broadcast address
- These addresses cannot be assigned to hosts

IPv4 Address Limit

- IPv4 supports about **4 billion addresses**
- Based on **2^{32} possible combinations**
- Actual usable addresses are fewer due to :
 - Network addresses
 - Broadcast addresses

Checking Your IP Address

On Windows

- Use command : *ipconfig*

On Linux / UNIX

- Use : *ifconfig* or *ip a s*

Subnet Mask and CIDR Notation

- 255.255.255.0 is the same as /24
- /24 means:
 - First 24 bits are fixed (network part)
 - Last 8 bits are for hosts
- Host address range:
 - 192.168.66.1 → 192.168.66.254
- Reserved:
 - 192.168.66.0 → Network
 - 192.168.66.255 → Broadcast

Public vs Private IP Addresses

Two Types of IP Addresses

1. Public IP addresses
2. Private IP addresses

Basis of Comparison	Private IP Address	Public IP Address
Definition	A Private IP address is used inside a local network (like home, office, or school) to identify devices.	A Public IP address is used on the internet to identify a network or device globally.
Scope	Works only within a local network (LAN).	Works globally across the internet.
Uniqueness	Unique only within the local network. Same private IP can exist in different networks.	Globally unique — no two devices on the internet have the same public IP.
Assigned By	Assigned by the router or network administrator.	Assigned by the Internet Service Provider (ISP).
Accessibility	Not accessible directly from the internet.	Accessible from the internet.
Security	More secure, as it is hidden from the internet.	Less secure, as it is visible on the internet.
Usage	Used for internal communication between devices (PCs, printers, phones).	Used for external communication with websites and online services.
Cost	Free to use; reused across networks.	Often limited and managed by ISPs due to scarcity.
Example Range	192.168.0.1 – 192.168.255.255 10.0.0.0 – 10.255.255.255	8.8.8.8 172.217.0.0 (Google)
Example	Laptop connected to home Wi-Fi using IP 192.168.1.5	Your router using public IP 49.36.128.10

How to check if an IP is private or public ?

Some IP ranges are reserved for private use. If an IP falls in any of these ranges, it is Private; otherwise, it is Public.

Private IP Address Ranges (IPv4)

Range	Class
10.0.0.0 – 10.255.255.255	Class A
172.16.0.0 – 172.31.255.255	Class B
192.168.0.0 – 192.168.255.255	Class C

If the IP is inside these ranges → PRIVATE IP ; If not → PUBLIC IP

Private IP Address Ranges (IPv6)

In IPv6, the term “Private IP” is not officially used like IPv4. Instead, IPv6 uses special address types.

The IPv6 equivalent of a Private IP is called **Unique Local Address (ULA)**.

How to Identify IPv6 Addresses ;

Key IPv6 Address Types

IPv6 Type	Prefix / Range	Public or Private?	Explanation
Global Unicast	2000::/3	 Public	Used on the internet; globally routable
Unique Local Address (ULA)	fc00::/7 (commonly fd00::/8)	 Private	Used inside local networks only
Link-Local	fe80::/10	 Private (local only)	Used for communication within the same network link
Loopback	::1	 Private	Refers to the local device itself
Multicast	ff00::/8	 Neither	Used to send data to multiple devices
Unspecified	::	 Neither	Means “no address assigned”

Simple Identification Rule



Private-type IPv6 addresses:

- Starts with fd → Unique Local (Private)
- Starts with fe80 → Link-Local (Private)
- ::1 → Loopback (Private)



Public IPv6 address:

- Starts with 2 or 3 (i.e. 2000::/3)

Internet Access for Private Ips

? How a Device with a Private IP Accesses the Internet

- A device (mobile/laptop) has a private IP address
- Private IP cannot work directly on the internet
- The device sends its internet request to the router
- The router has a public IP address
- The router uses NAT (Network Address Translation)
- NAT replaces the private IP with the public IP
- The request is sent to the internet using the public IP
- The website sends the response back to the public IP
- The router checks its NAT table
- The router forwards the response to the correct private IP device
- The device receives the data and accesses the internet

This way :

- Many devices share one public IP
- NAT provides basic security
- Private IPs are hidden from the internet
- Router acts as a gateway

End-to-End Flow of Email Communication Explained in Atomic Steps



Sender Side

- Application layer prepares the email message using SMTP.
- Presentation layer encrypts or formats the message if needed (e.g., TLS).
- Session layer manages the ongoing communication session with the client.
- Transport layer (TCP) breaks the message into segments and assigns port numbers.
- Network layer (IP) adds source and destination IP addresses.
- Data link layer frames the packet with MAC addresses for local delivery.
- Physical layer converts the frame into electrical signals or radio waves.
- Signals are transmitted across the network infrastructure (routers, switches, cables, Wi-Fi).



Receiver Side

- Physical layer receives electrical signals or radio waves.
- Data link layer extracts frames and checks MAC addresses.

- Network layer reads IP addresses and delivers the packet to the correct host.
- Transport layer reassembles TCP segments, checks sequence numbers, and acknowledges delivery.
- Session layer maintains the communication session with the server.
- Presentation layer decrypts or decodes the message if needed.
- Application layer (Outlook, Gmail web client, etc.) displays the email to the user.

Big Picture:

- Sender: Message flows **down the OSI stack** → transmitted physically.
- Receiver: Message flows **up the OSI stack** → reconstructed and shown to the user.

TCP and UDP Protocol

Why TCP and UDP are needed at all ?

- IP alone is not enough
 - IP only moves packets from one machine to another
 - It does not guarantee delivery, order, or correctness
- Applications need end-to-end communication rules
 - Apps must know *how* data is sent, received, checked, and managed
 - TCP and UDP provide these rules at the transport layer

- To allow multiple applications to use the network at the same time
 - TCP/UDP use port numbers to deliver data to the correct app
 - Without them, all data would arrive mixed together
- To handle unreliable networks
 - Networks can drop, duplicate, or reorder packets
 - Transport protocols exist to cope with this unreliability
- To avoid pushing all complexity into applications
 - Common problems (delivery, timing, flow control) are solved once in TCP/UDP
 - Apps don't need to reinvent networking logic
- To support different communication models
 - Some apps need guaranteed delivery
 - Some apps need fast, best-effort delivery
 - Transport protocols make both possible
- To make the Internet scalable
 - Standard transport protocols allow systems from different vendors to communicate
 - Without them, the Internet wouldn't interoperate at global scale

UDP (User Datagram Protocol)

- Type: Connection-less protocol (Layer 4).
- Key Features:
 - No connection setup required.
 - No delivery confirmation (no guarantee packet arrives).
 - Uses port numbers (range: 1-65535, port 0 reserved) to identify processes.

- Pros: Faster, less overhead.
- Cons: Unreliable (packets may be lost or duplicated).
- Example : Live audio/video communication where real-time delivery is important and late or re-transmitted packets are useless.

TCP (Transmission Control Protocol)

- Type: Connection-oriented protocol (Layer 4).
- Key Features:
 - Requires a connection before sending data.
 - Reliable delivery: uses sequence numbers and acknowledgements.
 - Ensures data arrives correctly and in order.
 - Uses port numbers (same range as UDP).
- Connection Setup:
 - Done via Three-Way Handshake:
 - 1.SYN → Client sends request with initial sequence number.
 - 2.SYN-ACK → Server responds with its own sequence number + acknowledgement.
 - 3.ACK → Client confirms, connection established.
- Example : Web browsing or file transfer where complete, ordered, and reliable data delivery is essential.

Feature	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection	Connection-oriented: requires a handshake before data transfer.	Connectionless: sends data without establishing a connection.
Reliability	Reliable: ensures data arrives correctly and in order.	Unreliable: no guarantee of delivery or order.
Acknowledgement	Uses acknowledgements to confirm packet receipt.	No acknowledgements; sender does not know if data was received.
Error Handling	Detects and retransmits lost or corrupted packets.	Minimal error handling; corrupted packets are simply discarded.
Speed	Slower due to overhead of reliability mechanisms.	Faster because it avoids connection setup and checks.
Use of Ports	Uses port numbers to identify processes (e.g., HTTP on port 80).	Also uses port numbers to identify processes (e.g., DNS on port 53).
Data Segmentation	Breaks data into segments with sequence numbers.	Sends data as independent datagrams without sequencing.
Best Use Cases	Web browsing, emails, file transfers where accuracy matters.	Streaming, gaming, VoIP where speed matters more than reliability.
Analogy	Like registered mail: delivery is tracked and confirmed.	Like standard mail: delivery is fast but not guaranteed.

Handshakes

What is a Handshake in Networking?

- A handshake is the process by which two devices establish communication rules before exchanging data.
- It ensures both sides agree on parameters like synchronization, acknowledgement, and readiness.

2-Way Handshake

- Step 1: Client sends a SYN (synchronize) packet to the server.
- Step 2: Server replies with an ACK (acknowledge) packet.
- Result: Connection is considered established.
- Limitation: Client cannot be sure the server is ready, because the server never sends its own sequence number. This can lead to reliability issues.

3-Way Handshake (TCP Standard)

- Step 1: Client sends a SYN packet with its initial sequence number.
- Step 2: Server responds with SYN-ACK (synchronize + acknowledge), including its own sequence number.
- Step 3: Client sends an ACK to confirm receipt of the server's sequence number.
- Result: Both sides are synchronized and ready for reliable communication.
- Advantage: Ensures both client and server agree on sequence numbers and are fully ready.

2-Way vs 3-Way Handshake

Feature	2-Way Handshake	3-Way Handshake
Steps	SYN → ACK	SYN → SYN-ACK → ACK
Reliability	Less reliable; client cannot confirm server readiness.	Highly reliable; both sides confirm readiness.
Sequence Numbers	Only client's sequence number is sent.	Both client and server exchange sequence numbers.
Use Case	Rarely used; insecure for TCP.	Standard in TCP connections.
Security	Vulnerable to half-open connections.	Prevents half-open connections by mutual confirmation.

A list of the different types of handshakes commonly referenced in networking and security contexts :

- 2-Way Handshake
- 3-Way Handshake
- 4-Way Handshake
- SSL/TLS Handshake
- SSH Handshake
- Bluetooth Handshake
- Wi-Fi WPA/WPA2 Handshake
- QUIC Handshake
- Kerberos Handshake
- Diffie-Hellman Key Exchange (Handshake)

Encapsulation in Networking

♦ Key Concept

- Encapsulation = Each OSI layer adds its own header (and sometimes trailer) to the data before passing it down to the next layer.
- This allows each layer to focus on its specific function without worrying about the others.
- At the receiver side, the process is reversed (decapsulation) until the original application data is extracted.

Steps of Encapsulation

1. Application Data

- User inputs data (e.g., writing an email or search query).
- Application formats it according to its protocol (e.g., HTTP, SMTP).

2. Transport Layer (TCP/UDP)

- Adds a TCP or UDP header.
- Creates a segment (TCP) or datagram (UDP).
- Identifies processes using port numbers.

3. Network Layer (IP)

- Adds an IP header with source and destination IP addresses.
- Creates an IP packet.

4. Data Link Layer (Ethernet/Wi-Fi)

- Adds a frame header and trailer (includes MAC addresses).
- Creates a frame ready for physical transmission.

5. Physical Layer

- Converts the frame into bits/signals (electrical, optical, or radio waves).
- Sends it across the medium (cable or Wi-Fi).

Life of a Packet (Example: Searching on TryHackMe)

- User enters a search query in the browser.
- Browser (Application Layer) prepares an HTTP request over HTTPS.
- TCP establishes a 3-way handshake with the TryHackMe server.
- TCP segments carry the HTTP request to the IP layer.
- IP adds source (your PC) and destination (TryHackMe server) addresses.
- Link layer adds headers/trailers and sends the frame to the router.
- Router strips link layer info, checks IP destination, and forwards packet.
- Routers repeat this until the packet reaches the destination server.

- At the server side, the process is decapsulated layer by layer until the HTTP request reaches the web application.
- The server processes the request and sends back a response using the same encapsulation steps in reverse.

What is Telnet ?

- Telnet (short for *Teletype Network*) is one of the earliest networking protocols (introduced in 1969).
- It allows a user to remotely log into another computer over a TCP/IP network using a text-based interface.
- Operates on TCP port 23 by default.
- Works on a client-server model:
 - Client = terminal emulator.
 - Server = remote host providing access.

Why Telnet Came Into Picture ?

- In the early days of networking, administrators needed a way to remotely access and control servers.
- Telnet provided a virtual terminal service, meaning you could sit at one computer and interact with another as if you were physically present.
- It solved the problem of remote system management before graphical interfaces and secure protocols existed.

What Service Telnet Provides / What Issue It Fixed

- Provides remote login capability to servers and network devices.
- Allowed network troubleshooting and remote configuration.
- Fixed the issue of physical dependency (no need to be at the server's location).
- Enabled multi-user access to centralized systems.

Pros and Cons of Telnet

Aspect	Pros (Advantages)	Cons (Disadvantages)
Ease of Use	Simple text-based interface, easy to implement.	No encryption, all data (including passwords) sent in plain text.
Remote Access	Allowed remote login and control of servers/devices.	Vulnerable to eavesdropping and man-in-the-middle attacks.
Lightweight	Very low overhead, fast communication.	Outdated, lacks modern security features.
Standardization	Early standard TCP/IP protocol for terminal emulation.	Replaced by secure alternatives like SSH.
Troubleshooting	Useful for testing connectivity and ports.	Not suitable for production environments today due to risks.

Other Protocols Similar to Telnet

- SSH (Secure Shell):
 - Secure replacement for Telnet.
 - Encrypts communication, including passwords and data.
 - Runs on port 22.
- RDP (Remote Desktop Protocol):
 - Provides graphical remote access to Windows systems.
 - More advanced than Telnet's text-based interface.

- VNC (Virtual Network Computing):
 - Remote desktop sharing protocol.
 - Works across platforms, graphical interface.
- FTP (File Transfer Protocol):
 - Focused on transferring files, not remote login.
 - Uses TCP ports 20/21.
- rlogin (Remote Login):
 - Similar to Telnet, but Unix-specific.
 - Also insecure, replaced by SSH.

In short :

- Telnet was revolutionary in the 1970s–1990s for remote access.
- Its biggest flaw: lack of encryption, making it insecure today.
- Modern alternatives like SSH, RDP, and VNC provide secure and more feature-rich remote access.

Room 02 : Networking Essentials

Dynamic Host Configuration Protocol (DHCP)

When a device connects to a new network — such as your laptop joining a coffee shop's WiFi — it must be properly configured before communication can occur. This configuration includes:

- An IP address and subnet mask (to identify the device and its network boundaries),
- A default gateway (to reach external networks like the Internet),
- A DNS server (to translate domain names into IP addresses).

Manually setting these values is possible, but impractical for mobile devices. Instead, networks rely on Dynamic Host Configuration Protocol (DHCP) to automate this process.

Why DHCP ?

1. Convenience: Mobile devices frequently change networks. DHCP saves users from manual setup.
2. Conflict Prevention: It avoids IP address conflicts, which occur when two devices share the same IP.
3. Scalability: Large networks with hundreds of devices can be managed easily.
4. Reliability: Ensures consistent configuration across devices.

Servers, however, often use static IPs because they must remain at predictable addresses for other systems to locate them.

DHCP Basics

- Protocol Type: Application-level protocol.
- Transport Layer: Uses UDP (User Datagram Protocol).
 - Server listens on UDP port 67.
 - Client communicates from UDP port 68.
- Default Behavior: Most laptops, smartphones, and IoT devices are configured to use DHCP automatically.

The DORA Process

DHCP operates through a four-step exchange, abbreviated as DORA :

1. Discover

- The client broadcasts a DHCPDISCOVER message.
- Since it has no IP yet, it uses source IP 0.0.0.0 and destination 255.255.255.255.
- At the link layer, it uses the broadcast MAC address ff:ff:ff:ff:ff:ff.

2. Offer

- The DHCP server replies with a DHCPOFFER.
- This includes an available IP address and other configuration details.

3. Request

- The client responds with a DHCPREQUEST.
- This indicates acceptance of the offered IP.

4. Acknowledge

- The server finalizes the process with a DHCPACK.
- The IP address is now officially leased to the client.

Example: Packet Capture

A packet capture illustrates the DORA sequence:

- 1 0.0.0.0 → 255.255.255.255 DHCP Discover
- 2 192.168.66.1 → 192.168.66.133 DHCP Offer
- 3 0.0.0.0 → 255.255.255.255 DHCP Request
- 4 192.168.66.1 → 192.168.66.133 DHCP ACK

- The client begins with no IP (0.0.0.0).
- The server assigns IP 192.168.66.133 in this example.
- Communication relies initially on MAC addresses until the IP is confirmed.

What DHCP Provides

At the end of the process, the client receives:

- Leased IP Address → Unique identifier on the network.
- Default Gateway → Pathway to external networks.
- DNS Server → Enables domain name resolution.

This ensures the device can access both local resources and the Internet seamlessly.

Address Resolution Protocol (ARP)

When two hosts communicate over a network, the data must travel through multiple layers of the networking stack. At Layer 3 (Network Layer), communication is based on IP addresses. However, at Layer 2 (Data Link Layer) — such as Ethernet (IEEE 802.3) or WiFi (IEEE 802.11) — communication requires MAC addresses.

Thus, whenever a host wants to send an IP packet to another host on the same local network, it must encapsulate the packet inside a data link frame. This frame includes:

- Destination MAC address
- Source MAC address
- Type field (indicating the protocol, e.g., IPv4)

The challenge: the host may know the target's IP address, but not its MAC address. This is where ARP (Address Resolution Protocol) comes in.

MAC Addresses

- A MAC address is a 48-bit identifier unique to each network interface card (NIC).
- Represented in hexadecimal, e.g., 7C:DF:A1:D3:8C:5C or 44:DF:65:D8:FE:6C.
- Used only within the local network segment (Layer 2).
- Devices don't need to know each other's MAC addresses all the time — only when they actually exchange frames.

The Need for ARP

Consider a typical scenario:

- Your device connects to a network and receives IP configuration (via DHCP).
- It knows the IP address of the gateway (router) and the DNS server.

- Communication at this stage is IP-based; MAC addresses are not revealed.
- But when two devices on the same Ethernet/WiFi need to exchange data, they must know each other's MAC addresses.

ARP bridges this gap by mapping IP addresses (Layer 3) to MAC addresses (Layer 2).

ARP Operation

ARP works through two message types:

1. ARP Request

- Sent when a host wants to know the MAC address corresponding to an IP address.
- Broadcasted to all devices on the local network (ff:ff:ff:ff:ff:ff).
- Example: Host 192.168.66.89 asks, *"Who has 192.168.66.1?"*

2. ARP Reply

- The device owning that IP responds with its MAC address.
- Sent directly to the requester's MAC address.
- Example: Host 192.168.66.1 replies, *"192.168.66.1 is at 44:df:65:d8:fe:6c."*

Note :

- The Request is broadcast.
- The Reply is unicast (direct to requester).

Encapsulation

- ARP messages are not encapsulated in IP or UDP packets.
- Instead, they are placed directly inside Ethernet frames.
- This makes ARP unique: it operates at the boundary of Layer 2 and Layer 3.

Layer Classification Debate

- Layer 2 View: ARP deals with MAC addresses, so it belongs to the Data Link Layer.
- Layer 3 View: ARP supports IP operations, so it can be considered part of the Network Layer.
- Practical Understanding: ARP is a translator — it maps Layer 3 addresses (IP) to Layer 2 addresses (MAC). Its role is to enable communication between these layers.

ICMP - Internet Control Message Protocol

The Internet Control Message Protocol (ICMP) is a network-layer protocol used by IP devices (routers and hosts) to send error messages and diagnostic information about network conditions. It plays an essential role in maintaining the integrity and functionality of IP networks by reporting issues like unreachable destinations, time-exceeded packets, or routing problems.

ICMP is part of the IP suite, which forms the foundation of the Internet. While IP handles the actual packet delivery, ICMP helps network devices communicate about the state of that delivery.

Key Characteristics :

- **Error Reporting:** ICMP reports issues that occur during the transmission of IP packets.
- **Diagnostics:** It helps test and measure network connectivity, delay, and packet loss.
- **No Application Data:** ICMP does not carry user data. It is used exclusively for network control and diagnostics.

Why ICMP is Important

ICMP provides several key functionalities that are crucial for network health and management :

1. Error Detection and Reporting:

- When routers or hosts encounter issues (e.g., unreachable destination, expired packet), they use ICMP to notify the sender.
- Examples include Destination Unreachable (Type 3) or Time Exceeded (Type 11) messages.

2. Network Diagnostics:

- It allows tools like ping and traceroute to test connectivity, measure round-trip time, and map network routes.
- Without ICMP, diagnosing network issues would be far more complex.

3. Route and Path Discovery:

- Tools like traceroute rely on ICMP's Time Exceeded messages to trace the route packets take through the network.

ICMP Message Structure

ICMP messages are encapsulated within IP packets and are structured to include specific fields that convey the type of message and additional information.

Fields in ICMP Message :

- 1.Type (8 bits): Specifies the type of ICMP message.
- 2.Code (8 bits): Provides additional detail for the message type (sub-type).
- 3.Checksum (16 bits): Used for error-checking the ICMP header and data.
- 4.Rest of the Header/Data: Depends on the message type; for example, the Echo Request/Reply includes a Sequence Number to match requests with replies.

Common ICMP Types and Their Purpose:

Type	Purpose	Common Use Case
Type 0	Echo Reply	Ping response
Type 3	Destination Unreachable	Network or host unreachable error
Type 4	Source Quench (deprecated)	Congestion control (rare today)
Type 5	Redirect	Route optimization (routing advice)
Type 8	Echo Request	Ping request
Type 11	Time Exceeded	Traceroute (TTL expired)
Type 12	Parameter Problem	Invalid packet header
Type 13	Timestamp Request	Request for timestamp
Type 14	Timestamp Reply	Response to timestamp request
Type 17	Address Mask Request	Request for subnet mask
Type 18	Address Mask Reply	Reply with subnet mask

ICMP-Based Commands

Two popular ICMP-based tools used for network diagnostics and troubleshooting are:

- Ping
- Traceroute

1. Ping

What is Ping Used For?

The ping command is used to test the reachability of a destination host on a network and measure the round-trip time (RTT) it takes for packets to travel from the source to the destination and back. It helps determine if the target is alive and if it can respond to requests.

How Ping Works

- The source sends an ICMP Echo Request (Type 8) to the destination.
- If the target is reachable and configured to respond, it will send back an ICMP Echo Reply (Type 0).
- The round-trip time (RTT) is measured by calculating the time taken for the request to reach the destination and for the reply to return.

Ping Syntax (Linux/Unix example):

`ping [destination IP or domain]`

Understanding Ping Output

The output from a ping command typically looks like this:

64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=3.45 ms

64 bytes from 192.168.1.1: icmp_seq=2 ttl=64 time=4.00 ms

Key metrics in the output:

- `icmp_seq`: The sequence number of the ping packet.
- TTL (Time to Live): The number of hops (routers) the packet can travel before being discarded. It decreases by 1 with each hop.
- Time (RTT): Round-trip time in milliseconds, indicating the network latency.

At the end of the output, you'll see summary statistics:

- Packets transmitted vs. received: Indicates packet loss.
- Min/Avg/Max RTT: Shows network performance (low RTT = good, high RTT = potential issues).

Why Ping May Fail Even When Host Is Up

- Firewall Block: Many firewalls block ICMP Echo Requests for security reasons.
- Network Configuration: Some hosts or routers are configured not to respond to ICMP requests.
- Rate Limiting: A target may limit the number of ICMP requests it responds to in a given time period.

What Ping Actually Measures

- Basic Reachability: Whether the target host is online and responding.
- Network Latency: How long it takes for data to travel from source to target and back.
- Packet Loss: How many requests didn't get a reply (indicating network instability or congestion).

2. Traceroute

What is Traceroute Used For?

The traceroute (or tracert in Windows) command helps determine the path that packets take from the source to the destination across a network. It shows the intermediate routers or hops that packets traverse to reach their destination.

How Traceroute Works

- Traceroute works by sending ICMP Echo Requests with increasing TTL (Time-to-Live) values. Each TTL value represents how many hops the packet can traverse before being discarded.
- When a router receives a packet with a TTL of 1, it decrements the TTL and sends back an ICMP Time Exceeded (Type 11) message.
- Traceroute increases the TTL value with each successive packet to discover the next hop in the network path.

Traceroute Syntax:

`traceroute [destination IP or domain]`

Understanding Traceroute Output

The output from traceroute typically shows something like this:

```
1  192.168.1.1 (192.168.1.1)  1.356 ms  1.302 ms  1.267 ms
2  10.0.0.1 (10.0.0.1)  3.723 ms  3.689 ms  3.660 ms
3  * * *
4  172.16.0.1 (172.16.0.1)  10.412 ms  10.376 ms  10.359 ms
...
```

Key components:

- Hop Number: The number of the router in the path.
- IP Address/Host Name: The IP or DNS-resolved hostname of each router.
- RTT: The round-trip time for each probe.
- Asterisks (* * *) indicate a timeout or lack of response from a router.

Why Traceroute May Fail

- Packet Drops: Some routers are configured to ignore ICMP Time Exceeded messages or limit the rate of ICMP replies.
- Firewalls: Some firewalls block ICMP Echo Requests or Time Exceeded messages to prevent path discovery.
- Network Congestion: High congestion or misconfigured routers may cause delays or packet drops.

What Traceroute Actually Measures

- Route Path: Identifies all the intermediate hops between the source and destination.
- Latency per Hop: Measures the time it takes to reach each hop, helping identify bottlenecks or problematic links in the network.

How These Types Are Used in Real-Life Networking

- Ping (Type 8, Type 0) is widely used to test basic network connectivity and latency.
- Traceroute relies on Type 11 to identify the path packets take across the network.
- Type 3 (Destination Unreachable) is useful for diagnosing problems like:
 - Misconfigured routers
 - Unreachable hosts

- Firewall blocking traffic
- Type 5 (Redirect) is used for dynamic routing adjustments and is especially important for optimizing paths in real-time.

Routing Algorithms and Protocols

In any interconnected network (including the Internet), there are multiple paths through which data can travel. Routing refers to the process by which packets of data are directed across these paths from the source to the destination. When a network has multiple routers, the routing algorithm is responsible for determining the most efficient way to deliver data.

Routing Process

Consider a network with multiple interconnected routers and multiple possible paths between devices (as seen in the given diagram). For a packet to be delivered across networks (like from Network 1 to Network 2 or Network 3), a routing algorithm is necessary to determine the best path.

1. Router Functions:

- Routers examine the destination address of each incoming packet.
- They use a routing table to decide the next hop or router to forward the packet to.
- The router ensures that data is forwarded in a manner that makes the most sense, considering factors like path cost, network topology, and traffic conditions.

2. Multiple Paths:

- In any given network, there are often multiple paths to reach the destination.
- The routing algorithm helps the router select the best possible path out of these alternatives based on different criteria such as distance, load, and reliability.

Routing Algorithms

Routing algorithms can be broadly classified into two categories:

1. Distance-Vector Routing

- Routers share their routing tables with neighboring routers at regular intervals to calculate the shortest path to a destination.
- Examples: RIP (Routing Information Protocol).

2. Link-State Routing

- Each router builds a complete map of the network by exchanging information with every other router.
- This allows routers to calculate the most efficient path to any destination.
- Examples: OSPF (Open Shortest Path First), EIGRP (Enhanced Interior Gateway Routing Protocol).

Popular Routing Protocols

1. OSPF (Open Shortest Path First)

- Category: Link-State Protocol
- Purpose: OSPF is designed to share detailed information about the network topology among routers and calculate the most efficient paths based on this information.
- How it Works:
 - Routers exchange link-state advertisements (LSAs) to inform other routers of the state of their directly connected links.
 - Each router uses these LSAs to build a complete map of the network topology.
 - The router then calculates the shortest path to any destination using Dijkstra's algorithm.
- Use Case: Ideal for large enterprise networks with multiple routers, offering fast convergence and scalability.

2. EIGRP (Enhanced Interior Gateway Routing Protocol)

- Category: Hybrid Protocol (combines features of Distance-Vector and Link-State protocols)
- Purpose: EIGRP is a Cisco proprietary routing protocol that enables routers to share information about reachable networks and the cost (e.g., bandwidth, delay) of those routes.
- How it Works:
 - EIGRP routers share routing information via hello packets and updates.
 - The protocol uses a metric system that factors in multiple criteria, including bandwidth, delay, load, and reliability.
 - EIGRP's Diffusing Update Algorithm (DUAL) ensures the router always has a loop-free path to its destination.
- Use Case: Commonly used in Cisco-based networks, offering fast convergence and flexibility for medium-to-large networks.

3. BGP (Border Gateway Protocol)

- Category: Path-Vector Protocol
- Purpose: BGP is the core routing protocol used across the Internet for inter-domain routing. It is responsible for exchange of routing information between different autonomous systems (ASes).
- How it Works:
 - BGP routers maintain a list of AS paths that data packets can take between different networks.
 - BGP uses path attributes, including AS path length, next hop, and network policies, to make routing decisions.
 - BGP uses a form of distance-vector routing, but unlike traditional distance-vector protocols, it does not exchange the full routing table but rather path information and updates.
- Use Case: Inter-domain routing on the Internet, allowing ISP networks to share information and route data between different networks.

4. RIP (Routing Information Protocol)

- Category: Distance-Vector Protocol
- Purpose: RIP is one of the simplest and oldest routing protocols used in smaller networks. It is used to determine the best path based on the fewest hops to the destination.
- How it Works:
 - Routers running RIP periodically send their entire routing table to their neighbors.
 - Each router calculates the distance to various networks, with a maximum hop count limit of 15 hops (anything beyond that is considered unreachable).
 - RIP uses a simple metric based on hop count to determine the best route.
- Use Case: Best for small networks where simplicity is more important than scalability or fast convergence.

Routing Protocols and Their Use Cases

Protocol	Category	Primary Purpose	Key Features	Use Case
OSPF	Link-State	Share network topology for efficient routing	Fast convergence, scalable, hierarchical	Large enterprise networks
EIGRP	Hybrid (Distance-Vector + Link-State)	Efficient routing with multiple metrics (bandwidth, delay)	Fast convergence, Cisco proprietary	Cisco-based networks
BGP	Path-Vector	Inter-domain routing for the Internet	Exchanges path information, uses AS paths, scalable	Internet routing between ISPs
RIP	Distance-Vector	Simple routing based on fewest hops	Limited to 15 hops, simple and easy to configure	Small networks or educational purposes

Real-Life Application Example:

- Scenario: A mobile user is trying to reach a web server.
 - Problem: The packet needs to travel through multiple routers, each potentially with a different path.
 - Routing Protocol Involved:
 - OSPF could be used within an enterprise to determine efficient internal routes.
 - BGP would likely be involved in routing between the user's ISP and the web server's ISP, as it's an inter-domain (Internet-wide) protocol.
 - RIP or EIGRP could be used within smaller sections of the network, depending on size and complexity.

Network Address Translation (NAT)

The rapid expansion of devices connecting to the internet — from computers and smartphones to IoT devices (such as security cameras, smart appliances, and even washing machines) — has exhausted the available IPv4 address space, which can accommodate a maximum of roughly 4 billion addresses. This limited address space was quickly filled, leading to the depletion of public IP addresses.

To solve this problem, Network Address Translation (NAT) was developed. NAT allows multiple devices on a private network to share a single or a small pool of public IP addresses for accessing the internet. Essentially, NAT enables address conservation and increases security by hiding internal network details.

Key Purpose of NAT

- **Address Conservation:** NAT enables many devices on an internal network to access the internet using one or a few public IP addresses. This saves valuable public IPs that would otherwise be exhausted quickly.
- **Security:** NAT helps protect internal devices by masking private IP addresses and only exposing the public IP address, making it harder for external systems to directly reach internal devices.

How NAT Works

NAT works by modifying the source or destination IP addresses and ports of network packets as they traverse a router, facilitating the translation between private IP addresses used within a local network and public IP addresses used on the internet. The key components of NAT are:

1. **Private IP Address:** These are addresses used by devices within a local network. They are not routable on the internet. Common ranges for private IP addresses include:
 - **IPv4 Private Address Ranges:**
 - 10.0.0.0 - 10.255.255.255
 - 172.16.0.0 - 172.31.255.255
 - 192.168.0.0 - 192.168.255.255
2. **Public IP Address:** These are the IP addresses assigned to devices on the internet. A public IP address is unique across the entire internet, and it's necessary to communicate with devices on external networks.

The Translation Process :

- Internal Network: Devices (e.g., computers, smartphones) within the private network have private IP addresses.
- NAT Router: A router with NAT functionality sits at the edge of the private network and has both a private IP address on the internal side and a public IP address on the external side.
- When a device (like a laptop) from the internal network attempts to connect to an external service (like a web server), the router modifies the packet's source IP address and source port to its public IP address and a newly assigned port number. The private IP address of the device is hidden from the external network.

Example of NAT Translation

Let's break this down with an example of a laptop connecting to a web server.

Scenario:

- Internal Device: A laptop has the private IP address 192.168.0.129 and initiates a connection to a web server on the internet.
- Public IP: The router's external (public) IP address is 212.3.4.5.
- NAT Translation: The NAT router will translate the source address of the laptop's packets from 192.168.0.129 to its public IP address 212.3.4.5 and also assign a new source port (e.g., 19273).

NAT Translation Table:

Internal IP	Internal Port	Public IP	Public Port
192.168.0.129	15401	212.3.4.5	19273

The laptop's internal IP (192.168.0.129) with port 15401 is translated to the router's public IP (212.3.4.5) with port 19273.

- The web server on the internet sees the connection as coming from 212.3.4.5 and port 19273 instead of the private IP address and internal port of the laptop.

Once the server responds, the router performs reverse translation, mapping the external public address (212.3.4.5 port 19273) back to the internal private IP address (192.168.0.129) and the original port (15401), and forwards the response to the correct internal device.

Types of NAT

1. Static NAT

- Purpose: Maps a specific internal IP address to a specific public IP address. This type of NAT is useful when an internal device needs to be consistently reachable from the outside (e.g., for hosting a web server).
- Example: 192.168.0.10 always maps to 203.0.113.5.

2. Dynamic NAT

- Purpose: Maps an internal IP address to any available public IP address from a pool of public IPs. This allows a group of internal devices to use multiple public IP addresses, but each internal device will only use one public IP address at a time.
- Example: 192.168.0.10 maps to 203.0.113.10, and 192.168.0.11 maps to 203.0.113.11.

3. Port Address Translation (PAT) (also known as NAT Overload)

- Purpose: A single public IP address is shared by many internal devices. Instead of mapping an internal IP address to a unique public IP, PAT uses port numbers to differentiate between devices.
- Example: Multiple internal devices (with private IP addresses like 192.168.0.10, 192.168.0.11) may all share the same public IP address (203.0.113.5), but each device uses a different source port number.
- Common Use Case: This is the most common form of NAT used in home and small business networks, where many devices share a single public IP address.

Advantages of NAT

1. **IP Address Conservation:** NAT helps conserve IPv4 address space by allowing many devices to share a single or a few public IP addresses.
2. **Security:** NAT enhances security by hiding internal IP addresses. External systems only see the public IP address of the router, which makes it harder for attackers to directly target internal devices.
3. **Flexible Network Addressing:** Networks can use private address space internally without needing to worry about exhaustion of public IP addresses.

Challenges of NAT

1. **Limited Peer-to-Peer Communication:** Some applications, particularly those that require direct communication between devices (e.g., VoIP or gaming), may face difficulties due to NAT, as it can interfere with the necessary peer-to-peer connections.
2. **Incompatibility with Some Protocols:** Protocols that embed IP addresses within the payload (e.g., FTP) may not work correctly with NAT without additional configuration or support (like NAT traversal).

Room 03 : Networking Core Protocols

Domain Name System (DNS)

The Domain Name System (DNS) is a fundamental component of modern computer networks and the internet. Its primary purpose is to translate human-readable domain names (*such as example.com*) into machine-readable IP addresses (*such as 93.184.215.14*).

Since IP addresses are numerical and difficult to memorize, DNS acts as a directory service, allowing users to access websites and services using easy-to-remember names instead of numerical identifiers.

Without DNS, users would need to remember the IP address of every website they wish to visit, which is impractical at scale.

DNS in the OSI Model

DNS operates at the Application Layer, which is Layer 7 of the ISO OSI (Open Systems Interconnection) model.

- Being an application-layer protocol means DNS directly interacts with user applications such as:
 - Web browsers
 - Email servers
 - Command-line tools

DNS Transport Protocols and Ports

DNS primarily uses two transport-layer protocols:

3.1 UDP (User Datagram Protocol)

- Default protocol for DNS queries
- Uses port 53
- Preferred because it is:
 - Fast
 - Lightweight
 - Suitable for small query/response exchanges

3.2 TCP (Transmission Control Protocol)

- Also uses port 53
- Acts as a fallback mechanism
- Typically used when:
 - Responses are too large for UDP
 - Zone transfers are required
 - Reliability is necessary

DNS Records (Resource Records)

DNS stores information in the form of records, also known as Resource Records (RRs). Each record serves a specific purpose. Among many available types, the following four are fundamental and commonly used.

A Record (Address Record)

- Maps a hostname to an IPv4 address
- Used when resolving domain names for most standard internet communication

Example : `example.com` → `172.17.2.172`

Key Points:

- Supports one or multiple IPv4 addresses
- Essential for web browsing
- Most frequently queried DNS record type

AAAA Record (IPv6 Address Record)

- Maps a hostname to an IPv6 address
- Functionally similar to an A record but for IPv6

Important Clarification:

- Written as AAAA (quad-A)
- Not to be confused with:
 - Battery sizes (AA, AAA)
 - AAA (Authentication, Authorization, Accounting), which is unrelated to DNS

Purpose:

- Enables DNS resolution in IPv6-enabled networks

CNAME Record (Canonical Name Record)

- Maps one domain name to another domain name
- Used to create aliases for domains

Examples:

`www.example.com` → `example.com`

www.example.com → example.org

Use Cases:

- Redirecting subdomains
- Simplifying domain management
- Avoiding duplication of DNS records

MX Record (Mail Exchange Record)

- Specifies the mail server responsible for handling emails for a domain

Purpose:

- Enables email delivery by directing email traffic to the correct server

Example Scenario:

- Sending an email to test@example.com
- The sending mail server queries DNS for the MX record of example.com

How DNS Resolution Works in Practice

Web Browsing Scenario

When a user types example.com into a web browser:

- 1.The browser sends a DNS query
- 2.The DNS server is asked for the A record
- 3.The server responds with the corresponding IP address
- 4.The browser uses this IP address to connect to the web server

Email Delivery Scenario

When an email is sent to `test@example.com`:

- 1.The mail server queries DNS
- 2.**The MX record for `example.com` is requested
- 3.The DNS server returns the mail server responsible for that domain
- 4.Email delivery proceeds accordingly

DNS Lookup Using Command-Line Tools

`nslookup` Tool

The `nslookup` command is commonly used to manually query DNS servers and retrieve DNS records.

Example Command : `nslookup www.example.com`

Observed Output Highlights:

- DNS server used: `127.0.0.53`
- IPv4 address returned via A record
- IPv6 address returned via AAAA record

This demonstrates that modern DNS resolution often retrieves both IPv4 and IPv6 addresses when available.

Domain Registration, DNS Authority, and WHOIS Records

Authority Over DNS Records

For a domain name to function on the internet, its DNS records must be created and managed by an authorized entity. This authority is automatically granted to the person or organization that registers the domain name.

- The domain registrant has full control over the domain's DNS configuration
- This includes the ability to create, modify, or delete DNS records such as:
 - A records
 - AAAA records
 - MX records
 - Other valid DNS record types
- For example, if a user registers `example.com`, they are legally and technically allowed to define how `example.com` resolves and behaves on the internet

Domain Name Registration Process

Registration Duration and Cost

- Domain names can be registered for one or more years
- Registration requires payment of an annual fee
- Continued ownership depends on timely renewal

Registrant Contact Information

When registering a domain, the registrant must provide accurate personal or organizational information, including:

- Name
- Phone number
- Email address
- Physical address

This information is stored as part of the domain's official registration data.

WHOIS Records

A WHOIS record is a publicly accessible database entry that contains information about a registered domain name and its owner.

Important clarification:

- Despite being written in uppercase, WHOIS is not an acronym
- It is pronounced "who is", reflecting its purpose of identifying domain ownership

Information Contained in WHOIS Records

A typical WHOIS record may include:

- Domain name
- Domain creation date
- Last update date
- Domain expiration date
- Registrar name and contact details
- Registrant's name, organization, address, phone number, and email

These records allow transparency and accountability in domain ownership.

Privacy Protection for WHOIS Data

Because WHOIS records are publicly accessible, registrants may not wish to expose their personal information.

Privacy Protection Services

- Domain registrars offer WHOIS privacy protection
- When enabled:
 - The registrant's real personal details are hidden
 - Placeholder information from a privacy service provider is displayed instead
- This helps protect registrants from:
 - Spam
 - Identity theft
 - Unwanted contact

Querying WHOIS Information

WHOIS records can be queried using:

- Online WHOIS lookup services
- Command-line tools such as `whois` (commonly available on Linux systems)

Command-Line WHOIS Lookup

Using the `whois` command returns structured information about a domain's registration status and ownership.

When privacy protection is enabled, the output typically shows:

- Generic registrant names such as *"Registration Private"*
- A privacy service organization (e.g., a proxy company)
- Registrar details remain visible

Registrar and Registry Information

WHOIS output also includes technical and administrative details such as:

- Registrar name (the company through which the domain was registered)
- Registrar WHOIS server
- Registrar URL
- Registrar IANA ID
- Abuse contact email and phone number

These details are important for:

- Domain administration
- Reporting abuse
- Legal or technical investigations

Lifecycle Information of a Domain

WHOIS records provide timestamps that describe the domain's lifecycle:

- Creation Date: When the domain was first registered
- Updated Date: When the record was last modified
- Expiration Date: When the domain registration will expire if not renewed

This information helps determine:

- Domain age
- Registration validity
- Risk of domain expiration

Entities Responsible for Domain Name Registration and Global DNS Management

Domain name registration is handled through a hierarchical system of organizations, each with clearly defined roles. No single company owns or controls all domain names.

The key entities involved are :

1. ICANN (Internet Corporation for Assigned Names and Numbers)
2. Registries
3. Registrars
4. DNS Root Server Operators

Each entity performs a specific function to ensure the internet works in a coordinated and globally consistent manner.

ICANN: The Global Coordinator

ICANN (Internet Corporation for Assigned Names and Numbers) is a non-profit international organization responsible for coordinating the global internet naming system.

ICANN does not sell domain names and does not own them. Instead, it provides governance and coordination.

ICANN's Responsibilities

- Managing the global DNS namespace
- Overseeing Top-Level Domains (TLDs) such as:
 - .com, .org, .net
 - Country-code TLDs like .in, .uk, .us

- Accrediting domain registrars
- Delegating authority to registries
- Ensuring uniqueness of domain names worldwide

Without ICANN, two people could register the same domain name independently, which would break the internet.

Registries: Owners of Top-Level Domains

What Is a Registry?

A registry is an organization responsible for managing a specific Top-Level Domain (TLD).

Examples:

- .com → Managed by a registry company
- .org → Managed by a different registry
- .in → Managed by the Indian national registry

Each TLD has exactly one registry.

Responsibilities of a Registry

A registry:

- Maintains the authoritative database of all domain names under its TLD
- Stores:
 - Registered domain names
 - Their name servers
- Operates the authoritative DNS servers for the TLD
- Enforces policies set in coordination with ICANN

Registries do not interact directly with end users.

Registrars: The Companies You Buy Domains From

What Is a Registrar?

A registrar is a company authorized (accredited) by ICANN to sell domain names to the public.

Examples include:

- Commercial domain providers
- Hosting companies offering domain services

Responsibilities of a Registrar

A registrar:

- Provides a user interface to:
 - Search domain availability
 - Register domains
 - Renew domains
- Collects registrant information
- Sends registration data to the appropriate registry
- Offers additional services such as:
 - DNS management
 - WHOIS privacy protection

Registrars act as intermediaries between users and registries.

How Do These Entities Get Their Authority ?

The authority flows as follows:

Global Internet Community → ICANN → Registries → Registrars → End Users

- ICANN operates with legitimacy granted by:
 - Internet stakeholders

- Governments
- Technical organizations
- Registries receive authority from ICANN through formal agreements
- Registrars receive accreditation from ICANN and contracts with registries

This system is based on global consensus, not ownership or force.

How Is DNS Managed and Synchronized Globally?

DNS is designed as a hierarchical and distributed system, not a single centralized database.

Hierarchy order:

1. Root DNS servers
2. Top-Level Domain servers (e.g., .com, .org)
3. Authoritative name servers for individual domains

Each level only knows what it needs to know.

Root DNS Servers and Global Synchronization

Root Servers

- There are 13 logical root server identifiers
- Each is operated by trusted organizations
- Physically replicated worldwide using anycast

How Synchronization Works

- Root zone data is maintained centrally under ICANN coordination
- Changes are:
 - Carefully reviewed
 - Cryptographically signed

- Distributed to root server operators
- DNS resolvers worldwide query their nearest root server instance
- This ensures:
 - Low latency
 - High availability
 - Global consistency

Why This System Scales Globally

The system works globally because:

- Responsibility is distributed
- Authority is clearly defined
- Data is replicated, not centralized
- DNS uses caching, reducing load and delay
- Standards are enforced through open protocols

No single failure can bring the entire DNS system down.

HTTP and HTTPS : Web Communication Fundamentals

When a user accesses a website through a web browser, the communication between the browser and the web server is primarily carried out using the HTTP or HTTPS protocol.

- HTTP stands for Hypertext Transfer Protocol
- HTTPS stands for Hypertext Transfer Protocol Secure

These protocols define how web clients (browsers) and web servers exchange data over a network.

Relationship Between HTTP/HTTPS and TCP

HTTP and HTTPS operate on top of the Transmission Control Protocol (TCP).

Key Characteristics:

- TCP provides:
 - Reliable data delivery
 - Ordered transmission
 - Error checking and retransmission
- HTTP relies on TCP to ensure that web content arrives correctly and completely

Thus, HTTP itself does not guarantee reliability; it depends on TCP for these services.

Common TCP Ports Used by HTTP and HTTPS

Web servers listen for incoming client connections on specific TCP ports.

Standard Ports:

- Port 80 → HTTP
- Port 443 → HTTPS

Alternative (Less Common) Ports:

- Port 8080 → Alternative HTTP
- Port 8443 → Alternative HTTPS

Using non-standard ports is common in testing, development, or restricted environments.

HTTP Request Methods (Commands)

HTTP defines a set of request methods that specify the action the client wants the server to perform. These methods are sometimes referred to as HTTP verbs.

GET Method

- Used to retrieve data from a server
- Commonly used to request:
 - Web pages (HTML)
 - Images
 - Stylesheets
 - Scripts

Characteristics:

- Does not modify server data
- Most frequently used HTTP method

POST Method

- Used to submit new data to the server
- Common use cases include:
 - Submitting web forms
 - Uploading files

Characteristics:

- Sends data in the request body
- Often results in server-side processing or data storage

PUT Method

- Used to create or update a resource on the server
- If the resource exists, it may be overwritten
- If it does not exist, it may be created

Characteristics:

- Typically used in APIs
- Requires careful permission control

DELETE Method

- Used to remove a resource from the server

Characteristics:

- Deletes a specified file or object
- Common in RESTful services
- Rarely exposed directly to end users via browsers

Manual HTTP Communication Using Telnet

It is possible to interact directly with a web server using a basic TCP client, such as telnet, to manually send HTTP requests.

Required Request Components:

- A request line, for example:

`GET / HTTP/1.1`

- A Host header, for example:

`Host: example.com`

Some servers may respond without the Host header, but many modern servers require it.

Requesting Specific Resources

HTTP allows requesting specific files or paths on a server.

Example:

- To request a file named `file.html`, the client sends:

`GET /file.html HTTP/1.1`

Depending on server configuration, shorter forms may also work.

Why Manual HTTP Interaction Is Useful

Directly sending HTTP requests is valuable for:

- Troubleshooting server issues
- Understanding how HTTP works internally
- Debugging incorrect server responses
- Learning protocol behavior without browser abstraction

This process is often described as “talking HTTP” directly with the server.

File Transfer Protocol (FTP)

The File Transfer Protocol (FTP) is a network protocol specifically designed for transferring files between a client and a server over a network.

Unlike HTTP, which is primarily designed to retrieve and display web content, FTP focuses entirely on efficient and structured file transfer. Because of this specialized design, FTP can often achieve higher transfer speeds than HTTP when network conditions are similar.

Purpose and Characteristics of FTP

Key Characteristics :

- Optimized for uploading and downloading files
- Uses TCP to ensure reliable data transfer
- Separates control commands from data transfer
- Supports both text and binary file transfers

FTP is commonly used in scenarios such as:

- Downloading files from public servers
- Uploading files to remote systems
- Managing files on remote servers

FTP Commands

FTP defines a set of standardized commands that the client uses to communicate with the server.

Common FTP Commands:

USER

- Used to provide the username for authentication
- Sent at the beginning of an FTP session

PASS

- Used to provide the password corresponding to the username
- May be optional in certain cases, such as anonymous access

RETR (Retrieve)

- Used to download a file from the FTP server to the client

STOR (Store)

- Used to upload a file from the client to the FTP server

These commands are exchanged over the control connection.

FTP Ports and Connections

Control Connection

- FTP servers listen on TCP port 21 by default
- Used for:
 - Authentication
 - Command exchange
 - Server responses

This connection remains open for the duration of the session.

Data Connections

- File transfers and directory listings occur over separate TCP connections
- A new data connection is created for:
 - Each directory listing
 - Each file transfer

This separation is a defining feature of FTP.

FTP Login and Anonymous Access

FTP supports multiple authentication modes.

Anonymous FTP:

- Username: anonymous
- Password: typically not required or left blank
- Used to allow public access to files

This is commonly used for distributing publicly available data.

FTP Client Interaction Workflow

A typical FTP session follows these steps:

1. Client connects to the FTP server on port 21
2. Server responds with a welcome message
3. Client sends username using USER
4. Client sends password using PASS
5. Authentication is completed
6. Client issues commands to interact with files and directories

Directory Listing and File Transfer

Listing Files

- Client command: `ls`
- Protocol command sent to server: `LIST`
- Server responds by opening a data connection
- Directory contents are sent over this data connection

Downloading Files

- Client command: `get filename`
- Protocol command sent to server: `RETR filename`
- A separate data connection is opened
- File contents are transmitted
- Data connection is closed after completion

Transfer Modes

FTP supports different transfer modes depending on file type.

ASCII Mode

- Used for text files
- Converts line endings to match the receiving system
- May produce warnings if file formatting differs

Binary Mode

- Used for non-text files
- Transfers data without modification
- Default mode for most FTP clients

Incorrect mode selection can result in corrupted files.

Observations from Network Traffic Analysis

Analysis of FTP traffic reveals several important behaviors:

- Client-issued commands differ from protocol-level commands
 - Example: `ls` → `LIST`
- Client and server messages are clearly separated
- Each directory listing and file transfer uses a new data connection
- Control traffic and data traffic are independent

This design improves flexibility but increases protocol complexity.

Advantages and Limitations of FTP

Advantages:

- Efficient file transfer
- Supports large files
- Clear separation of commands and data

Limitations:

- Credentials are sent in plaintext
- Multiple connections complicate firewall configuration
- Lacks built-in encryption

Because of these limitations, FTP is often replaced by secure alternatives in sensitive environments.

Email Protocols : SMTP, POP3, and IMAP

Email communication on the internet is not handled by a single protocol. Instead, it relies on multiple specialized protocols, each designed for a specific role in the email lifecycle.

The three primary email-related protocols are:

- SMTP → Sending emails
- POP3 → Retrieving emails (download-focused)
- IMAP → Retrieving and synchronizing emails (server-focused)

Together, these protocols enable end-to-end email communication.

Simple Mail Transfer Protocol (SMTP)

Purpose of SMTP

The Simple Mail Transfer Protocol (SMTP) defines:

- How an email client sends mail to a mail server
- How one mail server transfers mail to another

SMTP is responsible only for sending email, not retrieving it.

Conceptual Analogy

SMTP can be compared to sending a package at a post office:

- You identify yourself (sender)
- Specify the recipient
- Hand over the content
- The postal system handles delivery

Similarly, SMTP transfers responsibility for email delivery to the mail server.

- SMTP servers listen on TCP port 25 by default

Core SMTP Commands

SMTP is a text-based protocol, meaning commands and responses are sent as readable text.

Command	Purpose
HELO / EHLO	Initiates the SMTP session
MAIL FROM:	Specifies the sender's email address
RCPT TO:	Specifies the recipient's email address
DATA	Signals the start of the email content
.	Marks the end of the email message

SMTP Session Flow

A typical SMTP session follows this sequence:

1. Client connects to SMTP server
2. Client initiates the session (HELO or EHL0)
3. Sender address is provided
4. Recipient address is provided
5. Email content is transmitted
6. Session is terminated

Manual SMTP sessions can be performed using basic TCP tools, which helps expose how email clients operate internally.

Why SMTP Alone Is Not Enough

SMTP does not support downloading emails. Once an email is delivered to the recipient's mail server, another protocol is required for the recipient to access it.

This is where POP3 and IMAP are used.

Post Office Protocol Version 3 (POP3)

Purpose of POP3

The Post Office Protocol version 3 (POP3) is designed to:

- Allow an email client to retrieve emails from a mail server
- Typically download and remove emails from the server

POP3 focuses on offline access and simplicity.

Conceptual Analogy

- SMTP → Dropping mail into a post office box
- POP3 → Checking your personal mailbox and taking mail home
- POP3 servers listen on TCP port 110 by default

Common POP3 Commands

Command	Purpose
USER	Identifies the user
PASS	Provides the user's password
STAT	Displays message count and total size
LIST	Lists all messages and their sizes
RETR	Retrieves a specific message
DELE	Marks a message for deletion
QUIT	Ends the session and applies changes

POP3 Behavior Characteristics

- Emails are usually deleted from the server after retrieval
- Suitable when:
 - Email is accessed from a single device
 - Server storage should be minimized
- Authentication is required before access

Security Implications of POP3 and SMTP

Because both SMTP and POP3 are plaintext protocols by default:

- Usernames and passwords are sent in readable form
- Anyone capturing network traffic can view credentials and email content

This makes unencrypted POP3 and SMTP unsuitable for untrusted networks.

Internet Message Access Protocol (IMAP)

Purpose of IMAP

The Internet Message Access Protocol (IMAP) is designed to:

- Allow email access from multiple devices
- Keep emails stored on the server
- Synchronize:
 - Read status
 - Deleted messages
 - Folder organization

IMAP emphasizes centralized storage and synchronization.

Conceptual Comparison with POP3

Feature	POP3	IMAP
Email storage	Client	Server
Multi-device use	Poor	Excellent
Synchronization	No	Yes
Server storage usage	Low	High

- IMAP servers listen on TCP port 143 by default

Common IMAP Commands

Command	Purpose
LOGIN	Authenticates the user
SELECT	Selects a mailbox
FETCH	Retrieves message data
MOVE	Moves messages between folders
COPY	Copies messages
LOGOUT	Ends the session

IMAP commands are more complex because they manage state and synchronization.

IMAP Session Characteristics

An IMAP session typically involves:

- 1.Authentication
- 2.Mailbox selection
- 3.Message retrieval or manipulation
- 4.Synchronization of mailbox state
- 5.Logout

Unlike POP3, IMAP allows messages to remain on the server after retrieval.

Protocol Design Insights

From examining SMTP, POP3, and IMAP together, several protocol design principles emerge:

- Each protocol has a single, well-defined responsibility
- Protocols are text-based, making them human-readable and debuggable
- Client-server communication follows a command-response model
- Increased functionality leads to greater protocol complexity

Room 04 : Networking Secure Protocols

Introduction to Secure Networking Protocols

In early computer networks and the early Internet, most communication protocols were designed with functionality, not security, as the main priority. While these protocols worked well for browsing websites or sending emails, they did not protect sensitive information. This created serious risks as the Internet grew and began handling financial transactions, private communications, and personal data.

To address these risks, secure protocols were introduced. This section explains why security is necessary, what problems existed in older protocols, and how modern solutions such as TLS and SSH protect data today.

Core Security Problems in Early Network Protocols

Traditional networking protocols (such as HTTP, SMTP, POP3, IMAP, and TELNET) transmit data in plaintext, meaning the data is sent exactly as written, without protection.

This lack of protection affects three critical security principles :

1. Confidentiality

Confidentiality ensures that only the intended recipient can read the data.

- Without confidentiality, anyone monitoring the network traffic can:
 - Read usernames and passwords
 - View credit card details
 - Access private emails and documents
- For example, when using HTTP instead of HTTPS, login credentials can be captured by attackers.

2. Integrity

Integrity ensures that data is not altered during transmission.

- Without integrity protection:
 - An attacker can modify data in transit
 - For example, a payment authorization of £100 could be changed to £800 without the sender or receiver noticing

3. Authenticity

Authenticity ensures that you are communicating with the correct and legitimate server.

- Without authenticity:
 - Users can be tricked into connecting to fake or malicious servers
 - This enables phishing attacks and data theft

Without these three properties, online banking, shopping, email, and messaging would be unsafe.

Packet Sniffing and Promiscuous Mode (Historical Context)

In the past, attackers could easily steal sensitive information using packet-capturing tools.

- Network interface cards could be set to promiscuous mode
- This allowed them to capture:
 - All network traffic
 - Including packets not intended for their device
- Since data was un-encrypted :
 - Login credentials were visible in captured packets
 - Users had no way to protect themselves

Today, this type of attack is far less effective because most services encrypt data by default.

Introduction to SSL and TLS

SSL (Secure Sockets Layer)

- Developed by Netscape Communications in the early 1990s
- First public version: SSL 2.0 (1995)
- Designed to secure web communications

TLS (Transport Layer Security)

- Developed by the Internet Engineering Task Force (IETF)
- Introduced in 1999 as an upgrade to SSL
- TLS 1.0 improved upon SSL 3.0
- Latest major version: TLS 1.3 (released in 2018)

The key takeaway is not the dates, but the fact that TLS has evolved over decades to fix weaknesses and improve security.

What TLS Does ?

TLS is a cryptographic protocol that operates at the Transport Layer of the OSI model.

It provides:

- Confidentiality → encrypts data so it cannot be read
- Integrity → prevents data from being modified
- Authenticity → verifies the identity of servers (and sometimes clients)

TLS makes secure Internet activities possible, such as:

- Online shopping
- Online banking
- Secure email
- Private messaging

Without TLS, these everyday activities would not be safe.

Securing Existing Protocols Using TLS

Rather than creating entirely new protocols, TLS is added on top of existing ones.

This results in secure versions of familiar protocols :

Original Protocol	Secure Version	Purpose
HTTP	HTTPS	Secure web browsing
SMTP	SMTPS	Secure email sending
POP3	POP3S	Secure email retrieval
IMAP	IMAPS	Secure email synchronization
DNS	DoT (DNS over TLS)	Secure domain lookups
MQTT	MQTTS	Secure IoT messaging
SIP	SIPS	Secure VoIP signaling

The appended “S” stands for Secure, indicating the use of SSL/TLS.

Secure Remote Access: TELNET vs SSH

TELNET (Insecure)

- Sends all data, including passwords, in plaintext
- Easily exploitable by attackers
- Considered unsafe for modern use

SSH (Secure Shell)

- Created as a secure replacement for TELNET
- Encrypts all communication
- Provides:
 - Secure remote login
 - Secure file transfer

- Secure tunneling for other protocols
- SSH is also extensible, meaning it can support additional security features

TLS Certificates and Certificate Authorities (CAs)

Digital Certificates

To prove its identity, a server (or client) uses a TLS certificate.

Certificate Signing Process

1. Server administrator creates a Certificate Signing Request (CSR)
2. CSR is submitted to a Certificate Authority (CA)
3. CA verifies the request
4. CA issues a digitally signed certificate

This certificate allows others to verify the server's identity.

Trusted Certificate Authorities

- Operating systems and web browsers come with pre-installed trusted CAs
- If a certificate is signed by one of these trusted authorities, it is accepted automatically
- This is similar to trusting official stamps or seals in the physical world

Free vs Paid Certificates

- Traditionally, certificates required an annual payment
- Let's Encrypt provides:
 - Free TLS certificates
 - Widely trusted
 - Automated issuance and renewal
- This has significantly increased the adoption of HTTPS

Self-Signed Certificates

Some users create self-signed certificates.

Limitations:

- Not signed by a trusted CA
- Cannot prove authenticity to others
- Browsers and clients display warnings
- Suitable only for:
 - Testing
 - Internal or private systems

Architecture and Internals of Transport Layer Security (TLS)

Transport Layer Security (TLS) is a cryptographic protocol designed to secure communication between two parties—typically a client (such as a web browser) and a server (such as a web server). It operates above the transport layer (TCP) and below application protocols like HTTP, SMTP, or IMAP.

TLS does not replace these protocols; instead, it wraps around them, protecting the data they exchange.

High-Level TLS Architecture

TLS is composed of several logical components that work together:

1. TLS Handshake Protocol
2. TLS Record Protocol
3. Cryptographic Primitives
 - Symmetric encryption

- Asymmetric encryption
- Hash functions
- Message authentication codes (MACs) or AEAD ciphers

4. Certificate and Trust Infrastructure

Each component plays a role in achieving confidentiality, integrity, and authenticity.

TLS Handshake Protocol (Establishing Trust and Keys)

The TLS handshake is the initial phase of a TLS connection. Its purpose is to:

- Authenticate the server (and optionally the client)
- Agree on cryptographic algorithms
- Securely establish shared encryption keys

What Happens During the Handshake (Conceptual View)

1. Client Hello

- The client tells the server:
 - Which TLS versions it supports
 - Which encryption algorithms it supports
 - A random value for key generation

2. Server Hello

- The server selects:
 - TLS version
 - Encryption algorithms
- Sends its digital certificate to prove its identity

3. Certificate Verification

- The client verifies:
 - The certificate is signed by a trusted Certificate Authority (CA)
 - The certificate belongs to the intended server
 - The certificate is not expired or revoked

4. Key Exchange

- The client and server securely agree on a shared secret
- Asymmetric cryptography is used only at this stage

5. Handshake Completion

- Both sides derive identical session keys
- Secure communication begins

After the handshake, asymmetric cryptography is no longer used due to performance cost.

TLS Record Protocol (Protecting Data in Transit)

Once the handshake is complete, all application data passes through the TLS Record Protocol.

Responsibilities of the Record Protocol

- Fragment data into manageable chunks
- Encrypt data
- Protect data integrity
- Transmit securely over TCP

This is the component that actively protects every byte of transmitted data.

Cryptographic Building Blocks in TLS

TLS relies on multiple cryptographic techniques, each serving a specific purpose.

A. Asymmetric Cryptography (Public-Key Cryptography)

Used for:

- Authentication
- Secure key exchange (during handshake only)

How it works conceptually:

- The server has:
 - A public key (shared openly)
 - A private key (kept secret)
- Data encrypted with the public key can only be decrypted with the private key

Why TLS uses it:

- To prove the server's identity
- To securely establish shared session keys over an insecure network

B. Symmetric Cryptography (Session Encryption)

Used for:

- Encrypting actual data after the handshake

Why symmetric encryption is used:

- Much faster than asymmetric encryption
- Suitable for large volumes of data

Key point:

- Both client and server use the same secret key
- This key is generated securely during the handshake

C. Hash Functions

Used for:

- Data integrity
- Certificate validation
- Key derivation

Properties:

- One-way (cannot be reversed)
- Small input changes produce large output changes

Hashes ensure that any modification of data can be detected.

D. Message Authentication (MAC / AEAD)

Modern TLS versions (especially TLS 1.3) use AEAD (Authenticated Encryption with Associated Data) ciphers.

These provide:

- Encryption (confidentiality)
- Authentication (integrity and authenticity) in one operation

This prevents attackers from:

- Modifying encrypted data
- Replaying old packets
- Injecting malicious data

How TLS Protects the Three Core Security Properties

1. Confidentiality (Preventing Eavesdropping)

TLS ensures confidentiality by:

- Encrypting all application data using symmetric encryption
- Using session keys known only to the client and server

Even if an attacker:

- Captures all packets
- Has powerful packet-sniffing tools

They will only see unreadable ciphertext, not plaintext data.

2. Integrity (Preventing Data Modification)

TLS ensures integrity by:

- Applying cryptographic authentication (MAC or AEAD)
- Attaching integrity checks to each data record

If data is:

- Modified
- Corrupted
- Injected by an attacker

The receiving side will:

- Detect the change
- Reject the packet
- Terminate the connection if necessary

This prevents attacks such as:

- Altering payment amounts

- Modifying commands
- Injecting malicious content

3. Authenticity (Verifying Identity)

TLS ensures authenticity through:

Server Authentication

- Digital certificates signed by trusted Certificate Authorities
- Public key validation during the handshake

This confirms:

- You are communicating with the real server
- Not an attacker impersonating it

Optional Client Authentication

- Some TLS configurations also authenticate the client
- Common in:
 - Corporate systems
 - VPNs
 - Internal services

Why TLS Is Considered Secure by Design

TLS achieves security through layered defense:

- Trust is established before data transfer
- Keys are negotiated securely
- Encryption is strong and standardized
- Integrity is continuously verified
- Identity is validated using a global trust infrastructure

Even if attackers:

- Control the network
- Capture all traffic
- Perform man-in-the-middle attempts

TLS prevents them from reading, altering, or impersonating communication.

Default TCP Ports for Insecure and Secure Protocols

The table below combines plaintext (insecure) and TLS-secured versions of common Internet protocols. This unified view highlights how the same service uses different ports depending on whether encryption is enabled.

Service	Insecure Protocol (Port)	Secure Protocol (Port)	Purpose
Web browsing	HTTP (80)	HTTPS (443)	Accessing websites
Email sending	SMTP (25)	SMTPS (465, 587)	Sending emails securely
Email retrieval (download)	POP3 (110)	POP3S (995)	Downloading emails
Email access (sync)	IMAP (143)	IMAPS (993)	Accessing and syncing emails

Secure Shell (SSH): Secure Remote Access

TELNET was one of the earliest protocols used for remote login and system administration. While convenient, it has a critical security flaw :

- All communication is sent in cleartext
- Usernames and passwords can be easily captured
- Attackers monitoring network traffic can:
 - Steal login credentials
 - Observe commands
 - Modify data in transit

Because TELNET offers no confidentiality, integrity, or authenticity, it is considered unsafe for modern networks.

- TELNET default port: 23

This security weakness made a secure alternative necessary.

Introduction to SSH

Secure Shell (SSH) was developed by Tatu Ylönen and released in 1995 as a secure replacement for TELNET.

Key milestones:

- SSH-1 (1995): First public version
- SSH-2 (1996): More secure, redesigned protocol
- OpenSSH (1999): Open-source implementation released by the OpenBSD project

Today, OpenSSH is the most widely used SSH implementation and forms the basis of most SSH clients and servers.

- SSH default port: 22

Core Security Goals of SSH

SSH provides secure remote access by enforcing:

- Confidentiality → encrypts all traffic
- Integrity → prevents data tampering
- Authenticity → verifies server and client identities

Key Features of OpenSSH

Secure Authentication

SSH supports multiple authentication methods:

- Password-based authentication
- Public key authentication (most common and most secure)
- Two-factor authentication (where configured)

Public key authentication allows users to log in without transmitting a password over the network.

Confidentiality (Encryption)

- SSH encrypts all communication end-to-end
- Prevents attackers from reading:
 - Commands
 - Output
 - Files
- SSH warns users when a server's public key changes, helping detect man-in-the-middle attacks

Integrity Protection

- Cryptographic integrity checks ensure:
 - Data is not altered during transmission
 - Injected or modified packets are rejected

SSH Tunneling (Port Forwarding)

SSH can create secure tunnels to carry other protocols inside an encrypted SSH session.

- Local port forwarding
- Remote port forwarding
- Dynamic port forwarding (SOCKS proxy)

This can:

- Secure insecure protocols
- Create a VPN-like secure communication channel

X11 Forwarding

- Allows graphical applications running on a remote Unix-like system to be displayed locally
- All graphical data is encrypted through SSH
- Commonly used for remote administration of GUI tools

Using SSH (Command Overview)

To connect to a remote server :

ssh username@hostname

If the username matches the local system user:

ssh hostname

Authentication behavior :

- Password-based → prompts for password
- Public key-based → logs in automatically

SSH Architecture (Internal Components)

SSH is not a single protocol but a suite of layered protocols working together.

SSH consists of three main layers:

- 1.** Transport Layer Protocol
- 2.** User Authentication Protocol
- 3.** Connection Protocol

SSH Transport Layer Protocol

Purpose

- Establishes a secure channel
- Handles encryption, integrity, and server authentication

How It Works

- Negotiates cryptographic algorithms
- Performs key exchange (e.g., Diffie-Hellman)
- Authenticates the server using its public key
- Establishes shared session keys

This layer is responsible for confidentiality and integrity.

SSH User Authentication Protocol

Purpose

- Authenticates the client to the server

Supported Methods

- Password authentication
- Public key authentication
- Keyboard-interactive authentication
- Multi-factor authentication

Public key authentication is preferred because:

- No password is transmitted
- Strong cryptographic proof of identity is used

SSH Connection Protocol

Purpose

- Manages multiple secure channels over a single SSH connection

Examples of Channels

- Interactive shell sessions
- File transfer (SCP, SFTP)
- Port forwarding (tunneling)
- X11 forwarding

This design allows multiple services to run securely at the same time within one SSH session.

How SSH Works Internally (Step-by-Step)

Step 1: Client Initiates Connection

- Client connects to server on TCP port 22

Step 2: Server Identification

- Server sends its public host key
- Client checks against known host keys

Step 3: Key Exchange

- Secure key exchange algorithm establishes shared session keys

Step 4: Secure Channel Established

- All communication is encrypted and integrity-protected

Step 5: Client Authentication

- Client proves identity using password or public key

Step 6: Secure Session Use

- Commands, file transfers, tunnels, and GUI applications run securely

How SSH Protects the Three Security Properties

Confidentiality

- Strong encryption ensures no one can read the traffic

Integrity

- Cryptographic checks detect any data modification

Authenticity

- Server identity is verified via host keys
- Client identity is verified via authentication mechanisms

TELNET vs SSH (Conceptual Comparison)

Feature	TELNET	SSH
Encryption	None	Yes
Credential protection	No	Yes
Integrity checks	No	Yes
Authentication	Password only	Multiple methods
Secure tunneling	No	Yes

Secure File Transfer : SFTP and FTPS

SFTP (SSH File Transfer Protocol)

SFTP stands for SSH File Transfer Protocol. It is a secure file transfer mechanism built directly into the SSH protocol suite.

- Uses SSH for security
- Runs on TCP port 22
- Provides encrypted file upload and download
- Not related to traditional FTP internally (despite the name)

How SFTP Works Internally (Architecture)

SFTP operates inside an SSH session, which means :

1. An SSH connection is established between client and server
2. SSH performs:
 - Server authentication
 - Key exchange
 - Session encryption
3. Once the secure SSH channel exists:
 - SFTP runs as a subsystem inside that channel
 - File transfer commands are sent securely

Key Architectural Characteristics

- Single encrypted connection
- No separate data and control channels
- Benefits from all SSH security mechanisms:
 - Encryption
 - Integrity protection
 - Strong authentication

Using SFTP

Connection command : `sftp username@hostname`

Common commands:

- `get filename` → download file
- `put filename` → upload file

SFTP commands are Unix-like, not FTP-like, which can feel different to users familiar with traditional FTP.

Why SFTP Is Secure

SFTP provides:

- Confidentiality → files are encrypted
- Integrity → file contents cannot be altered unnoticed
- Authenticity → server identity is verified using SSH host keys

Because it relies on SSH, no additional certificates are required.

FTPS (FTP Secure)

FTPS stands for File Transfer Protocol Secure.

- Traditional FTP secured using TLS
- Conceptually similar to HTTPS
- Requires TLS certificates

How FTPS Works Internally

FTPS uses two separate connections:

1. Control channel – for commands
2. Data channel – for file transfer

Security is added by wrapping these channels in TLS encryption.

Key Architectural Characteristics

- Uses TLS instead of SSH
- Requires certificate configuration
- More complex firewall handling
- Typically uses:
 - FTP: port 21
 - FTPS: port 990 (implicit TLS)

SFTP vs FTPS Comparison

Feature	SFTP	FTPS
Security protocol	SSH	TLS
Default port	22	990
Number of connections	One	Multiple
Certificate required	No	Yes
Firewall friendliness	High	Lower
Ease of setup	Simple	More complex

Key Takeaway (File Transfer)

- SFTP is simpler, more firewall-friendly, and easier to deploy
- FTPS integrates with TLS-based infrastructures but is more complex
- SFTP is often preferred in modern environments

Virtual Private Networks (VPNs)

A Virtual Private Network (VPN) allows devices in different locations to connect securely over the public Internet as if they were part of the same private network.

VPN focuses on:

- Virtual → uses public infrastructure
- Private → traffic is encrypted and protected

Why VPNs

The Internet (TCP/IP) was designed for reliability, not security.

TCP/IP:

- Ensures packets arrive
- Resends lost data
- Adapts to network failures

But it does not:

- Encrypt traffic
- Prevent eavesdropping
- Prevent data modification

VPNs fill this security gap.

VPN Architecture (Conceptual)

A VPN consists of :

- VPN client (user device or branch gateway)
- VPN server (central or corporate location)
- Encrypted tunnel between them

Site-to-Site VPN

- Connects entire networks (branch offices)
- All devices communicate securely
- Appears as one internal network

Remote Access VPN

- Connects individual users
- Common for remote employees
- Secures traffic from a single device

How a VPN Works Internally

- 1.VPN client authenticates to VPN server
- 2.Cryptographic keys are exchanged
- 3.An encrypted tunnel is created
- 4.Traffic inside the tunnel is:
 - Encrypted before transmission
 - Decrypted only at the VPN endpoint

Traffic outside the tunnel remains normal Internet traffic.

Traffic Routing Through a VPN

In many VPN setups:

- All Internet traffic is routed through the VPN tunnel
- External services see the VPN server's IP address
- User's real IP address is hidden

This enables:

- Access to private company resources
- Circumventing geographic restrictions
- Reduced ISP visibility into user activity

VPNs and Geolocation

When connected to a VPN server:

- Websites detect the VPN server's location
- Content may be localized accordingly
- Example:
 - VPN server in Japan → services behave as if the user is in Japan

Room 05 : Wireshark - The Basics

Wireshark is an open-source, cross-platform network packet analyzer used to capture, inspect, and analyze network traffic in real time or from saved capture files (PCAP files).

It allows analysts to:

- Observe communication between devices
- Inspect packet-level data
- Analyze protocols
- Troubleshoot network or security issues

Wireshark operates at a very granular level — it examines individual packets traveling across a network.

Understanding PCAP Files

A PCAP (Packet Capture) file stores captured network traffic. It is essentially a recording of network communication that can be reviewed later.

Wireshark can:

- Open existing PCAP files
- Capture live traffic and save it as PCAP
- Merge multiple PCAP files

Core Use Cases of Wireshark

Wireshark is widely used in networking and cybersecurity for :

1) Network Troubleshooting

- Detecting network congestion
- Identifying bottlenecks
- Finding misconfigured devices
- Analyzing dropped packets

2) Security Monitoring & Investigation

- Detecting rogue hosts
- Identifying unusual port usage
- Spotting suspicious traffic patterns
- Investigating potential attack activity

3) Protocol Analysis & Learning

- Understanding protocol behavior
- Viewing response codes
- Inspecting payload data
- Studying TCP handshakes, DNS queries, HTTP responses, etc.

Note : Wireshark is NOT an Intrusion Detection System (IDS). It does not automatically block threats, does not modify packets or does not actively alert on attacks. It only captures, displays and analyzes

All conclusions depend entirely on the analyst's knowledge and investigative skills.

Wireshark Graphical User Interface (GUI)

When you open Wireshark, you see an all-in-one interface designed for efficient traffic analysis.

The main interface contains five key sections :

1 Toolbar

The toolbar includes:

- Capture controls
- File operations
- Filtering tools
- Sorting options
- Export/merge features

This area gives quick access to core functions.

2 Display Filter Bar

This is one of the most powerful features.

It allows you to:

- Filter visible packets
- Apply protocol-based queries
- Narrow down large captures

Example:

- `http`
- `ip.addr == 192.168.1.10`
- `tcp.port == 80`

Display filters affect what you see, not what is captured.

3 Recent Files Section

- Lists previously opened PCAP files
- Allows quick reopening via double-click

4 Capture Filter & Interfaces

This section shows:

- Available network interfaces (e.g., eth0, lo, ens33)
- Capture filter options

What is a Network Interface?

A network interface is the connection point between a computer and a network.

Examples:

- lo → Loopback interface
- eth0 → Ethernet interface
- ens33 → Virtual interface

Capture filters determine what traffic gets captured, unlike display filters which only affect visibility.

5 Status Bar

Located at the bottom, it displays:

- Active interface
- Packet count
- Profile in use
- Processing status

6. Loading a PCAP File

You can load a PCAP file by :

- File → Open
- Drag and drop
- Double-clicking the file
- Opening from Recent Files

Once loaded, Wireshark displays detailed packet information across three panes.

Packet Analysis Layout (Three-Pane View)

After opening a capture file, Wireshark divides analysis into three major panels:

1 Packet List Pane (Top Pane)

Displays:

- Packet number
- Time
- Source IP
- Destination IP
- Protocol
- Brief information summary

This is the high-level overview.

Clicking a packet here selects it for detailed analysis.

2 Packet Details Pane (Middle Pane)

Shows:

- Layer-by-layer protocol breakdown
- Ethernet layer
- IP layer
- TCP/UDP layer
- Application layer (HTTP, DNS, etc.)

Each section can be expanded to view fields and values.

This is where deep analysis occurs.

3 Packet Bytes Pane (Bottom Pane)

Displays:

- Raw hexadecimal representation
- ASCII interpretation

When you click a field in the Details Pane, the corresponding bytes are highlighted here.

This connects protocol fields to their raw binary representation.

Packet Colouring

Wireshark automatically colors packets based on protocol or predefined rules.

Why Coloring Matters

Coloring:

- Helps quickly identify protocols
- Highlights anomalies
- Improves visual scanning of traffic

Example:

- TCP traffic might appear one color
- HTTP traffic another
- Errors another

Types of Coloring Rules

Temporary Rules

- Active only during current session
- Not saved

Permanent Rules

- Saved in profile
- Available in future sessions

Access via:

- View → Coloring Rules
- Right-click menu
- Toggle using “Colorize Packet List”

You can create custom rules using display filters.

Traffic Sniffing (Live Capture)

Wireshark allows real-time packet capture.

Capture Controls

-  Blue Shark Button → Start capturing
-  Red Button → Stop capturing
-  Green Button → Restart capture

The status bar shows:

- Active interface
- Number of packets collected

Important: You must select the correct network interface before starting.

Merging PCAP Files

Wireshark allows combining two PCAP files.

Steps:

1. Open first PCAP file
2. Navigate to: File → Merge
3. Select second file
4. Wireshark shows packet count
5. Click Open
6. Save merged file before analysis

This creates a new combined PCAP file.

Use case:

- Combining traffic from multiple time periods
- Merging related investigations

Viewing Capture File Details

When analyzing multiple files, knowing file metadata is critical.

You can access file properties via:

- Statistics → Capture File Properties
- Clicking the PCAP icon (bottom-left)

Information Available:

- File hash (for integrity verification)
- Capture start and end time
- File comments
- Interface used
- Packet statistics
- File size

This is useful for:

- Documentation
- Evidence handling
- Investigation tracking
- File validation

Packet Dissection in Wireshark

Packet dissection (also called protocol dissection) is the process of :

- Breaking down a captured network packet
- Decoding its protocol layers
- Interpreting individual fields
- Presenting structured, human-readable information

In simple terms :

Packet dissection converts raw binary data into meaningful protocol information.

Wireshark automatically performs packet dissection using built-in protocol decoders (called dissectors). It supports hundreds of protocols and even allows developers to write custom dissectors.

Dissection and the OSI Model

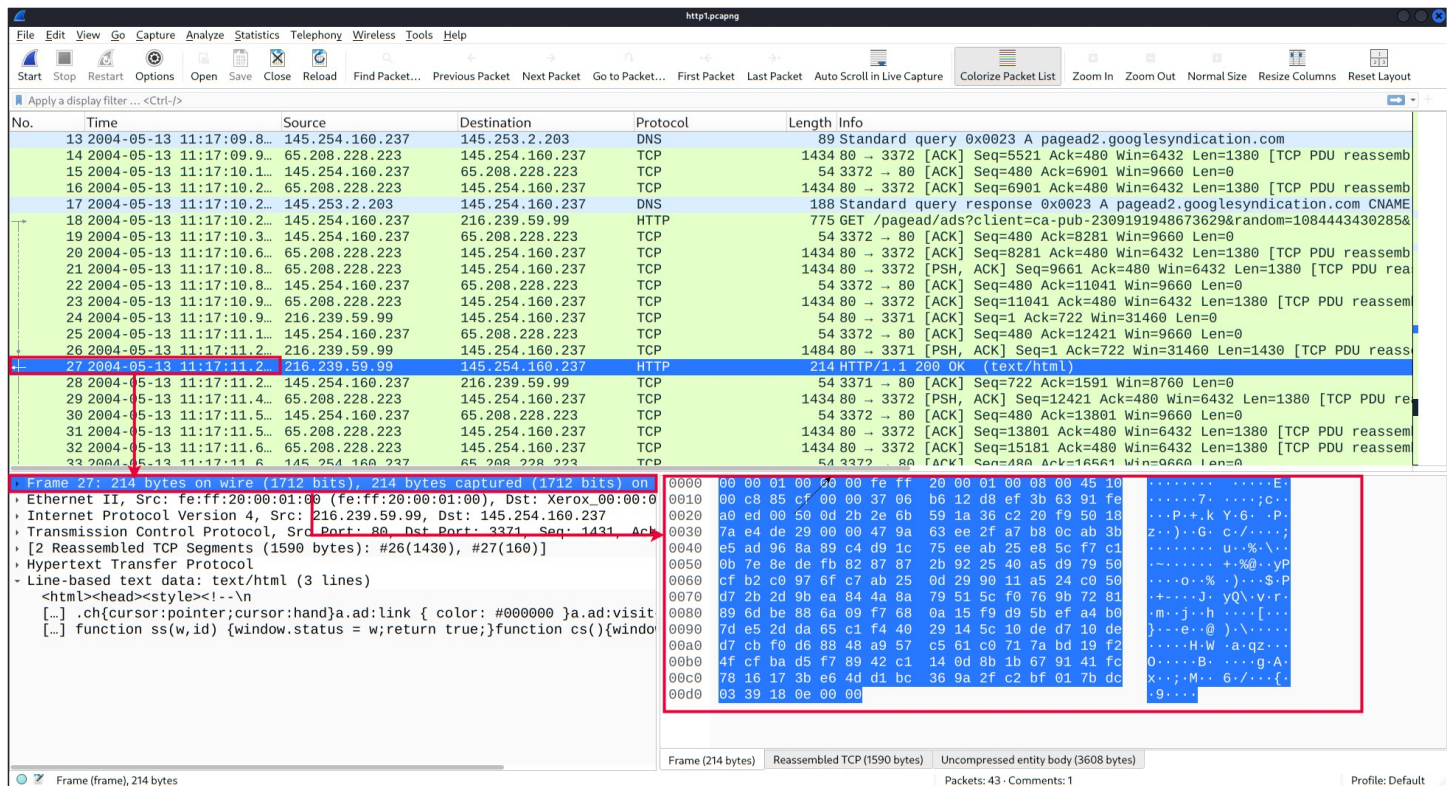
Wireshark organizes packet information based on the OSI Model (Open Systems Interconnection Model).

The OSI model divides network communication into logical layers. Each layer has a specific responsibility.

Although the OSI model contains 7 layers, captured packets typically display 5–7 layers, depending on:

- The protocol used
- Encapsulation type
- Whether additional headers exist
- Error or reassembly information

Viewing Packet Details in Wireshark



How to View Packet Details

1. Click a packet in the Packet List Pane
2. The details appear in the Packet Details Pane
3. Double-clicking opens the packet in a new window
4. Clicking any field highlights corresponding bytes in the Packet Bytes Pane

This linkage between structured fields and raw data is critical for deep analysis.

Understanding Packet Layers in Wireshark

Below is a structured explanation of each layer typically visible when analyzing an HTTP packet.

Layer 1 – Frame (Physical Layer Representation)

What It Shows:

- Frame number
- Arrival time
- Frame length (captured vs. actual)
- Interface used
- Encapsulation type

Purpose:

This layer represents metadata about how the packet was captured rather than the actual network communication itself.

Important points:

- Frame length may differ from actual wire length
- Time stamps are essential for traffic timeline analysis
- Helps detect capture truncation or anomalies

Layer 2 – Source/Destination MAC (Data Link Layer)

What It Shows:

- Source MAC address
- Destination MAC address
- EtherType (e.g., IPv4, ARP, IPv6)

Purpose:

MAC addresses identify devices **within the same local network (LAN)**.

Key Insight:

- MAC addresses change at every hop.
- They are only meaningful within a local broadcast domain.

This layer helps identify:

- Local devices
- Switch-level communication
- ARP activity
- Spoofing attempts

Layer 3 – Source/Destination IP (Network Layer)

What It Shows:

- Source IP address
- Destination IP address
- IP version
- TTL (Time to Live)
- Fragmentation details

Purpose:

IP addresses identify devices across networks.

Key Insight:

- IP addresses remain constant across hops (unless NAT is involved).
- TTL helps detect routing loops and number of hops.

This layer is critical for:

- Identifying communication endpoints
- Detecting scanning activity
- Tracking malicious IP addresses

Layer 4 – Transport Protocol (TCP/UDP)

What It Shows:

- Protocol type (TCP or UDP)
- Source port
- Destination port
- Sequence numbers (TCP)
- Acknowledgment numbers (TCP)
- Flags (SYN, ACK, FIN, RST)
- Window size

Purpose:

The transport layer manages communication between applications.

TCP:

- Connection-oriented
- Reliable
- Uses handshake and acknowledgments

UDP:

- Connectionless
- Faster
- No reliability mechanism

This layer helps detect:

- Port scanning
- Service identification
- Connection resets
- Suspicious port usage

Protocol Errors (TCP Reassembly & Issues)

This section appears when:

- TCP segments require reassembly
- There are retransmissions
- Out-of-order packets exist
- Missing segments occur

Wireshark may indicate:

- “TCP segment of a reassembled PDU”
- “Previous segment not captured”
- “Duplicate ACK”
- “Retransmission”

Why This Matters:

These indicators help identify:

- Network congestion
- Packet loss
- Service degradation
- Potential evasion attempts

Understanding TCP reassembly is critical when analyzing HTTP, SMB, or other multi-segment protocols.

Layer 5 – Application Protocol (Application Layer)

What It Shows:

Protocol-specific details such as:

- HTTP request/response
- FTP commands
- SMB operations

- DNS queries
- SMTP transactions

Example in HTTP:

- Request method (GET/POST)
- Host header
- Response code (200, 404, 500)
- Content-Type

This layer gives meaning to the communication.

Without it, packets are just transport segments.

Application Data

This is an extension of the application layer and may show:

- HTTP payload
- Uploaded files
- JSON data
- Form submissions
- File transfer content

Depending on encryption:

- Plaintext protocols show readable content.
- Encrypted protocols (HTTPS) will show encrypted payload unless decrypted.

This section is crucial for:

- Data exfiltration analysis
- Credential exposure detection
- Malware traffic analysis

Relationship Between Details Pane and Bytes Pane

One of Wireshark's most powerful features:

When you click a field in the details pane:

- The exact raw bytes are highlighted below.

This allows analysts to:

- Validate protocol interpretation
- Perform forensic-level verification
- Understand binary encoding

It bridges structured analysis and raw packet data.

Practical Example: HTTP Packet Breakdown

A typical HTTP packet may contain:

1. Frame metadata
2. Ethernet header (MAC addresses)
3. IP header (source/destination IP)
4. TCP header (ports, sequence numbers)
5. HTTP header (GET request, response code)
6. HTTP data (payload)

Each layer encapsulates the next — this is called encapsulation.

Why Packet Dissection Matters

Packet dissection allows analysts to:

- Understand how communication is structured
- Identify abnormal protocol behavior
- Detect manipulation or tampering

- Perform forensic investigations
- Troubleshoot application errors
- Analyze attack patterns

Without understanding layers, analysis becomes guesswork.

Key Analytical Strategy

When analyzing a packet:

Step 1: Identify Communication

- Who is talking to whom? (IP addresses)

Step 2: Identify Service

- Which port and protocol?

Step 3: Check Transport Health

- Retransmissions?
- Reset connections?
- Abnormal flags?

Step 4: Inspect Application Layer

- Legitimate request?
- Suspicious commands?
- Data leakage?

Step 5: Correlate with Other Packets

- Follow stream
- Check timing
- Review related traffic

Important Concept : Encapsulation

Each OSI layer wraps the data from the layer above it.

Think of it like nested envelopes:

Application Data

→ Wrapped inside TCP

→ Wrapped inside IP

→ Wrapped inside Ethernet

→ Wrapped inside Frame metadata

Wireshark visually separates these layers for clarity.

Professional Insight

Understanding packet dissection is foundational for:

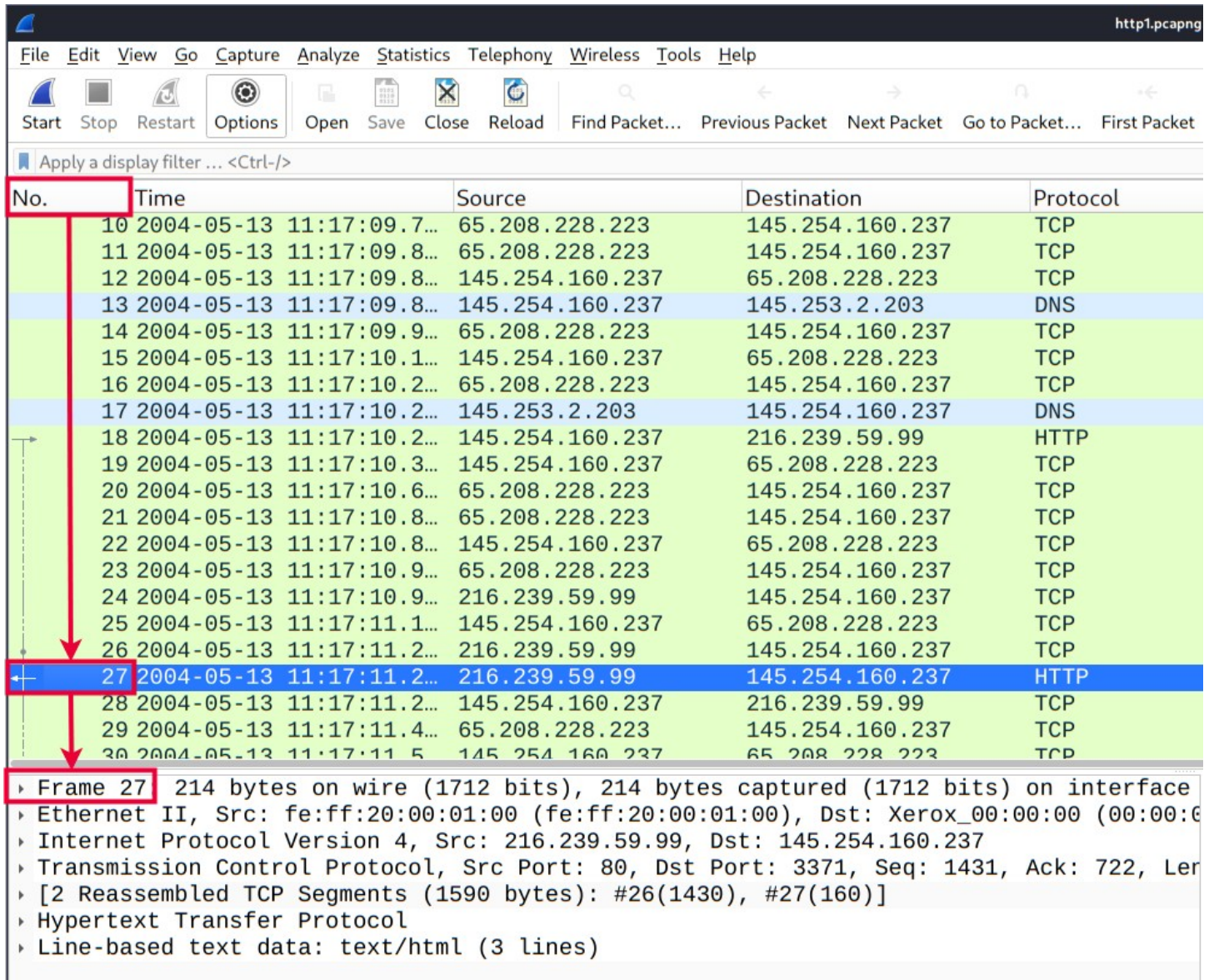
- Network Engineers
- SOC Analysts
- Threat Hunters
- Malware Analysts
- Digital Forensic Experts

Mastery of layer-by-layer analysis enables:

- Root cause analysis
- Deep packet inspection
- Protocol abuse detection
- Advanced troubleshooting

Advanced Packet Navigation & Analysis in Wireshark

Packet Numbers



The screenshot shows the Wireshark interface with a packet capture file named 'http1.pcapng'. The packet list pane displays a table of captured packets. Packet 27 is highlighted in blue. A red arrow points from the 'No.' column of packet 27 to the packet details pane below.

No.	Time	Source	Destination	Protocol
10	2004-05-13 11:17:09.7...	65.208.228.223	145.254.160.237	TCP
11	2004-05-13 11:17:09.8...	65.208.228.223	145.254.160.237	TCP
12	2004-05-13 11:17:09.8...	145.254.160.237	65.208.228.223	TCP
13	2004-05-13 11:17:09.8...	145.254.160.237	145.253.2.203	DNS
14	2004-05-13 11:17:09.9...	65.208.228.223	145.254.160.237	TCP
15	2004-05-13 11:17:10.1...	145.254.160.237	65.208.228.223	TCP
16	2004-05-13 11:17:10.2...	65.208.228.223	145.254.160.237	TCP
17	2004-05-13 11:17:10.2...	145.253.2.203	145.254.160.237	DNS
18	2004-05-13 11:17:10.2...	145.254.160.237	216.239.59.99	HTTP
19	2004-05-13 11:17:10.3...	145.254.160.237	65.208.228.223	TCP
20	2004-05-13 11:17:10.6...	65.208.228.223	145.254.160.237	TCP
21	2004-05-13 11:17:10.8...	65.208.228.223	145.254.160.237	TCP
22	2004-05-13 11:17:10.8...	145.254.160.237	65.208.228.223	TCP
23	2004-05-13 11:17:10.9...	65.208.228.223	145.254.160.237	TCP
24	2004-05-13 11:17:10.9...	216.239.59.99	145.254.160.237	TCP
25	2004-05-13 11:17:11.1...	145.254.160.237	65.208.228.223	TCP
26	2004-05-13 11:17:11.2...	216.239.59.99	145.254.160.237	TCP
27	2004-05-13 11:17:11.2...	216.239.59.99	145.254.160.237	HTTP
28	2004-05-13 11:17:11.2...	145.254.160.237	216.239.59.99	TCP
29	2004-05-13 11:17:11.4...	65.208.228.223	145.254.160.237	TCP
30	2004-05-13 11:17:11.5...	145.254.160.237	65.208.228.223	TCP

Frame 27: 214 bytes on wire (1712 bits), 214 bytes captured (1712 bits) on interface
Ethernet II, Src: fe:ff:20:00:01:00 (fe:ff:20:00:01:00), Dst: Xerox_00:00:00 (00:00:00:00:00:00)
Internet Protocol Version 4, Src: 216.239.59.99, Dst: 145.254.160.237
Transmission Control Protocol, Src Port: 80, Dst Port: 3371, Seq: 1431, Ack: 722, Len: 160
[2 Reassembled TCP Segments (1590 bytes): #26(1430), #27(160)]
Hypertext Transfer Protocol
Line-based text data: text/html (3 lines)

What Are Packet Numbers?

Every packet captured in Wireshark is automatically assigned a unique sequential number.

This number:

- Represents the packet's position in the capture
- Is calculated in order of arrival
- Allows quick reference during investigations

Why Packet Numbers Matter

In large captures (thousands or millions of packets), packet numbers:

- Help analysts return to a specific event
- Enable accurate documentation in reports
- Allow correlation between investigation notes and packet evidence
- Support collaborative investigations

Example:

“Suspicious activity begins at packet 15432.”

Go To Packet - Navigation Feature

The Go To Packet functionality enhances packet navigation beyond simple scrolling.

Accessible via:

- Go menu
- Toolbar navigation options

Capabilities:

- Jump directly to a specific packet number
- Move forward or backward through packets
- Navigate within a conversation stream
- Track packets related to a specific flow

Practical Use Case:

If Expert Info reports an issue at packet 24567:

- Use “Go To Packet”
- Jump instantly to that location
- Begin contextual analysis

This saves time in large capture files.

Finding Packets by Content

Wireshark allows searching inside packet contents.

Accessible via:

- **Edit → Find Packet**

This feature helps analysts locate:

- Intrusion patterns
- Error messages
- Suspicious payload strings
- Indicators of compromise (IOCs)

Input Types for Packet Search

Wireshark supports four search input types:

1 Display Filter

Uses filter syntax (e.g., `ip.addr == 10.0.0.5`)

2 Hex

Searches raw hexadecimal values

3 String (Most Common)

Searches readable text in packets

4 Regular Expression (Regex)

Advanced pattern matching for complex searches

Important Note :

- Searches are case-insensitive by default
- Case sensitivity can be enabled via radio button

Choosing the Correct Search Field

Searches can be performed in:

1. Packet List Pane
2. Packet Details Pane
3. Packet Bytes Pane

⚠ Critical Concept :

If you search for data present in the Details Pane but conduct the search in the Packet List Pane, Wireshark will not find it.

Understanding what each pane contains is essential for effective searching.

Marking Packets

Packet marking helps analysts visually tag important packets.

Accessible via:

- Right-click menu
- Edit menu

What Happens When You Mark a Packet?

- Packet is displayed in black
- Overrides original protocol coloring
- Makes it visually distinct

Use Cases:

- Flagging suspicious traffic
- Identifying attack start/end points
- Marking evidence for reporting
- Preparing packets for export

⚠ Important Limitation:

Marked packets are not saved permanently.
They are cleared when the file is closed.

Packet Comments

Packet comments provide persistent annotations.

Unlike marking:

- Comments are saved within the capture file
- Remain until manually removed
- Support collaboration

Why Packet Comments Are Important

They allow analysts to:

- Document findings
- Explain anomalies
- Add contextual notes
- Assist other team members

Example:

“Possible command-and-control beacon attempt.”

Comments transform raw data into documented evidence.

Exporting Packets

Large captures often contain irrelevant traffic.

Wireshark allows exporting selected packets.

Accessible via:

- File → Export Specified Packets

Why Export Packets?

- Share only suspicious traffic
- Reduce noise
- Perform deeper forensic analysis
- Submit evidence for reporting

Export scope can include:

- Displayed packets
- Marked packets
- Selected packets
- Packet range

This helps isolate relevant evidence.

Exporting Objects (Extracting Files)

Wireshark can extract files transferred across the network.

Supported protocols include:

- HTTP
- SMB
- TFTP
- IMF
- DICOM

Accessible via:

- File → Export Objects

Why This Matters

For security analysts, this is extremely powerful.

You can extract:

- Downloaded malware
- Transferred documents
- Suspicious executables
- Data exfiltration artifacts

This feature converts network evidence into analyzable files.

⚠ **Note:**

Only available for supported protocols and unencrypted streams.

Time Display Format

By default, Wireshark shows time as:

Seconds Since Beginning of Capture

While useful for relative timing, this is not always ideal.

Recommended Setting:

Use UTC Time Format for better timeline analysis.

Accessible via:

- View → Time Display Format

Why Change Time Format?

- Correlate logs with other systems
- Perform forensic timeline reconstruction
- Align packet timestamps with SIEM alerts

Time accuracy is critical in investigations.

Expert Information (Expert Info)

Wireshark includes an intelligent feature called Expert Information. It highlights unusual or noteworthy protocol behaviors.

Accessible via:

- Status bar (lower-left)
- Analyze → Expert Information

1 Severity Levels

Severity	Colour	Meaning
Chat	Blue	Informational workflow details
Note	Cyan	Notable events (e.g., app errors)
Warning	Yellow	Suspicious or unusual behavior
Error	Red	Serious issues (e.g., malformed packets)

2 Common Information Groups

Group	Meaning
Checksum	Packet integrity issues
Deprecated	Outdated protocol usage
Comment	Packet comment detected
Malformed	Improperly structured packets

3 Important Caution

Expert Info provides suggestions, not definitive conclusions.

There may be False positives, False negatives
Human analysis is still required.

Practical Investigation Workflow

When analyzing a suspicious capture:

1. Check Expert Info for alerts
2. Go to specific packet numbers
3. Search for keywords or indicators
4. Mark suspicious packets
5. Add comments for documentation
6. Export relevant packets or objects
7. Adjust time display for timeline analysis

This structured approach improves efficiency and accuracy.

Key Analytical Advantages

Mastering these features enables:

- Faster navigation in large captures
- Efficient anomaly detection
- Better evidence documentation
- Structured forensic workflow
- Improved collaboration

Packet Filtering in Wireshark - Professional Study Notes

Packet filtering is one of the most powerful capabilities in Wireshark. In real-world captures, thousands or even millions of packets may exist. Without filtering, analysis becomes inefficient and error-prone.

Filtering allows analysts to:

- Reduce noise
- Focus on relevant traffic
- Isolate suspicious activity
- Investigate specific hosts, ports, or protocols

Two Types of Filters

Wireshark provides two filtering approaches :

1 - Capture Filters

- Applied before traffic is captured
- Only matching packets are recorded
- Reduces file size and noise from the start
- Cannot retrieve excluded traffic later

Used when:

- You know exactly what traffic you need
- You want minimal storage usage

2 - Display Filters

- Applied after traffic is captured
- Do not remove packets from file
- Only control what is shown

Used when:

- Investigating an existing capture
- Performing exploratory analysis

⚠ In this section, we focus on Display Filters, as they are most commonly used during investigations.

Two Ways to Apply Display Filters

There are two main filtering methods:

1 Writing Queries (Manual Filtering)

Using the Display Filter Bar.

2 Right-Click GUI Filtering

Golden Rule:

“If you can click on it, you can filter and copy it.”

This makes filtering accessible even for beginners.

Apply as Filter

This is the simplest filtering method.

How It Works:

1. Click a field in the Packet Details Pane
2. Right-click
3. Select Apply as Filter

Wireshark automatically generates and applies the filter.

Use Case:

If you click on:

- Source IP: 192.168.1.10
You can instantly filter all packets from that IP.

The Status Bar always shows:

- Total packets
- Displayed packets

This helps measure filter impact.

Conversation Filter

“Apply as Filter” isolates a single value.

“Conversation Filter” isolates an entire communication session.

What It Does:

Filters all packets related to:

- Source IP
- Destination IP
- Source Port
- Destination Port

This allows full visibility into a single communication flow.

Accessible via:

- Right-click → Conversation Filter
- Analyze → Conversation Filter

Practical Use Case:

If packet 150 looks suspicious:

- Use Conversation Filter
- See full interaction between both endpoints

This is essential for:

- Attack investigations
- Service debugging
- Following connection lifecycle

Colourise Conversation

This feature is similar to Conversation Filter but:

- Does NOT hide other packets
- Only changes the color of related packets

Benefits:

- Visual tracking without filtering
- Keeps context intact
- Avoids changing displayed packet count

Accessible via:

- View → Colourise Conversation

You can reset it via:

- View → Reset Colourisation

This is useful when comparing multiple simultaneous streams.

Prepare as Filter

This feature helps build filters without immediately applying them.

Key Difference:

- “Apply as Filter” executes instantly
- “Prepare as Filter” places the query in the filter bar but waits

This allows:

- Combining conditions
- Adding AND / OR logic
- Refining queries before execution

Example:

- Prepare source IP filter
- Add destination port condition
- Execute combined filter

This supports advanced query construction.

Apply as Column

By default, the Packet List Pane shows:

- Packet number
- Time
- Source
- Destination
- Protocol
- Length
- Info

You can add custom columns.

How :

- 1.Right-click a field
- 2.Select “Apply as Column”

The selected field now appears as a new column.

Why This Is Useful:

It allows you to Compare a field across all packets, Identify patterns quickly and Spot anomalies visually

Example:

Adding “HTTP Host” as a column to see domain requests across packets.

Columns can be enabled/disabled by right-clicking the column header.

Follow Stream

One of the most powerful features in Wireshark.

Wireshark captures traffic as individual packets.

But applications communicate in continuous streams.

Follow Stream reconstructs:

- TCP streams
- UDP streams
- HTTP streams

Accessible via:

- Right-click → Follow TCP/UDP/HTTP Stream
- Analyze menu

What Happens When You Follow a Stream?

- A new window opens
- Server data appears in blue
- Client data appears in red
- Full conversation is reconstructed
- A display filter is automatically applied

To return to full capture:

- Click the “X” on the Display Filter Bar

Why Follow Stream Is Important

It allows analysts to:

- Reconstruct login attempts
- View usernames and passwords (if unencrypted)
- Detect malicious commands
- Understand request/response flow
- Investigate command-and-control traffic

⚠ Encrypted traffic (e.g., HTTPS) will not display readable content unless decrypted.

Writing Simple Display Filter Queries

The Display Filter Bar is located below the toolbar.

It changes color:

- Green → Valid filter
- Red → Syntax error

Basic Filtering Examples

Filter by Protocol Name

Simply type:

`http`

Other examples:

- `arp`
- `dhcp`
- `ftp`
- `smtp`
- `pop`
- `imap`

This displays only packets of that protocol.

Filter by Port Number

Structure:

```
tcp.port == <port>
```

```
udp.port == <port>
```

Example for HTTP:

```
tcp.port == 80
```

This shows all TCP traffic using port 80.

Filter by IP Address

Structure:

```
ip.addr == <IP>
```

Example:

```
ip.addr == 192.168.1.2
```

This displays:

- Packets where IP is source OR destination

For direction-specific filtering:

```
ip.src == 192.168.1.2
```

```
ip.dst == 192.168.1.2
```

Logical Operators (Very Important)

Display filters can be combined using:

- and
- or
- not

Example:

```
ip.addr == 192.168.1.2 and tcp.port == 80
```

This narrows results significantly.

Investigation Strategy Using Filters

When analyzing a large capture:

- 1.Filter by IP of interest
- 2.Narrow by protocol
- 3.Narrow by port
- 4.Follow stream
- 5.Add additional conditions
- 6.Export suspicious packets

Filtering should move from broad → specific.

Common Beginner Mistakes

- Confusing capture filters with display filters
- Filtering in wrong pane during search
- Forgetting to clear applied filters
- Using incorrect syntax (missing ==)
- Not checking filter bar color

Note : Wireshark filter usage varies by role. SOC analysts, threat hunters, red teamers, and incident responders apply filters differently based on their operational objectives. Mastery of these filters is best developed through practical, hands-on analysis.

Source to dive deeper into Wireshark : <https://www.wireshark.org/docs/>

Room 06 : Tcpdump - The Basics

When studying networking protocols, one of the biggest challenges is visibility.

Although networks are constantly exchanging data, users typically interact only with graphical interfaces (web browsers, apps, file explorers). The underlying protocol communications remain hidden. For example:

- When accessing a device on a local network, an ARP (Address Resolution Protocol) request is generated, but the user never sees it.
- When connecting to a website, a TCP three-way handshake occurs in the background before any data is exchanged.
- When browsing the internet for years, most users never see a single packet unless they intentionally inspect network traffic.

Why Packet Capture Matters

The best way to understand networking protocols is to:

1. Capture network traffic
2. Inspect the captured packets
3. Analyze protocol behavior

This process exposes the “conversations” between devices and allows learners to understand how protocols operate in real environments.

Introduction to Tcpdump

Tcpdump is a command-line packet analyzer used to capture and display network traffic.

- Written in C and C++
- Originally released for Unix-like systems in the late 1980s/early 1990s
- Known for :
 - Stability
 - High performance
 - Reliability

It relies on the libpcap library, which is the foundation for many modern packet capture tools.

Related Technologies

- libpcap: Packet capture **library** used by Tcpdump and other tools.
- WinPcap: Windows port of libpcap.
- Wireshark: A graphical packet analyzer that can read Tcpdump capture files (.pcap).

Learning Objectives

This module focuses on developing foundational Tcpdump skills. By the end, you should be able to:

1. Capture packets and save them to a file.
2. Apply filters to captured packets.
3. Control how packets are displayed.

These skills are essential for:

- Network troubleshooting
- Security monitoring
- Forensic analysis
- Protocol learning

Using Tcpdump : Core Command-Line Options

Running `tcpdump` without arguments only verifies installation. In practical usage, you must define:

- Which interface to listen on
- How many packets to capture
- Whether to save to file
- How to display results

Specifying the Network Interface (-i)

Before capturing traffic, you must choose a network interface.

What is a Network Interface?

A network interface is the point where your system connects to a network. Examples include:

- `eth0` - Wired Ethernet
- `wlo1` - Wireless interface
- `ens5` - Predictable Linux interface naming
- `lo` - Loopback interface (localhost communication)

Listing Available Interfaces

Use : *ip address* show or *ip a s*

This command displays:

- Interface names
- IP addresses
- Status (UP/DOWN)
- MTU values

Capturing on an Interface

tcpdump -i ens5

To listen on all interfaces:

tcpdump -i any

Saving Captured Packets (-w)

In real-world scenarios, you often need to analyze traffic later.

Save to File : *tcpdump -i ens5 -w capture.pcap*

Important points:

- File extension is typically .pcap
- No packets are displayed on screen when using -w
- The file can be opened later in:
 - Tcpdump
 - Wireshark
 - Other forensic tools

This is particularly useful for:

- Incident investigations
- Capturing suspicious behavior
- Long-duration traffic monitoring

Reading Packets from a File (-r)

To analyze a previously saved capture:

```
tcpdump -r capture.pcap
```

Benefits:

- Review traffic offline
- Apply filters after capture
- Investigate attacks or anomalies

This is common in cybersecurity and digital forensics.

Limiting Packet Capture (-c)

By default, Tcpcap captures packets continuously until interrupted

(CTRL-C).

To limit capture : *tcpdump -i ens5 -c 10*

This captures only 10 packets.

Why use this?

- Avoid overwhelming output
- Capture only what's needed
- Reduce file size
- Perform controlled testing

Disabling Name Resolution (-n and -nn)

By default, Tcpdump tries to :

- Convert IP addresses to domain names
- Convert port numbers to service names (e.g., 80 → HTTP)

This can:

- Slow down capture
- Generate additional DNS traffic
- Obscure raw data

Disable DNS Resolution

tcpdump -n Prevents hostname resolution.

Disable DNS and Port Resolution

tcpdump -nn prevents both Hostname resolution and Service name resolution

Example output format:

```
IP 10.10.117.2.22 > 10.11.81.126.53378
```

Meaning:

- Source IP: 10.10.117.2
- Source Port: 22
- Destination IP: 10.11.81.126
- Destination Port: 53378

This raw format is preferred for technical analysis.

Increasing Verbosity (-v, -vv, -vvv)

Tcpdump offers multiple verbosity levels.

Basic Verbose Output : *tcpdump -v*

Displays additional information such as:

- TTL (Time To Live)
- Packet length
- IP identification field
- Header options

Higher Verbosity

- -vv
- -vvv

Each level reveals more protocol details.

This is useful for:

- Deep protocol inspection
- Debugging
- Learning packet structure

Summary of Core Tcpdump Options

Command	Purpose
-i INTERFACE	Capture on specific interface
-w FILE	Write packets to file
-r FILE	Read packets from file
-c COUNT	Capture limited number of packets
-n	Disable hostname resolution
-nn	Disable hostname and service resolution
-v, -vv, -vvv	Increase verbosity

Practical Example Breakdown

Example 1

```
tcpdump -i eth0 -c 50 -v
```

- Listen on Ethernet interface
- Capture 50 packets
- Display verbose output

Example 2

```
tcpdump -i wlo1 -w data.pcap
```

- Capture on WiFi interface
- Save to file
- Continue until interrupted

Example 3

```
tcpdump -i any -nn
```

- Listen on all interfaces
- Show raw IP addresses and ports
- No resolution performed

Tcpdump Filtering Techniques

Filtering by Host

Host-based filtering limits captured traffic to communication involving a specific IP address or hostname.

Basic Host Filter : `sudo tcpdump host example.com -w http.pcap`

This captures all packets sent to or received from example.com and saves them to http.pcap. Root privileges (or sudo) are required for live packet capture because tcpdump interacts directly with network interfaces.

Source and Destination Filtering

To filter only packets originating from a specific host :

`tcpdump src host IP`

To filter only packets sent to a specific host :

`tcpdump dst host IP`

This distinction is important when analyzing traffic direction, such as identifying outgoing requests versus incoming responses.

Filtering by Port

Port filtering isolates traffic associated with specific services because network services operate on defined port numbers.

Example: DNS Traffic (Port 53)

`sudo tcpdump -i ens5 port 53 -n`

This captures all traffic to or from port 53. DNS typically uses UDP 53, but can also use TCP 53.

Source and Destination Port Filters

To filter by source port only :

```
tcpdump src port PORT_NUMBER
```

To filter by destination port only:

```
tcpdump dst port PORT_NUMBER
```

This is useful when identifying client-side versus server-side behavior.

Filtering by Protocol

Protocol filtering restricts capture to a specific network protocol.

Common Protocol Filters

```
tcpdump tcp
```

```
tcpdump udp
```

```
tcpdump icmp
```

```
tcpdump ip
```

```
tcpdump ip6
```

Example: ICMP Traffic

```
sudo tcpdump -i ens5 icmp -n
```

This captures only ICMP packets. Observations may include:

- ICMP Echo Request (generated by ping).
- ICMP Echo Reply.
- ICMP Time Exceeded (commonly generated during traceroute).
- ICMP Destination Unreachable.

Filtering by protocol is especially useful in diagnosing connectivity issues or identifying scanning behavior.

Logical Operators in Tcpcap

Tcpcap supports logical operators to combine filtering conditions, allowing more precise traffic selection.

AND Operator

```
tcpcap host 1.1.1.1 and tcp
```

Captures only TCP packets involving host 1.1.1.1. Both conditions must be true.

OR Operator

```
tcpcap udp or icmp
```

Captures packets that are either UDP or ICMP.

NOT Operator

```
tcpcap not tcp
```

Captures all traffic except TCP. This may include UDP, ICMP, and ARP packets.

Logical operators significantly increase analytical precision and are frequently used in incident investigations.

Combining Filters for Real-World Scenarios

Complex filtering expressions allow targeted monitoring.

Examples:

- Capture SSH traffic on all interfaces:

```
tcpcap -i any tcp port 22
```

This isolates TCP traffic associated with SSH.

- Capture NTP traffic on WiFi interface:

```
tcpcap -i wlan0 udp port 123
```

This filters UDP traffic for Network Time Protocol.

- Capture HTTPS traffic for a specific host:

```
tcpdump -i eth0 host example.com and tcp port 443 -w  
https.pcap
```

This filters encrypted web traffic associated with example.com and saves it for later analysis.

Combining host, protocol, and port filters is standard practice in security monitoring.

Reading from a Capture File with Filters

Tcpdump can analyze previously captured traffic using the `-r` option.

Example : *tcpdump -r traffic.pcap -c 5 -n*

This reads the first five packets from traffic.pcap without resolving IP addresses.

Unlike live capture, reading from a file does not require root privileges.

Counting Packets with a Specific Filter

You can pipe tcpdump output to other Linux utilities.

```
tcpdump -r traffic.pcap src host 192.168.124.1 -n | wc
```

This command:

- Reads packets from traffic.pcap.
- Filters packets with source IP 192.168.124.1.
- Sends output to wc to count lines, words, and bytes.

Advanced Filtering and Output Customization in Tcpdump

Filtering by Packet Length

Tcpdump allows filtering packets based on their total size. This is useful when investigating abnormal traffic patterns, such as unusually large payloads or suspicious small packets.

- **greater LENGTH** → Captures packets with length greater than or equal to the specified value.
- **less LENGTH** → Captures packets with length less than or equal to the specified value.

Example logic: You might filter very large packets to inspect possible file transfers or very small packets to analyze control traffic.

This form of filtering helps reduce noise and focus on packets relevant to the investigation.

Understanding Binary Operations

Advanced filtering sometimes requires working directly with bits inside packet headers. Since header fields are stored in binary format, binary operations are used to evaluate specific flag combinations.

1 AND Operator (&)

Returns 1 only if both input bits are 1.

Used when checking whether a specific bit is set.

2 OR Operator (|)

Returns 1 if at least one input bit is 1.

Used when checking if any one of multiple bits is set.

3 NOT Operator (!)

Inverts a bit value.

Used to exclude specific bit conditions.

Binary logic is fundamental when filtering based on TCP flags because each TCP flag corresponds to a specific bit in the header.

Filtering by Header Bytes (pcap-filter Syntax)

Tcpdump allows direct access to protocol header bytes using the format:

`proto[expr:size]`

Where:

- `proto` → Protocol (e.g., `arp`, `ether`, `icmp`, `ip`, `ip6`, `tcp`, `udp`).
- `expr` → Byte offset (0 = first byte).
- `size` → Number of bytes to inspect (1, 2, or 4; default is 1).

This technique allows deep inspection of protocol internals without needing a graphical analyzer.

Conceptual Examples

- `ether[0] & 1 != 0` → Checks if a specific bit in the Ethernet header indicates multicast traffic.
- `ip[0] & 0xf != 5` → Identifies IPv4 packets containing options.

Although these examples appear complex, they demonstrate the power of byte-level inspection for advanced packet analysis.

Filtering TCP Packets by Flags

TCP flags are individual bits in the TCP header that define connection behavior. Tcpdump simplifies filtering these flags using : `tcp[tcpflags]`

Common TCP Flags

- tcp-syn → Synchronize (connection initiation)
- tcp-ack → Acknowledge
- tcp-fin → Finish (connection termination)
- tcp-rst → Reset
- tcp-push → Push

Example Filters

Capture packets with only SYN set:

```
tcpdump "tcp[tcpflags] == tcp-syn"
```

Capture packets where SYN is set (possibly along with others):

```
tcpdump "tcp[tcpflags] & tcp-syn != 0"
```

Capture packets with either SYN or ACK set:

```
tcpdump "tcp[tcpflags] & (tcp-syn|tcp-ack) != 0"
```

These filters are particularly useful for:

- Detecting connection attempts.
- Identifying scanning behavior.
- Monitoring handshake sequences.
- Investigating abnormal resets.

Mastering TCP flag filtering significantly improves analytical capability.

Customizing Packet Display Output

Tcpdump offers multiple options to control how captured packets are displayed. Choosing the appropriate display format depends on the investigation goal.

Quick Output (-q)

The `-q` option provides concise output showing only essential details such as timestamp, IP addresses, ports, protocol, and packet length.

```
tcpdump -r file.pcap -q
```

Use this when you need high-level traffic summaries rather than detailed header analysis.

Displaying Link-Level Header (-e)

The `-e` option includes Ethernet-level details such as MAC addresses and EtherType.

```
tcpdump -r file.pcap -e
```

This is useful when:

- Investigating ARP or DHCP traffic.
- Tracking suspicious devices by MAC address.
- Understanding local network behavior.

It adds visibility to Layer 2 (Data Link Layer).

Displaying Packet Data as ASCII (-A)

The -A option displays packet payloads mapped to ASCII characters.

```
tcpdump -r file.pcap -A
```

This works well when:

- Inspecting unencrypted HTTP traffic.
- Viewing plain-text protocols.
- Extracting readable strings from packets.

However, it is ineffective for encrypted or compressed traffic.

Displaying Packet Data in Hexadecimal (-xx)

The -xx option prints packet data in hexadecimal format.

```
tcpdump -r file.pcap -xx
```

Hexadecimal is useful because:

- Every byte (8 bits) is represented by two hex digits.
- It works regardless of encoding.
- It allows inspection of raw headers and payload.

This is ideal for:

- Protocol reverse engineering.
- Header field verification.
- Detailed forensic inspection.

Combined Hex and ASCII View (-X)

The -X option displays packet contents in both hexadecimal and ASCII.

```
tcpdump -r file.pcap -X
```

This is often the most informative format because:

- Hex view shows exact byte values.
- ASCII view shows readable text where applicable.
- Both formats appear side by side for comparison.

It is particularly helpful when analyzing partially encrypted or structured payloads.

Key Takeaways

Advanced packet filtering enables precise extraction of relevant traffic from massive datasets.

Length-based filtering isolates packets by size,

Binary operations allow inspection of specific bits, and header byte access provides deep protocol control.

TCP flag filtering is a powerful method for analyzing connection states and detecting unusual behavior.

Output customization options such as -q, -e, -A, -xx, and -X allow analysts to tailor packet visibility according to investigative needs.

Mastering these techniques transforms Tcpcap from a basic capture tool into a powerful forensic and diagnostic instrument.

Room 07 : Nmap - The Basics

Before performing any detailed security assessment or network scan, the first and most fundamental question is:

Which systems are actually online?

Scanning inactive or non-existent hosts wastes time and resources.

Therefore, the initial step in network reconnaissance is host discovery — identifying which IP addresses belong to active devices.

The tool used in this context is Nmap, one of the most widely used network scanning and reconnaissance tools in cybersecurity.

Nmap provides several intelligent and efficient techniques to determine whether a host is "up" (online) or "down" (offline).

Specifying Targets in Nmap

Before scanning, we must tell Nmap what to scan. Nmap supports multiple target specification formats, allowing flexibility depending on the situation.

IP Range Using Hyphen (-)

Example : `192.168.0.1-10`

What it does : Scans IP addresses from : `192.168.0.1` → `192.168.0.10`

Why use it ?

- Useful when scanning a small, specific segment of a network.
- Efficient when you do not need to scan the entire subnet.

Subnet Using CIDR Notation (/)

Example : 192.168.0.1/24

What it means :

This represents the entire subnet : 192.168.0.0 – 192.168.0.255

A /24 subnet contains 256 IP addresses.

Why use it?

- Most local networks are configured as /24.
- Ideal for scanning an entire network quickly.

Hostname

Instead of IP addresses, you can specify : example.thm

Why use it?

- When the IP address is unknown.
- When DNS resolution is available.
- Useful in external/Internet-based scans.

Running Nmap with Root Privileges

Nmap behaves differently depending on user privileges.

Why run as root (or with sudo)?

Running Nmap as a non-root user limits it to:

- ICMP echo requests
- TCP connect scans

Running as root allows:

- ARP scans
- TCP SYN scans
- Advanced host discovery techniques
- More precise packet crafting

Without elevated privileges, Nmap cannot generate certain low-level packets required for advanced scanning techniques.

Understanding the -sn Option (Ping Scan)

The option : -sn

Means:

Scan for live hosts only — do NOT perform port scanning.

This is called a Ping Scan.

Important Clarification

Despite the name, this is NOT limited to traditional ICMP ping.

Depending on the situation, Nmap may use :

- ARP requests
- ICMP echo requests
- ICMP timestamp requests
- TCP SYN packets
- TCP ACK packets

The technique used depends on whether the target is on a local or remote network.

Scanning a Local Network

A local network means:

- You are directly connected (WiFi or Ethernet).
- No routers separate you from the target hosts.

Example:

- Your IP: 192.168.66.89
- Target subnet: 192.168.66.0/24

Command:

```
nmap -sn 192.168.66.0/24
```

How Nmap Discovers Hosts on a Local Network

When scanning a directly connected network, Nmap primarily uses :

ARP Requests (Address Resolution Protocol)

Why ARP?

- ARP works at Layer 2 (Data Link layer).
- It directly asks : *“Who has this IP address?”*
- Devices that own the IP respond with their MAC address.

If a device responds : Host is up

Why ARP Is Reliable on Local Networks

- ARP cannot be blocked by most host firewalls.
- Every device on the LAN must respond to ARP to function properly.
- Faster and more accurate than ICMP in local environments.

MAC Address Discovery

In the output, Nmap shows :

MAC Address: 7C:DF:A1:D3:8C:5C (Espressif)

Why is this important?

The MAC address reveals:

- The network interface manufacturer.
- Sometimes the device type.

Examples:

- Espressif → Often IoT devices (ESP8266, ESP32).
- Tuya → Smart home devices.
- Router manufacturers.

Why This Matters in Security

Knowing the vendor helps:

- Identify device categories.
- Spot IoT devices.
- Recognize routers.
- Prioritize high-value targets.

Scanning a Remote Network

A remote network means:

- At least one router separates you from the target.
- Traffic must pass through routers.

Example :

- Your IP: 192.168.66.89
- Target: 192.168.11.0/24
- Separated by 2+ hops

Command : `nmap -sn 192.168.11.0/24`

Why ARP Cannot Be Used Remotely ? : [ARP is not routable.](#)

Routers:

- Do NOT forward ARP requests.
- Only forward IP traffic.

Therefore:

- Nmap must use Layer 3 techniques (IP-based).

Techniques Used for Remote Discovery

Nmap uses a combination of IP-based probe packets designed to trigger a response from live systems. These probes test different network behaviors to increase reliability.

From traffic analysis (observed via Wireshark), Nmap sent :

- 1.ICMP Echo (Ping)
- 2.ICMP Timestamp requests
- 3.TCP SYN packets to port 443
- 4.TCP ACK packets to port 80

Why Use Multiple Methods?

Some hosts:

- Block ICMP
- Block SYN packets

- Drop unexpected traffic

Using multiple probe types increases:

- Accuracy
- Chances of detecting live systems

Example Analysis

Host 192.168.11.1

- Responded to ICMP echo
- Marked as up

Host 192.168.11.2

- Did not respond to:
 - ICMP echo
 - ICMP timestamp
 - TCP SYN
 - TCP ACK
- Router returned:
 - ICMP destination unreachable

Result:

- Considered down

Why Remote Scans Take Longer

Local scan : ~2.64 seconds

Remote scan : ~10.67 seconds

Reason :

- More hops
- Packet filtering
- Retransmissions
- Waiting for timeouts

Controlling Host Discovery Methods

Nmap allows fine control over discovery probes :

Option	Purpose
-PS[port]	TCP SYN discovery
-PA[port]	TCP ACK discovery
-PU[port]	UDP discovery

Why Customize ?

Some networks:

- Block ICMP
- Allow HTTPS (443)
- Allow HTTP (80)

Sending probes to allowed ports increases success rate.

List Scan (-sL)

Command : `nmap -sL 192.168.0.1/24`

What It Does

- Lists all targets.
- Does NOT send packets.
- Does NOT check if hosts are up.

Why Use It?

- Confirm target range.
- Avoid accidental scanning.
- Validate CIDR notation.
- Prevent scanning wrong network.

Example :

Nmap scan report for 192.168.0.1

Nmap scan report for 192.168.0.2

...

No actual host discovery occurs.

Noise and Stealth Considerations

The -sn option:

- Does NOT scan ports.
- Only checks if hosts are alive.
- Generates minimal traffic compared to full scans.

Why This Matters

- Reduces detection risk.
- Good for reconnaissance phase.
- Useful when you want visibility without aggressive probing.

However:

It does NOT tell:

- Open ports
- Running services
- Service versions

For that, more advanced scans are required.

Key Differences: Local vs Remote Discovery

Feature	Local Network	Remote Network
ARP Used	Yes	No
MAC Address Visible	Yes	No
Speed	Faster	Slower
Accuracy	Very High	Depends on filtering
Firewall Impact	Minimal	Significant

Practical Workflow

1. Start with : `nmap -sL target_range`

Confirm scope.

2. Perform : `nmap -sn target_range`

Identify live hosts.

3. Move to :

- Port scanning
- Service detection
- Version enumeration

Important Takeaways

- Host discovery is the foundation of scanning.
- ARP is superior for local network discovery.
- Remote scans rely on ICMP and TCP-based probes.
- Root privileges unlock full Nmap capabilities.
- -sn reduces noise but does not reveal services.
- -sL is useful for verification before scanning.

Network Service Discovery with Nmap

After identifying which hosts are online using -sn, the next logical step is to determine :

What services are running on those live systems?

A network service is any application listening for incoming connections on a TCP or UDP port.

Examples:

- Web servers → TCP 80 (HTTP), TCP 443 (HTTPS)
- DNS → UDP 53 (and sometimes TCP 53)
- SSH → TCP 22

Both TCP and UDP have 65,535 ports (1-65535). The goal of port scanning is to determine:

- Which ports are open
- Which are closed
- Which are filtered (blocked by firewall)

Nmap provides multiple scanning techniques to achieve this.

TCP Connect Scan (-sT)

The TCP Connect Scan attempts to complete the full TCP three-way handshake:

- 1.SYN → (from scanner)
- 2.SYN-ACK → (from target if open)
- 3.ACK → (from scanner)

If the handshake succeeds:

- The port is open
- Nmap immediately closes the connection (RST)

If the port is closed:

- Target responds with RST

This method relies on standard TCP behavior. Since a full connection is attempted:

- Open port → connection succeeds
- Closed port → RST reply

It is reliable because it uses the operating system's networking stack.

Advantages

- Does not require root privileges
- Very accurate
- Simple to understand

Disadvantages

- More easily logged
- Slower
- Noisy (full connection is established)

TCP SYN Scan (-sS) - “Stealth Scan”

Instead of completing the three-way handshake, it only sends:

1. SYN → (from scanner)
2. SYN-ACK → (if port open)

Then Nmap immediately sends:

- RST (to avoid completing connection)

Why It Is Called Stealth

Because:

- The connection is never fully established.
- Many systems log only completed connections.
- Generates less logging in some environments.

How It Identifies Port Status

Target Response	Meaning
SYN-ACK	Port is open
RST	Port is closed
No response / ICMP error	Port filtered

Advantages

- Faster than connect scan
- Less likely to be logged
- Requires root privileges
- Most commonly used TCP scan

UDP Scan (-sU)

Why Scan UDP ?

Many important services use UDP :

- DNS (53)
- DHCP (67/68)
- NTP (123)
- SNMP (161)
- VoIP

Unlike TCP, UDP :

- Has no handshake
- Is connectionless

Nmap sends a UDP packet to a port.

Possible outcomes:

Target Response	Meaning
ICMP Port Unreachable	Port is closed
No response	Possibly open or filtered
UDP reply	Port is open

Why UDP Scanning Is Slower

- No handshake
- No guaranteed response
- Requires waiting for timeouts
- Firewall behavior complicates results

UDP scans are typically slower and less precise than TCP scans.

Controlling Which Ports to Scan

By default:

- Nmap scans the 1000 most common ports

This is a balance between speed and coverage.

However, you may want more control.

Fast Mode (-F)

Scans:

- Top 100 most common ports

Why Use It?

- Quick reconnaissance
- Limited time
- Reduce noise

Specify Port Range (-p)

Examples :

- -p10-1024 : Scans ports 10 through 1024
- -p1-25 : Scans ports 1 through 25
- -p- : Scans all 65,535 ports

Well-Known Ports (1-1023)

These are reserved for core services:

- 22 → SSH
- 25 → SMTP
- 80 → HTTP
- 443 → HTTPS

Command : -p1-1023

Why Scan Well-Known Ports First?

- Most common services run here
- Faster initial assessment
- High probability of findings

Summary of Key Options

Option	Purpose
-sT	TCP Connect Scan (full handshake)
-sS	TCP SYN Scan (stealth, half-open)
-sU	UDP Scan
-F	Scan top 100 ports
-p[range]	Specify port range
-p-	Scan all 65,535 ports

Practical Scanning Strategy

1. Discover live hosts :

```
nmap -sn target_range
```

2. Perform quick scan :

```
nmap -sS -F target_ip
```

3. Scan well-known ports :

```
nmap -sS -p1-1023 target_ip
```

4. Perform full scan if necessary :

```
nmap -sS -p- target_ip
```

5. Scan UDP if required :

```
nmap -sU target_ip
```

Advanced Nmap Scanning: OS Detection, Service Enumeration & Performance Control

After discovering open ports, the next step in professional network reconnaissance is answering deeper questions:

- What operating system is running?
- What exact service versions are installed?
- How can we control scan speed and detection risk?

All of this can be achieved using advanced options in Nmap.

OS Detection (-O)

The -O option enables Operating System detection.

```
nmap -sS -O target_ip
```

How It Works

Nmap does not log into the system to read its OS. Instead, it:

- Sends specially crafted TCP/IP packets.
- Analyzes how the system responds.
- Compares responses against its OS fingerprint database.

Different operating systems:

- Handle TCP flags differently
- Set TTL values differently
- Structure responses uniquely

By analyzing these subtle differences, Nmap makes an educated guess.

Important Concept

OS detection is not perfectly accurate.

Why ?

- Firewalls modify packets.
- NAT devices alter responses.
- Some systems are hardened to mask fingerprinting.

Therefore, results are always probabilistic (e.g., Linux 4.x or 5.x).

Why OS Detection Matters

Knowing the OS helps:

- Identify potential vulnerabilities
- Select correct exploits
- Choose appropriate attack techniques
- Understand system architecture

Without OS detection, you are operating blindly.

Service & Version Detection (-sV)

Purpose

The -sV option identifies:

- What service is running
- The exact version of that service

`nmap -sS -sV target_ip`

How It Works

Nmap connects to open ports and:

- Sends service-specific probes
- Reads banners or protocol responses
- Matches responses to its version database

Example output :

```
22/tcp open  ssh    OpenSSH 8.9p1 Ubuntu 3ubuntu0.10
```

This tells us:

- Service: SSH
- Software: OpenSSH
- Exact version
- Operating system context (Ubuntu)

Why Version Detection Is Critical

Because vulnerabilities are version-specific.

Example:

- OpenSSH 8.9 → May be secure
- OpenSSH 6.x → Likely vulnerable

Without version detection:

- You only know the door is open
- You don't know what is behind it

Aggressive Scan (-A)

```
nmap -A target_ip
```

The -A option combines:

- OS detection (-O)
- Version detection (-sV)
- Traceroute
- Script scanning (default scripts)

Why Use It ?

- Quick comprehensive scan
- Saves time typing
- Good for lab environments

Warning

It is :

- Noisy
- Easily detected
- Not stealthy

Best used when stealth is not required.

Forcing a Scan (-Pn)

During host discovery :

- If a host does not respond to ping or probes,
- Nmap marks it as "down"
- Port scan is skipped.

This may happen if :

- ICMP is blocked
- Firewall filters probes

Solution

```
nmap -Pn target_ip
```

This tells Nmap :

“Assume the host is up. Scan it anyway.”

Why This Is Important

Some secure systems :

- Do not respond to ping
- Block discovery probes

Without -Pn, you might completely miss a live system.

Controlling Scan Speed (Timing Templates)

Nmap provides six timing levels :

Template	Number	Speed	Purpose
Paranoid	T0	Very slow	Avoid IDS detection
Sneaky	T1	Very slow	Low detection risk
Polite	T2	Slower	Reduce bandwidth usage
Normal	T3	Default	Balanced
Aggressive	T4	Fast	Quick scans
Insane	T5	Very fast	Extremely aggressive

Example : `nmap -sS -T4 target_ip`

Why Timing Matters

Fast scans :

- Generate many packets quickly
- Trigger IDS/IPS alerts
- Can overwhelm weak systems

Slow scans :

- Avoid detection
- Reduce noise
- Take significantly longer

Example comparison (lab case) :

- T0 → 9.8 hours
- T1 → 27 minutes
- T2 → 40 seconds
- T3 → 0.15 seconds
- T4 → 0.13 seconds

Real-world results vary by network quality.

Controlling Parallelism

Options :

`--min-parallelism`

`--max-parallelism`

These control :

- How many probes are sent simultaneously.

Why It Matters

High parallelism :

- Faster scans
- More network load

Low parallelism :

- Slower scans
- Less suspicious
- Better for unstable networks

Nmap automatically adjusts this unless overridden.

Controlling Packet Rate

Options:

`--min-rate`

`--max-rate`

Controls:

- Packets per second (PPS)

Example : `--max-rate 50`

Limits scan to 50 packets per second.

Why Use It ?

- Avoid IDS detection
- Prevent network congestion
- Maintain stealth

Important : Rate applies to the entire scan, not per host.

Host Timeout

Option :

`--host-timeout <time>`

Example :

`--host-timeout 30s`

This means:

- If a host takes longer than 30 seconds,
- Nmap stops scanning it.

Why It Is Useful

Some hosts:

- Are extremely slow
- Drop packets
- Cause long scan delays

Host timeout prevents:

- One slow system from delaying the entire scan.

Summary of Advanced Options

Option	Purpose
-O	OS detection
-sV	Service & version detection
-A	Combined advanced scan
-Pn	Skip host discovery
-T0-T5	Timing control
--min/max-parallelism	Control simultaneous probes
--min/max-rate	Control packets per second
--host-timeout	Limit scan time per host

Nmap Output Control & Reporting

When running scans with Nmap, especially large or slow ones, two practical needs arise:

1. Seeing what is happening in real time
2. Saving scan results in a useful format

Understanding these features makes scanning more professional, controlled, and efficient.

Verbosity (-v) - Seeing More During a Scan

The Problem

Sometimes :

- A scan takes a long time.
- No output appears for several seconds or minutes.
- You are unsure if the scan is stuck or progressing.

By default, Nmap only shows final results and minimal progress updates.

The Solution : -v

```
nmap target -v
```

The -v option increases verbosity, meaning :

Nmap shows more details about what it is doing while the scan runs.

What You See with -v

With verbosity enabled, Nmap displays :

- When host discovery starts (e.g., ARP Ping Scan)
- When DNS resolution begins
- When port scanning starts
- When open ports are discovered
- Packet statistics
- Stage transitions

This helps you understand:

- Scan progress
- Scan phases
- Network responsiveness
- Whether the scan is functioning correctly

Increasing Verbosity

You can increase detail levels :

- -v → More output
- -vv → Even more
- -vvvv → Very detailed

Or specify directly :

-v2

-v4

You can even press **v** while the scan is running to increase verbosity live.

Why Verbosity Is Important

- Helps beginners understand scan phases
- Helps professionals monitor large scans
- Useful for troubleshooting slow scans
- Shows discovered open ports immediately

Debugging Mode (-d)

If verbosity is not enough, use : -d

This enables debug-level output.

You can increase debugging intensity :

- -d
- -dd
- -d9 (maximum level)

What Debugging Shows

- Internal decision-making
- Packet construction details
- Timing adjustments
- Scan logic
- Network behavior handling

Warning :

-d9 produces massive output. Use it only when diagnosing serious issues.

Saving Scan Results

In professional environments, scan results must often be:

- Documented
- Shared
- Automated
- Parsed by scripts

Nmap supports multiple output formats.

Normal Output (-oN)

`-oN filename`

Example : `nmap target -oN report.txt`

Produces:

- Human-readable output
- Same format as screen display
- Best for manual review

XML Output (-oX)

`-oX filename`

Produces :

- Structured XML format
- Machine-readable
- Used for importing into tools

Useful for:

- Security dashboards
- Vulnerability scanners
- Automation pipelines

Grepable Output (-oG)

`-oG filename`

Designed for :

- Command-line processing
- Tools like grep and awk

Useful for :

- Extracting specific data
- Filtering open ports quickly

All Formats at Once (-oA)

`-oA basename`

Example : `nmap target -oA scan_results`

Creates :

- `scan_results.nmap` → Normal
- `scan_results.xml` → XML
- `scan_results.gnmap` → Grepable

This is the most practical option in real-world work.

Why Root Privileges Matter

Running Nmap with sudo unlocks full capabilities.

Without root:

- Nmap defaults to -sT (connect scan)
- Cannot craft raw SYN packets
- Cannot perform some advanced techniques

With root:

- Default scan becomes -sS (SYN scan)
- Full packet crafting is allowed
- Maximum control and accuracy

Reason:

Crafting raw packets (like TCP SYN without full connection) requires elevated privileges.

Complete Feature Overview

Host Discovery

- -sL – List targets only (no actual scanning performed)
- -sn – Discover live hosts only (no port scanning)

Port Scanning

- -sT – TCP Connect scan (completes full three-way handshake)
- -sS – TCP SYN scan (stealth / half-open scan)
- -sU – UDP scan
- -F – Fast mode (scan top 100 most common ports)

- **-p[range]** – Specify port range (e.g., -p1-1024, -p- for all ports)
- **-Pn** – Skip host discovery and treat hosts as online

Service Detection

- **-O** – Operating System detection
- **-sV** – Service and version detection
- **-A** – Aggressive scan (includes OS detection, version detection, traceroute, and more)

Timing Control

- **-T0-T5** – Timing templates (0 = slowest/paranoid, 5 = fastest/insane)
- **--min-parallelism / --max-parallelism** – Control number of simultaneous probes
- **--min-rate / --max-rate** – Control packet sending rate (packets per second)
- **--host-timeout** – Set maximum time to wait for a host

Real-Time Output

- **-v** – Increase verbosity (more scan progress details)
- **-d** – Enable debugging output (internal scan details)

Report Saving

- **-oN** – Save in normal (human-readable) format
- **-oX** – Save in XML format (machine-readable)
- **-oG** – Save in grepable format (for command-line processing)
- **-oA** – Save in all major formats (normal, XML, grepable)

Section 06 : Cryptography

Room 01 : Cryptography Basics

Cryptography is the practice and study of techniques used to secure communication and protect data in environments where adversaries may be present.

It ensures that :

- Unauthorized parties cannot read the data (Confidentiality)
- Unauthorized parties cannot modify the data (Integrity)
- You can verify the identity of the communicating party (Authenticity)

Why Cryptography Exists

In a networked world, devices communicate across untrusted environments such as the internet. The internet was not originally designed with security as its primary goal. Without cryptography :

- Attackers could intercept sensitive data.
- Data could be modified in transit.
- Users could be redirected to fake servers.
- Digital trust would collapse.

Therefore, cryptography provides the trust layer on top of communication protocols.

Networking allows systems to communicate.

Cryptography allows them to communicate securely and reliably.

The Core Goal of Cryptography

The ultimate purpose of cryptography is :

Secure communication in the presence of adversaries.

An adversary may attempt to :

- Eavesdrop (intercept communication)
- Modify data
- Impersonate a legitimate server
- Replay captured data

Cryptography protects against these risks by ensuring:

Confidentiality

Only authorized parties can access the information.

Why is this important ?

If login credentials, credit card details, or medical records are transmitted without encryption, attackers could steal them.

Confidentiality is typically achieved through encryption.

Integrity

Ensures that data has not been altered during transmission or storage.

Why is this important ?

An attacker might change :

- A bank transfer amount
- A software download file
- A configuration file

Integrity guarantees that the data received is exactly the same as the data sent.

Integrity is commonly achieved through cryptographic hashes and message authentication codes.

Authenticity

Ensures that you are communicating with the legitimate party and not an impostor.

Why is this important ?

Without authenticity :

- Attackers could create fake banking websites.
- Man-in-the-middle attacks could intercept traffic.
- Users might unknowingly send sensitive data to attackers.

Authenticity is often verified using digital certificates and public-key cryptography.

Real-World Examples of Cryptography in Daily Use

- Logging into Websites: Usernames and passwords are encrypted before transmission, protecting credentials from attackers on public networks.
- Secure Shell (SSH): Creates an encrypted tunnel so commands and responses remain private during remote server access.
- Online Banking: Digital certificates verify the bank's identity, and encrypted connections prevent impersonation or data theft.
- File Downloads & Hashing: Hashes confirm files are authentic and untampered after download.

Encryption States

- Data in Motion (In Transit): Information sent across networks (e.g., logins, money transfers) is protected with encrypted channels like TLS, since networks are untrusted.
- Data at Rest: Stored data (e.g., credit card records, medical files) is secured with disk, database, or backup encryption, ensuring protection even if devices are stolen.

Legal and Regulatory Requirements

Cryptography is not only a best practice — in many industries, it is a legal requirement.

PCI DSS (Payment Card Industry Data Security Standard)

Applies to organizations handling credit card information.

Requires:

- Encryption of cardholder data
- Protection during storage (at rest)
- Protection during transmission (in motion)

Why ?

Because financial data is highly sensitive and attractive to attackers.

Failure to comply can result in:

- Heavy fines
- Loss of payment processing ability
- Reputational damage

Healthcare Regulations

Handling medical records requires compliance with strict regulations.

Examples include :

- Health Insurance Portability and Accountability Act (HIPAA)
- Health Information Technology for Economic and Clinical Health Act (HITECH)
- General Data Protection Regulation (GDPR)
- Data Protection Act 2018 (DPA)

These regulations require :

- Protection of personal data

- Encryption where appropriate
- Secure storage and transmission
- Protection against unauthorized disclosure

Why are such laws necessary ?

Because medical and personal data:

- Can be exploited for identity theft
- May cause severe personal harm if exposed
- Must maintain patient privacy

Core Encryption Concepts

- Plaintext: Original readable data.
- Cipher: Public algorithm for transforming data.
- Key: Secret value controlling encryption/decryption.
- Encryption: Plaintext → Ciphertext using cipher + key.
- Ciphertext: Scrambled, unreadable output.
- Decryption: Ciphertext → Plaintext with correct key.
- Security = *infeasible* to break without the key.

Historical Context : Cryptography is ancient; simple methods evolved into modern systems.

Caesar Cipher

- Shifts letters by fixed positions (mod 26).
- Example: Key = 3 → "TRYHACKME" → "WUBKDFNPH".
- Decryption = reverse shift.
- Weakness: Only 25 possible keys → easy brute force.

Lessons

- Algorithm + key are both required.
- Small keyspaces = insecure.
- Public algorithms are fine; secrecy lies in the key.
- Modern cryptography uses math (number theory, modular arithmetic).

Other Classical Ciphers (*details not included*)

- Vigenère: Multiple Caesar shifts via keyword.
- Enigma: WWII rotor machine, complex but broken.
- One-time pad: Perfect secrecy, impractical key management.

The Two Main Categories of Encryption

Modern encryption techniques fall into two primary categories:

1. Symmetric Encryption
2. Asymmetric Encryption

Understanding the differences between them is essential because each solves different security problems.

Symmetric Encryption (Private-Key Cryptography)

Symmetric encryption uses the same key for both :

- Encryption
- Decryption

If a message is encrypted with a specific key, it must be decrypted using that exact same key.

Because the same key is shared, this method is also called:

- Private-key cryptography
- Shared-key cryptography

How It Works

Let's imagine two people (traditionally called Alice and Bob in cryptography):

1. Alice encrypts a message using a secret key.
2. She sends the ciphertext to Bob.
3. Bob uses the same secret key to decrypt the message.

Mathematically:

`Ciphertext = Encrypt(Plaintext, Key)`

`Plaintext = Decrypt(Ciphertext, Key)`

The security entirely depends on keeping the key secret.

The Key Distribution Problem

The biggest challenge in symmetric encryption is:

How do we securely share the secret key?

Consider a practical example:

- You create a password-protected document.
- You email the encrypted document to your colleague.
- You cannot email the password in the same inbox.

Why?

If an attacker compromises the mailbox, they gain:

- The encrypted document
- The password

At that point, encryption becomes useless.

Secure Key Sharing Options

- Meeting in person
- Using a separate secure communication channel
- Using asymmetric encryption to exchange the key (common in modern systems)

When there are many recipients, the problem becomes worse:

- Each recipient must securely receive the key.
- The more people involved, the higher the risk of key leakage.
- In environments like industrial espionage, this risk becomes severe.

Examples of Symmetric Encryption

1 Data Encryption Standard (DES)

- Adopted in 1977.
- Uses a 56-bit key.
- Once considered secure.

Problem:

By 1999, advances in computing allowed DES to be broken in less than 24 hours.

Lesson : Increasing computing power reduces security over time.

2 Triple DES (3DES)

- Applies DES three times.
- Uses a 168-bit key.
- Effective security ~112 bits.
- Deprecated in 2019.

It was a temporary fix when DES became insecure.

3 Advanced Encryption Standard (AES)

- Adopted in 2001.
- Key sizes: 128, 192, or 256 bits.
- Widely used today.
- Considered secure when implemented correctly.

AES is currently the global standard for symmetric encryption.

Why Symmetric Encryption Is Still Important

Despite key-sharing challenges, symmetric encryption has major advantages:

- Very fast
- Efficient for large data
- Used for disk encryption, VPNs, TLS sessions, file encryption

In practice :

Asymmetric encryption is often used to exchange keys, and symmetric encryption is used for actual data transmission.

Asymmetric Encryption (Public-Key Cryptography)

Asymmetric encryption uses two different keys :

- Public key (shared openly)
- Private key (kept secret)

These keys are mathematically related.

How It Works

1. Bob generates:

- A public key
- A private key

2. Bob shares the public key with everyone.

3. Alice encrypts her message using Bob's public key.

4. Only Bob can decrypt it using his private key.

Important rule:

Data encrypted with the public key can only be decrypted with the private key.

Why Is This Powerful ?

It solves the key distribution problem.

- No need to secretly share encryption keys.
- Anyone can encrypt messages for Bob.
- Only Bob can decrypt them.

However:

Bob must protect his private key at all costs.

Examples of Asymmetric Encryption

RSA

- Common key sizes: 2048, 3072, 4096 bits
- 2048-bit is minimum recommended
- Based on difficulty of factoring large prime numbers

Diffie-Hellman

- Used for secure key exchange
- Minimum recommended key size: 2048 bits
- Allows two parties to generate a shared secret over an insecure channel

Elliptic Curve Cryptography (ECC)

- Uses elliptic curve mathematics
- Much shorter keys for same security level
- Example: 256-bit ECC $\approx$ 3072-bit RSA security

This makes ECC:

- Faster
- More efficient
- Suitable for mobile and IoT devices

Why Asymmetric Encryption Is Slower

Asymmetric encryption relies on:

- Mathematical problems that are easy to compute in one direction
- Extremely difficult to reverse

Examples:

- Prime factorization
- Discrete logarithm problem
- Elliptic curve discrete logarithm

These are called **one-way functions**.

They are:

- Easy to compute
- Practically infeasible to reverse (would take millions of years with current technology)

Because of complex mathematics, asymmetric encryption is computationally expensive.

Symmetric vs Asymmetric

Feature	Symmetric	Asymmetric
Keys Used	One shared key	Public + Private key pair
Speed	Fast	Slower
Key Distribution	Difficult	Easier
Typical Use	Encrypt bulk data	Key exchange, authentication
Security Depends On	Secret key	Private key secrecy

Modern systems combine both approaches.

Foundational Mathematics in Cryptography (Simplified)

XOR (Exclusive OR)

- Definition: A bitwise operation that outputs 1 if bits differ, 0 if they're the same.
- Truth Table :

A : 0 0 1 1

B : 0 1 0 1

$A \oplus B$: 0 1 1 0

- Properties:
 - $A \oplus A = 0$ (self-cancels)
 - $A \oplus 0 = A$ (identity)
 - Commutative & associative (order doesn't matter)
- Use in Encryption:
 - Encryption: $C = P \oplus K$
 - Decryption: $P = C \oplus K$
 - Works because $K \oplus K = 0$, so the key cancels itself out.
- Security Note: XOR is secure only if the key is random, as long as the plaintext, and never reused. Otherwise, attackers can exploit patterns.

Modulo

- Definition: Gives the remainder after division. Example: $23 \% 6 = 5$
- Why Important:
 - Keeps numbers within a fixed range (0 to $n-1$).
 - Creates “wrap-around” behavior, essential in modular arithmetic.
 - Used in RSA, Diffie-Hellman, ECC.
- Non-reversible:
 - Example: $x \% 5 = 4 \rightarrow x$ could be 4, 9, 14, 19... (infinite possibilities).
 - This property is crucial for asymmetric cryptography.
- Large Numbers:
 - Modern cryptography uses very large integers (hundreds/thousands of bits).
 - Large modulus values make brute-force attacks computationally infeasible.
 - Languages like Python support big integers, making them practical for crypto.

Room 02 : Public Key Cryptography Basics

To understand cyber security principles, consider a face-to-face meeting with a business partner discussing confidential information.

In such a setting, several natural security properties exist :

1 Authentication – Verifying Identity

When meeting someone physically:

- You see their face.
- You hear their voice.
- You recognize their mannerisms.

Because of these factors, you are confident about *who* you are speaking with.

Why Authentication Matters

In digital communication, identity cannot be verified visually. Anyone can:

- Create a fake account
- Send messages impersonating someone else
- Spoof email addresses

Authentication ensures :

You are communicating with the intended party — not an impersonator.

Without authentication, trust collapses entirely.

2 Authenticity – Verifying Message Origin

In person, you can distinguish :

- What your partner says
- What others in the room say

You know the words are coming from them.

Why Authenticity Matters

In cyberspace, messages travel through :

- Routers
- Servers
- Internet service providers
- Wireless networks

An attacker could:

- Inject fake messages
- Replace content mid-transmission

Authenticity guarantees:

The message truly originates from the claimed sender.

This prevents impersonation at the message level.

3 Integrity – Ensuring No Alteration

Across a table, words cannot be altered in transit. What your partner says reaches you unchanged.

Why Integrity Matters

Digital data:

- Is broken into packets
- Travels through multiple nodes
- Can be intercepted or modified

An attacker could:

- Change numbers in a bank transfer
- Modify a contract
- Insert malicious instructions

Integrity ensures:

The data received is exactly the data sent.

Even a one-bit change could have catastrophic consequences.

4 Confidentiality – Preventing Eavesdropping

In a coffee shop, you:

- Sit away from others
- Speak quietly

Only authorized participants hear the conversation.

Why Confidentiality Matters

Online communications travel through shared infrastructure. Without protection:

- Internet providers
- Hackers on public Wi-Fi
- Malicious insiders

could read everything.

Confidentiality ensures:

Only authorized parties can access the information.

This is typically achieved using encryption.

How Cryptography Solves These Problems

Cryptography provides mathematical techniques to enforce :

- Authentication
- Authenticity
- Integrity
- Confidentiality

Two major types of cryptography are used :

1 Symmetric Cryptography (Private Key)

- Same key is used for encryption and decryption.
- Fast and efficient.
- Best suited for encrypting large amounts of data.

Primary Strength

Confidentiality

Main Problem

Securely sharing the secret key.

If someone intercepts the key, security is lost.

2 Asymmetric Cryptography (Public Key)

Uses two keys:

- Public key (shared openly)
- Private key (kept secret)

What one key encrypts, the other decrypts.

Strengths

- Authentication
- Authenticity
- Integrity
- Secure key exchange

Weakness

Slower than symmetric encryption

Because of this, asymmetric cryptography is typically used only to:

- Exchange symmetric keys securely
- Authenticate parties
- Create digital signatures

After that, symmetric encryption handles the actual data.

Secure Key Exchange - The Lock and Box Analogy

The key challenge :

How do you share a secret key without exposing it?

Analogy

- Secret code → Symmetric key
- Lock → Public key
- Lock's key → Private key

Process:

1. Your friend gives you a lock (public key).
2. You place your secret inside a box.
3. You lock it using their lock.
4. Only your friend can unlock it using their private key.

Why This Works

- The public key can be shared safely.
- Only the private key can open what was locked.
- The symmetric key is never exposed in plaintext.

This allows:

- Secure symmetric key exchange
- Fast encrypted communication afterward

RSA - A Foundational Public Key Algorithm

RSA is named after its inventors: Rivest, Shamir, and Adleman.

RSA is a public-key encryption algorithm used for:

- Secure communication
- Digital signatures
- Key transport
- Authentication

Why RSA Is Secure

RSA security depends on :

The mathematical difficulty of factoring large numbers.

Easy Operation:

Multiplying two large prime numbers.

Hard Operation:

Given the product, finding the original primes.

For example:

- $113 \times 127 = 14351$
Multiplication is easy.

But:

- Given 14351, determining 113 and 127 is harder.

In real systems:

- Each prime is ~ 300 digits long.
- Their product exceeds 600 digits.

Factoring such numbers is computationally infeasible with current technology.

This asymmetry in difficulty creates security.

How RSA Works

Step 1: Key Generation

1. Choose two large prime numbers, p and q .
2. Compute $n = p \times q$.
 - n is the modulus used for both encryption and decryption.
3. Compute Euler's totient function $\phi(n) = (p - 1)(q - 1)$.
4. Choose an integer e such that:
 - $1 < e < \phi(n)$
 - $\gcd(e, \phi(n)) = 1$ (e is coprime with $\phi(n)$)
 - e is the public exponent.
5. Compute d as the modular inverse of e modulo $\phi(n)$:
 - $d \times e \equiv 1 \pmod{\phi(n)}$
 - d is the private exponent.

Public key = (n, e) Private key = (n, d)

Step 2: Encryption

- Suppose the message is m (where $0 \leq m < n$).
- Compute ciphertext: $c = m^e \bmod n$

Step 3: Decryption

- To recover the message, compute: $m = c^d \bmod n$

Why This Works

- By construction, $e \times d \equiv 1 \pmod{\phi(n)}$.
- That means $e \times d = 1 + k\phi(n)$ for some integer k .
- So during decryption: $c^d = (m^e)^d = m^{ed} = m^{(1 + k\phi(n))}$.
- If $\gcd(m, n) = 1$, Euler's theorem guarantees: $m^{\phi(n)} \equiv 1 \pmod{n}$.
- Therefore: $m^{(1 + k\phi(n))} \equiv m \times (m^{\phi(n)})^k \equiv m \times 1^k \equiv m \pmod{n}$.
- Thus, the decrypted message equals the original message.

Important Notes

- The correctness proof assumes m is coprime with n .
- In practice, even if m shares a factor with n , RSA implementations handle this edge case correctly.
- Security of RSA relies not on Euler's theorem itself, but on the difficulty of factoring n into p and q .

Why Modular Arithmetic Is Used

The modulo operation :

- Keeps numbers within manageable size
- Maintains reversible properties
- Enables mathematical guarantees of correctness

Without modular arithmetic :

- Numbers would grow astronomically large
- Computation would be impractical

RSA in Security Practice

RSA is widely used in:

- TLS/HTTPS
- Digital certificates
- Digital signatures
- Email encryption

In security competitions (CTFs), RSA problems often require understanding :

- p (prime 1)
- q (prime 2)
- $n = p \times q$
- e (public exponent)
- d (private exponent)
- m (plaintext)
- c (ciphertext)

If p and q are small or poorly chosen, RSA can be broken.

Diffie-Hellman Key Exchange

One major problem remains :

How can two parties create a shared key without ever sending it ?

The solution is Diffie-Hellman.

What Diffie-Hellman Does

Whitfield Diffie
Martin Hellman

Diffie-Hellman allows two parties to:

- Create a shared secret
- Over an insecure channel
- Without prior shared keys

Why It Works

It relies on :

The difficulty of the discrete logarithm problem.

Exponentiation modulo a prime is easy.

Reversing it (finding the exponent) is extremely difficult.

Step-by-Step Process

1. Public agreement:
 - Prime number p
 - Generator g
2. Private secrets:
 - Alice chooses a
 - Bob chooses b
3. Public values:
 - Alice sends $A = g^a \bmod p$
 - Bob sends $B = g^b \bmod p$
4. Shared secret:
 - Alice computes $B^a \bmod p$
 - Bob computes $A^b \bmod p$

Both compute :

$$g^{(ab)} \bmod p$$

Even if an attacker sees:

- p
- g
- A
- B

They cannot compute the shared secret without solving a discrete logarithm problem.

Why Diffie-Hellman Is Important

It allows :

Secure key establishment

No need to transmit secret directly

Secure symmetric encryption afterward

Combining RSA and Diffie-Hellman

In real-world protocols :

- Diffie-Hellman → Establishes shared symmetric key
- RSA → Provides authentication and digital signatures

This prevents :

- Man-in-the-middle attacks
- Impersonation
- Key interception

Together, they provide :

- Confidentiality
- Authentication
- Integrity
- Authenticity

Why Modern Systems Use Hybrid Cryptography

Modern secure communication (e.g., HTTPS) works as follows:

1. Authenticate server using RSA and certificates
2. Perform Diffie-Hellman key exchange
3. Generate symmetric session key
4. Encrypt all further traffic using symmetric encryption

Why Hybrid Approach?

Because:

- Symmetric encryption is fast
- Asymmetric encryption solves trust and key distribution problems
- Combining both gives speed + security

Authenticating the Server (SSH Example)

When connecting to a server using SSH for the first time, you typically see a warning like:

“The authenticity of host can't be established...
Are you sure you want to continue connecting?”

This message is not an error — it is a security feature.

When you initiate an SSH connection:

1. The server sends its public key.
2. Your SSH client checks whether it has seen this key before.
3. If not, it displays the server's public key fingerprint and asks for confirmation.

Why Is This Important ?

This step protects against Man-in-the-Middle (MITM) attacks.

What Is a MITM Attack?

An attacker intercepts your connection and pretends to be the legitimate server. If SSH did not verify the server's identity:

- You might unknowingly send credentials to the attacker.
- You could execute commands on a malicious server.

The first-time verification ensures:

You consciously trust the server's identity.

Once you confirm the fingerprint:

- It is stored in `~/.ssh/known_hosts`.
- Future connections will fail if the server presents a different key.

This prevents silent impersonation later.

Authenticating the Client (Proving Who You Are)

After confirming the server's identity, the server must authenticate you.

There are two common methods:

1 Password Authentication

You provide:

- Username
- Password

Why Passwords Are Weak

- Can be guessed.
- Can be brute-forced.
- Can be reused across services.
- Can be phished.

For these reasons, password-based SSH authentication is discouraged for sensitive systems.

2 Key-Based Authentication (Best Practice)

Instead of passwords, SSH can use public/private key pairs.

By default, many systems use RSA, but modern systems prefer Ed25519.

How It Works

1. You generate a key pair using `ssh-keygen`.
2. Your public key is copied to the server.
3. Your private key remains on your machine.
4. During login:
 - The server challenges you.
 - Your private key proves your identity.
 - The server verifies it using your public key.

The private key is never transmitted.

Public vs Private Keys

After generating a key pair:

- `id_ed25519.pub` → Public key (share this)
- `id_ed25519` → Private key (never share)

Why Private Keys Must Remain Private

Anyone with your private key:

- Can authenticate as you
- Can log into servers that trust it
- Can impersonate you

It is equivalent to giving someone your password — except worse.

What Does a Passphrase Do?

When generating a key, you can add a passphrase.

Important distinction:

The passphrase protects your private key locally.

It does NOT:

- Get sent to the server
- Authenticate you to the server

It only encrypts your private key file on disk.

If someone steals your private key file:

- Without a passphrase → Immediate compromise.
- With a passphrase → They must crack it first.

Tools like John the Ripper can attempt cracking weak passphrases.

SSH Keys in Offensive Security

During penetration testing and CTFs:

- Reverse shells are unstable.
- SSH access is stable and interactive.
- Placing a public key in `authorized_keys` creates persistent access.

However, in real-world systems:

- This would be considered a backdoor.
- Proper key management and auditing are required.

What Is a Digital Signature ?

A digital signature:

- Proves who signed the document.
- Proves the document was not modified.

It uses asymmetric cryptography.

How Digital Signatures Work ?

Basic conceptual model:

1. Create a hash of the document.
2. Encrypt the hash using your private key.
3. Share:
 - The document
 - The encrypted hash (signature)

The recipient:

1. Decrypts the signature using your public key.
2. Computes the hash of the received document.
3. Compares the two hashes.

If they match:

- Integrity confirmed.
- Authenticity confirmed.

Why Hashing Is Used Instead of Signing the Whole Document

- Encrypting large documents is inefficient.
- Hashes are fixed-length summaries.
- Small change in file → completely different hash.

This makes tampering detectable.

Certificates and HTTPS

How does your browser know a website is legitimate?

For example, how does it know the real identity of : TryHackMe

The answer : Digital Certificates.

What Is a Certificate ?

A certificate:

- Contains a public key.
- States the identity of its owner.
- Is digitally signed by a trusted authority.

Certificate Authorities (CAs)

A CA is a trusted organization that verifies identities.

Examples include:

- Global certificate providers trusted by browsers.

Your browser ships with:

- A list of trusted root CAs.
- Known as the “Root Store”

Chain of Trust

The trust model works like this:

1. Root CA (trusted by your browser)
2. Intermediate CA (trusted by Root)
3. Website certificate (trusted by Intermediate)

If the chain validates:

- Your browser trusts the website.

Getting Certificates

Website owners can :

- Purchase from commercial CAs.
- Use free services like : Let's Encrypt

Modern websites must use HTTPS because:

- It encrypts traffic.
- It ensures integrity.
- It verifies identity.

OpenPGP, GPG, and Secure Communication

What Is PGP ?

Pretty Good Privacy

PGP is software for :

- Encryption
- Digital signing
- Secure communication

The open-source implementation is : GnuPG

What Can GPG Do?

- Encrypt files
- Sign files
- Encrypt and sign emails
- Verify signatures

Key Generation Process

When generating a GPG key :

You select :

- Algorithm (RSA or ECC)
- Curve (e.g., Curve25519)
- Expiry date
- User ID (name + email)

This creates :

- Public key
- Private key

Why Expiry Dates Matter

Keys may expire to :

- Limit damage if compromised.
- Encourage rotation.
- Enforce good security hygiene.

Using GPG in Practice

To decrypt : `gpg --decrypt file.gpg`

To import a key : `gpg --import backup.key`

Key backup is critical:

- Loss of private key = permanent loss of encrypted data.

Attacking Cryptography

Understanding cryptography also requires understanding attacks.

Cryptography : The science of securing communication.

Cryptanalysis : The science of breaking cryptographic systems.

Brute-Force Attack

Try every possible password.

Effective only when:

- Password space is small.

Dictionary Attack

Try:

- Common passwords
- Dictionary words
- Likely combinations

More efficient than brute-force for human-generated passwords.

[Note that certain rooms are not included in this notes, as they are either straightforward to understand or primarily practice-based rather than theory focused.

The list covers theory related to Hashing, Web app basics, JavaScript, SQL, Shell, Digital Forensics and Incident response (DFIR), SIEM, IDS/IPS, Firewall and an intro to OWASP TOP 10.

and the following tools – Metasploit, Burp Suite, Hydra, Gobuster, SQLMap, CyberChef, CAPA, REMnux, FlareVM.]

End of File